SKRIPSI

IMPLEMENTASI *SOFT ACTOR-CRITIC* BERBASIS *Q-SWITCH MIXTURE OF PRIMITIVES HINDSIGHT EXPERIENCE REPLAY* PADA SISTEM KENDALI ROBOT MANIPULATOR UNTUK TUGAS PENYISIPAN OBJEK



MUJADID FAUZAN SALIM TAMIN
NIM. 163221057

PROGRAM SARJANA
TEKNIK ROBOTIKA DAN KECERDASAN BUATAN
DEPARTEMEN TEKNIK
FAKULTAS TEKNOLOGI MAJU DAN MULTIDISIPLIN
UNIVERSITAS AIRLANGGA
2026

LEMBAR PENGESAHAN

IMPLEMENTASI *SOFT ACTOR-CRITIC* BERBASIS *Q-SWITCH MIXTURE OF PRIMITIVES HINDSIGHT EXPERIENCE REPLAY* PADA SISTEM KENDALI ROBOT MANIPULATOR UNTUK TUGAS PENYISIPAN OBJEK

SKRIPSI

Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
Fakultas Teknologi Maju Dan Multidisiplin
Universitas Airlangga

Nama	: Mujadid Fauzan Salim Tamin
NIM	: 163221057
Tanggal Sidang Skripsi	: 25 Juni 2026

Disetujui oleh:

Pembimbing I	Pembimbing II
Dr. Eng. Ananta Adhi Wardana, S.T., M.Eng	**Yutika Amelia Effendi, S.Kom., M.Kom., Ph.D**
NIP. 199102162022103101	**NIP. 199404142018083201**

Mengetahui,
Koordinator Program Studi
S1 Teknik Robotika dan Kecerdasan Buatan

Ir. Asif Ali Zamzami, S.ST., M.Sc., Ph.D., IPM.
NIP. 199207222022103101

PERNYATAAN ORISINALITAS SKRIPSI

Saya, Mujadid Fauzan Salim Tamin, 163221057, penulis Skripsi yang berjudul **Implementasi *Soft Actor-Critic* Berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* pada Sistem Kendali Robot Manipulator untuk Tugas Penyisipan Objek** menyatakan bahwa:

1. Skripsi ini adalah asli dan benar-benar hasil karya saya sendiri, bukan hasil karya pihak lain dengan mengatasnamakan saya, bukan merupakan hasil tiruan atau jiplakan (*plagiarism*) dari karya pihak lain, dan/atau bukan tulisan yang dibuat dengan bantuan kecerdasan buatan.
2. Skripsi ini belum pernah diajukan untuk mendapatkan gelar akademik, baik di Universitas Airlangga, maupun di perguruan tinggi lainnya.
3. Dalam Skripsi ini tidak terdapat karya atau pendapat yang telah ditulis atau dipublikasikan orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dengan disebutkan nama pengarang dan dicantumkan dalam daftar kepustakaan.

Pernyataan ini saya buat dengan sebenar-benarnya, dan apabila di kemudian hari terdapat penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya tulis Skripsi ini, serta sanksi-sanksi lainnya sesuai dengan norma dan peraturan yang berlaku di Universitas Airlangga.

Surabaya, ...

Mujadid Fauzan Salim Tamin
NIM. 163221057

KETENTUAN PENGGUNAAN SKRIPSI

Ketentuan hak cipta bagi skripsi yang tidak dipublikasikan, terdaftar, tersedia, serta terbuka untuk umum di Perpustakaan Universitas Airlangga, dimiliki penulis dengan mengikuti aturan HKI yang berlaku di Universitas Airlangga. Referensi kepustakaan diperkenankan dicatat, tetapi pengutipan atau peringkasan hanya dapat dilakukan dengan seizin penulis dan harus disitasi sesuai dengan kaidah ilmiah. Memperbanyak atau menerbitkan sebagian atau seluruh skripsi haruslah seizin penulis.

Sitasi Skripsi ini dapat ditulis sebagai berikut:

Tamin, M. F. S. (2026). Implementasi *Soft Actor-Critic* Berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* pada Sistem Kendali Robot Manipulator untuk Tugas Penyisipan Objek. Skripsi. Surabaya: Universitas Airlangga.

Tamin, M. F. S. (2026). *Implementation of Soft Actor-Critic Based on Q-Switch Mixture of Primitives Hindsight Experience Replay in a Robot Manipulator Control System for Object Insertion Tasks. Undergraduate Thesis*. Surabaya: Universitas Airlangga.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah Subhanahu wa Ta'ala atas segala rahmat, taufik, dan hidayah-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul "Implementasi *Soft Actor-Critic* Berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* pada Sistem Kendali Robot Manipulator untuk Tugas Penyisipan Objek". Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana pada Program Studi Teknik Robotika dan Kecerdasan Buatan, Fakultas Teknologi Maju dan Multidisiplin, Universitas Airlangga. Penulis menyadari bahwa penyusunan skripsi ini tidak akan terselesaikan tanpa bantuan, bimbingan, dan dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan rasa terima kasih yang sebesar-besarnya kepada:

1. Keluarga dan kedua orang tua penulis tercinta yang selalu memberikan doa, dukungan, serta kasih sayang tanpa henti sehingga penulis dapat menyelesaikan studi dengan baik.
2. Bapak Dr. Eng. Ananta Adhi Wardana, S.T., M.Eng dan Ibu Yutika Amelia Effendi, S.Kom., M.Kom., Ph.D selaku dosen pembimbing yang telah memberikan arahan, bimbingan, serta motivasi yang sangat berharga selama proses penyusunan skripsi ini.
3. Bapak Rizki Putra Prastio, S.Si., M.T., selaku dosen wali yang senantiasa memberikan bimbingan, nasihat, dan perhatian selama masa studi.
4. Seluruh dosen Program Studi Teknik Robotika dan Kecerdasan Buatan, yang telah berbagi ilmu, wawasan, serta pengalaman yang sangat bermanfaat bagi penulis.
5. Wanda Desi Rahmawati, yang dengan ketulusan hati dan kesabaran selalu hadir di setiap langkah perjalanan ini, terima kasih atas doa, semangat, serta cahaya dukungan yang tak pernah padam yang menjadi sumber kekuatan bagi penulis hingga lembar terakhir skripsi ini terselesaikan.
6. Teman-teman seperjuangan serta semua pihak yang tidak dapat disebutkan satu per satu, yang telah memberikan bantuan, semangat, dan kebersamaan selama proses penelitian dan penyusunan skripsi ini.

Penulis menyadari sepenuhnya bahwa skripsi ini masih jauh dari kesempurnaan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi penyempurnaan di masa mendatang.

Surabaya, 6 Juni 2026
Penulis

ABSTRAK

IMPLEMENTASI *SOFT ACTOR-CRITIC* BERBASIS *Q-SWITCH MIXTURE OF PRIMITIVES HINDSIGHT EXPERIENCE REPLAY* PADA SISTEM KENDALI ROBOT MANIPULATOR UNTUK TUGAS PENYISIPAN OBJEK

Oleh
MUJADID FAUZAN SALIM TAMIN
NIM: 163221057
Program Sarjana Teknik Robotika dan Kecerdasan Buatan

Tugas penyisipan objek pada robot manipulator membutuhkan kendali yang presisi karena robot harus mampu menggenggam, memindahkan, menyelaraskan, dan menempatkan objek ke lubang target dengan toleransi kecil. Penelitian ini bertujuan untuk mengimplementasikan dan menganalisis performa algoritma *Soft Actor-Critic* (SAC) berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* (QMP-HER) pada sistem kendali robot manipulator untuk tugas penyisipan objek. Objek uji yang digunakan terdiri atas kubus, silinder, dan prisma segitiga dengan dimensi karakteristik 30 mm dan toleransi lubang sebesar 5 mm. Pelatihan dilakukan pada lingkungan simulasi MuJoCo menggunakan dua *primitive policy*, yaitu *grasp policy* dan *insert policy*, yang kemudian digunakan sebagai sumber kandidat aksi pada pelatihan *target policy*. Hasil pelatihan menunjukkan bahwa SAC berbasis QMP-HER mampu membantu *target policy* belajar lebih efisien dibandingkan SAC standar dan SAC+HER pada lingkungan simulasi. SAC standar dan SAC+HER belum menunjukkan peningkatan performa yang signifikan hingga 6.000.000 *timestep*, sedangkan SAC berbasis QMP-HER mampu meningkatkan *reward* dan *success rate* melalui total proses pelatihan sebesar 5.500.000 *timestep* hingga diperoleh *target policy*. Hasil pengujian simulasi menunjukkan bahwa *target policy* memperoleh *success rate* sebesar 84% pada kubus, 74% pada silinder, dan 74% pada prisma segitiga. Namun, pada pengujian lingkungan nyata menggunakan robot myCobot 280 Pi, *success rate* menurun menjadi 0% pada kubus, 10% pada silinder, dan 0% pada prisma segitiga. Hasil ini menunjukkan adanya *sim-to-real gap* yang besar. Penurunan performa dipengaruhi oleh *error* deteksi AprilTag, ketidakstabilan *grasp*, perbedaan kondisi fisik antara simulasi dan robot nyata, serta *trajectory replay* yang belum melakukan koreksi secara *real-time*. Dengan demikian, SAC berbasis QMP-HER mampu membantu pembelajaran tugas penyisipan objek pada simulasi, tetapi belum berhasil ditransfer secara andal ke robot nyata.

Kata kunci: *Soft Actor-Critic*, *primitive behaviour*, QMP-HER, robot manipulator, penyisipan objek

ABSTRACT

IMPLEMENTATION OF SOFT ACTOR-CRITIC BASED ON Q-SWITCH MIXTURE OF PRIMITIVES HINDSIGHT EXPERIENCE REPLAY IN A ROBOT MANIPULATOR CONTROL SYSTEM FOR OBJECT INSERTION TASKS

By
MUJADID FAUZAN SALIM TAMIN
Student ID Number: 163221057
Undergraduate Program in Robotics and Artificial Intelligence Engineering

Object insertion tasks in robot manipulators require precise control because the robot must be able to grasp, move, align, and place an object into a target hole with a small tolerance. This research aims to implement and analyze the performance of Soft Actor-Critic based on Q-Switch Mixture of Primitives Hindsight Experience Replay in a robot manipulator control system for object insertion tasks. The tested objects consist of a cube, a cylinder, and a triangular prism with a characteristic dimension of 30 mm and a hole tolerance of 5 mm. The training process was conducted in the MuJoCo simulation environment using two primitive policies, namely grasp policy and insert policy, which were then used as candidate action sources during target policy training. The training results show that SAC based on QMP-HER helped the target policy learn more efficiently in simulation compared to standard SAC and SAC+HER. Standard SAC and SAC+HER did not show significant performance improvement up to 6,000,000 timesteps, while SAC based on QMP-HER improved both reward and success rate through a total training process of 5,500,000 timesteps until the target policy was obtained. The simulation evaluation showed that the target policy achieved a success rate of 84% for the cube, 74% for the cylinder, and 74% for the triangular prism. However, in the real-world evaluation using the myCobot 280 Pi robot, the success rate decreased to 0% for the cube, 10% for the cylinder, and 0% for the triangular prism. These results indicate a large sim-to-real gap. The performance decrease was affected by AprilTag detection errors, unstable grasping, physical differences between simulation and the real robot, and trajectory replay without real-time correction. Therefore, SAC based on QMP-HER can support the learning process of object insertion tasks in simulation, but it has not yet been transferred reliably to the physical robot.

Keywords: Soft Actor-Critic, QMP-HER, primitive behaviour, robot manipulator, object insertion

DAFTAR ISI

HALAMAN JUDUL i
LEMBAR PENGESAHAN ii
PERNYATAAN ORISINALITAS SKRIPSI iii
KETENTUAN PENGGUNAAN SKRIPSI iv
KATA PENGANTAR v
ABSTRAK vi
ABSTRACT vii
DAFTAR ISI viii
DAFTAR TABEL x
DAFTAR GAMBAR xi
DAFTAR LAMPIRAN xiii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 5
1.3 Tujuan Penelitian 6
1.4 Manfaat Penelitian 6
1.4.1 Manfaat Teoretis 6
1.4.2 Manfaat Praktis 6
1.5 Batasan Masalah 6
BAB II TINJAUAN PUSTAKA 8
2.1 Robot Manipulator 8
2.1.1 Kinematika Robot Manipulator 9
2.1.2 myCobot 280 PI 2023 11
2.2 *Reinforcement Learning* 12
2.2.1 *Markov Decision Process* 14
2.2.2 Fungsi Nilai dan Persamaan Bellman 15
2.3 *Soft Actor-Critic* 19
2.3.1 Konsep Dasar *Soft Actor-Critic* 19
2.3.2 Arsitektur dan Komponen *Soft Actor-Critic* 20
2.4 *Hindsight Experience Replay* 25
2.5 *Q-Switch Mixture of Primitives Hindsight Experience Replay* 27
2.6 *Sim-to-Real Transfer* 30
2.6.1 *Domain Randomization* 31
2.7 Kalibrasi Kamera 32
2.8 AprilTag 36
2.9 Metrik Evaluasi 37
2.9.1 Tingkat Keberhasilan 38
2.9.2 Waktu Penyelesaian 38
2.9.3 Efisiensi Sampel 39
2.9.4 Analisis Trajektori *End-Effector* 39
2.10 Penelitian Terdahulu 40
BAB III METODE PENELITIAN 43
3.1 Waktu dan Tempat Penelitian 43
3.2 Alat dan Bahan Penelitian 43
3.3 Metode Penelitian 44

3.3.1 Studi Pustaka 45
3.3.2 Perancangan Sistem 46
3.3.3 Pelatihan Model 59
3.3.4 Pengujian di Lingkungan Simulasi 60
3.3.5 Pengujian di Lingkungan Nyata 63
3.3.6 Analisis Hasil 64
BAB IV HASIL DAN PEMBAHASAN 65
4.1 Implementasi Sistem 65
4.1.1 Implementasi Lingkungan Simulasi 65
4.1.2 Implementasi Ruang Keadaan dan Ruang Aksi 66
4.1.3 Implementasi Alur Pelatihan 68
4.2 Hasil Pelatihan *Policy* 69
4.2.1 Hasil Pelatihan *Grasp Policy* 70
4.2.2 Hasil Pelatihan *Insert Policy* 76
4.2.3 Hasil Pelatihan *Target Policy* 85
4.3 Hasil Pengujian di Lingkungan Simulasi 102
4.4 Hasil Pengujian di Lingkungan Nyata 113
4.4.1 Hasil Kalibrasi Kamera 113
4.4.2 Hasil Pengujian Robot 115
4.5 Analisis Performa Keseluruhan 122
4.6 Pembahasan 126
BAB V KESIMPULAN DAN SARAN 132
5.1 Kesimpulan 132
5.2 Saran 133
DAFTAR PUSTAKA 135
LAMPIRAN 142

DAFTAR TABEL

Tabel 2.1 Spesifikasi Robot myCobot 280 Pi 2023 12
Tabel 2.2 Penelitian Terdahulu..... 40
Tabel 3.1 Perancangan Ruang Keadaan pada *Grasp Policy* 54
Tabel 3.2 Perancangan Ruang Keadaan pada *Insert* dan *Target Policy*..... 55
Tabel 3.3 Perancangan Ruang Aksi pada Semua *Policy* 56
Tabel 4.1 Konfigurasi *Hyperparameter Grasp Policy*..... 70
Tabel 4.2 Konfigurasi *Hyperparameter Insert Policy*..... 77
Tabel 4.3 Konfigurasi *Hyperparameter Target Policy*..... 86
Tabel 4.4 Konfigurasi *Q-Switch* pada Pelatihan *Target Policy*..... 87
Tabel 4.5 Ringkasan Perbandingan Pelatihan *Target Policy*..... 94
Tabel 4.6 Ringkasan Hasil Pengujian Simulasi..... 104
Tabel 4.7 Ringkasan Jumlah *Step* pada Pengujian di Lingkungan Simulasi..... 107
Tabel 4.8 Hasil Pengujian *Target Policy* di Lingkungan Nyata 120

DAFTAR GAMBAR

Gambar 2.1 Ilustrasi Komponen pada Robot Manipulator 9
Gambar 2.2 Struktur dan Konfigurasi Robot myCobot 280 Pi 2023 12
Gambar 2.3 Diagram Interaksi MDP 15
Gambar 2.4 Ilustrasi Skema Algoritma SAC 21
Gambar 2.5 Algoritma *Soft Actor-Critic* 24
Gambar 2.6 Arsitektur *Soft Actor-Critic* 24
Gambar 2.7 Ilustrasi Cara Kerja Metode QMP 28
Gambar 2.8 *Pseudocode* Algoritma QMP-HER 30
Gambar 2.9 Ilustrasi AprilTag 37
Gambar 3.1 Alur Prosedur Penelitian 45
Gambar 3.2 Arsitektur Pelatihan Model pada Lingkungan Simulasi 47
Gambar 3.3 Arsitektur Pengujian Model pada Lingkungan Nyata 47
Gambar 3.4 Desain Gripper Tampak Isometri 48
Gambar 3.5 Desain Gripper Tampak Atas 48
Gambar 3.6 Ilustrasi Isometrik Konfigurasi Lingkungan Eksperimen 49
Gambar 3.7 Ilustrasi Tampak Atas Konfigurasi Lingkungan Eksperimen 50
Gambar 3.8 Pola *chessboard* yang digunakan untuk kalibrasi kamera 51
Gambar 3.9 Alur Kendali Cartesian Berbasis *Inverse Kinematics* 54
Gambar 3.10 *Pseudocode* Algoritma *SAC* dengan pendekatan *QMP–HER* 58
Gambar 3.11 Ilustrasi lingkungan simulasi Mujoco 61
Gambar 4.1 Tiga Bentuk Objek Uji 65
Gambar 4.2 Batas Ruang Kerja Robot 66
Gambar 4.3 Ilustrasi Jarak *End-Effector* dan Objek 67
Gambar 4.4 Ilustrasi Jarak Objek dan Target 67
Gambar 4.5 Diagram Alur Pelatihan 69
Gambar 4.6 Grafik *Reward* Posisi *End-Effector* dan Objek 71
Gambar 4.7 Grafik *Reward* Orientasi *End-Effector* dan Objek 71
Gambar 4.8 Grafik Total *Reward Grasp Policy* 72
Gambar 4.9 Trajektori *End-Effector* Robot pada *Step* ke-5000 73
Gambar 4.10 Trajektori *End-Effector* Robot pada *Step* ke-500.000 74
Gambar 4.11 Trajektori *End-Effector* Robot pada *Step* ke-2.000.000 75
Gambar 4.12 Grafik *Reward* Posisi Objek dan Target 78
Gambar 4.13 Grafik *Reward* Orientasi Objek dan Target 79

Gambar 4.14 Grafik Total *Reward Insert Policy* 80
Gambar 4.15 Trajektori *End-Effector* Robot pada Step ke-5.000..... 81
Gambar 4.16 Trajektori *End-Effector* Robot pada *Step* ke-500.000..... 83
Gambar 4.17 Trajektori *End-Effector* Robot pada *Step* ke-1.500.000..... 84
Gambar 4.18 Grafik Total *Reward Target Policy*..... 89
Gambar 4.19 Grafik *Success Rate*..... 91
Gambar 4.20 Grafik Total *Reward* SAC Standar..... 93
Gambar 4.21 Grafik Total *Reward* SAC+HER..... 94
Gambar 4.22 Grafik Frekuensi Pemilihan Q-Switch..... 96
Gambar 4.23 Grafik Nilai Q Terpilih..... 98
Gambar 4.24 Grafik *Actor Loss* 99
Gambar 4.25 Grafik *Critic Loss*..... 101
Gambar 4.26 Contoh Randomisasi Pose Awal Objek dan Lubang Target..... 102
Gambar 4.27 Trajektori Episode Berhasil pada Objek Kubus 109
Gambar 4.28 Trajektori Hampir Berhasil pada Objek Prisma Segitiga 110
Gambar 4.29 Trajektori Episode Gagal pada Objek Silinder.....111
Gambar 4.30 Contoh hasil deteksi titik sudut *chessboard* 114
Gambar 4.31 Konfigurasi Lingkungan Pengujian Nyata.....116
Gambar 4.32 Desain Alur Pengujian *Target Policy* di Lingkungan Nyata 117
Gambar 4.33 Contoh *Trajectory Replay* 119
Gambar 4.34 Deteksi Menggunakan AprilTag..... 120
Gambar 4.35 Contoh Kegagalan Penyisipan Objek Kubus pada Pengujian..... 122

DAFTAR LAMPIRAN

Lampiran 1. Hasil Pengujian *Target Policy* di Lingkungan Simulasi 142
Lampiran 2. Hasil Pengujian *Target Policy* di Lingkungan Nyata 147

BAB I
PENDAHULUAN

1.1 Latar Belakang

Robot manipulator merupakan salah satu dari jenis robot yang saat ini banyak digunakan di industri seperti perakitan, manufaktur, hingga layanan medis (Han dkk., 2023). Robot ini merupakan sistem mekanis yang dilengkapi dengan aktuator dan sensor. Sistem tersebut dapat dikendalikan untuk melakukan gerakan yang presisi serta terkontrol untuk memanipulasi objek di lingkungannya (Roldan Sanchez, 2025). Secara karakteristik, robot manipulator merupakan sistem nonlinier yang memiliki keterkaitan dinamis antara subsistem. Sistem ini termasuk dalam kategori *multi-input multi-output* yang terdiri dari *link* (lengan), *joint* (sambungan), dan *end-effector* (efektor akhir) yang terpasang pada *link* terakhir yang akan langsung berinteraksi dengan objek (Calderon-Cordova dkk., 2024; R. Liu dkk., 2021).

Tingkat presisi merupakan tolok ukur utama dalam kinerja sebuah robot manipulator. Kemampuan bergerak dengan tingkat presisi tinggi membuatnya banyak dimanfaatkan pada berbagai bidang. Semakin presisi gerakan yang dihasilkan, semakin aman dan optimal pula penggunaannya. Sebaliknya, kekurangan dalam presisi bisa berakibat pada malfungsi sistem bahkan membahayakan keselamatan (Weiming dkk., 2023).

Salah satu bentuk tugas yang menuntut tingkat presisi tinggi adalah tugas penyisipan objek (*object insertion*), di mana robot harus memasukkan suatu komponen ke dalam celah atau lubang dengan toleransi posisi yang sangat kecil (Marougkas dkk., 2025). Tugas ini membutuhkan ketepatan dari posisi *end-effector* robot yang mengharuskan sistem mampu beradaptasi secara dinamis terhadap perubahan lingkungan dan ketidakpastian yang terjadi. Oleh karena itu, dibutuhkan sistem kendali yang mampu beradaptasi terhadap variasi objek dan kondisi (Prakash dkk., 2020).

Sistem kendali tradisional yang umum digunakan masih memiliki banyak keterbatasan. Pendekatan ini sangat bergantung pada model eksplisit dari robot dan lingkungannya. Akibatnya, perancangan sistem kontrol memerlukan model

matematis yang sangat akurat dan waktu yang lama untuk memperoleh hasil optimal. Selain itu, pendekatan ini juga sering gagal ketika diterapkan pada lingkungan dinamis dan penuh ketidakpastian (Avhad dkk., 2024; Harib, 2023).

Reinforcement Learning (RL) menjadi salah satu pendekatan menjanjikan untuk mengoptimalkan kinerja sistem robot, terutama pada tugas-tugas yang sulit diselesaikan oleh metode kendali tradisional. Metode ini memungkinkan sistem robot memiliki kemampuan beradaptasi pada lingkungan yang tidak terduga dan kompleks (Shao dkk., 2023). Dengan pendekatan ini, sistem belajar secara otonom dengan berinteraksi dan bertukar informasi langsung dengan lingkungan hingga sistem dapat menemukan solusi paling optimal (H. Zhang dkk., 2023).

RL merupakan salah satu cabang dari *Machine Learning* yang berfokus pada bagaimana agen menemukan strategi perilaku optimal untuk memaksimalkan hasil (*reward*) melalui proses *trial and error* secara dinamis dan otonom (Roldan Sanchez, 2025). Dalam perkembangannya, RL dikombinasikan dengan *Deep Learning* dan melahirkan pendekatan baru yang dikenal sebagai *Deep Reinforcement Learning* (DRL) (Shao dkk., 2023). *Deep Learning* berperan penting dalam RL sebagai pendekatan aproksimasi fungsi (*function approximator*) yang memungkinkan sistem memproses data berdimensi tinggi, seperti citra atau sensor robotik, secara efisien. Kombinasi ini memungkinkan agen tidak hanya membuat keputusan adaptif seperti pada RL tradisional, tetapi juga memahami kondisi kompleks dari lingkungan. Oleh karena itu, DRL sangat relevan sebagai sistem kendali robot pada lingkungan yang dinamis (R. Liu dkk., 2021).

Salah satu algoritma DRL yang banyak digunakan untuk pengendalian sistem pada ruang aksi kontinu seperti robot manipulator adalah *Soft Actor-Critic* (SAC) (S. Liu, 2024). Algoritma ini termasuk dalam kategori *off-policy Reinforcement Learning* yang menggabungkan efisiensi eksplorasi melalui pendekatan *maximum entropy* (Roldan Sanchez, 2025). SAC unggul dibandingkan algoritma lain seperti *Deep Deterministic Policy Gradient* (DDPG) atau *Proximal Policy Optimization* (PPO) karena mampu mempertahankan keseimbangan antara eksplorasi dan eksploitasi, menghindari *overestimation bias* melalui penggunaan *double Q-network*, serta memiliki efisiensi sampel yang tinggi karena dapat

memanfaatkan kembali pengalaman dari *replay buffer* (Husakovic dkk., 2025; H. Zhang dkk., 2023). Keunggulan-keunggulan tersebut menjadikan SAC sangat cocok diterapkan pada tugas pengendalian kontinu seperti pergerakan robot manipulator yang memerlukan presisi dan adaptivitas tinggi.

Salah satu kendala umum dalam RL adalah masalah *sparse reward*, yaitu kondisi ketika agen sulit memperoleh *reward* karena imbalan hanya muncul di akhir episode (Andrychowicz dkk., 2018). Untuk mengatasi masalah ini, Andrychowicz dkk. (2018) dari OpenAI memperkenalkan teknik *Hindsight Experience Replay* (HER). HER merupakan metode *relabeling* pengalaman yang memungkinkan agen belajar secara efisien bahkan ketika hanya sedikit episode yang berhasil. Metode ini sangat sesuai untuk algoritma *off-policy* seperti SAC karena dapat memanfaatkan kembali pengalaman dalam *replay buffer* tanpa bergantung pada urutan pengambilan sampel.

Penelitian yang dilakukan oleh Xia dkk. (2023) menerapkan kombinasi algoritma SAC dengan metode HER pada sistem robot manipulator 6 *degree of freedom* (DOF*)* untuk melakukan *trajectory planning*. Tujuan penelitian tersebut adalah agar *end-effector* dapat mencapai posisi dan orientasi target secara akurat. Hasil penelitian menunjukkan bahwa pendekatan ini mampu mencapai tingkat keberhasilan 97% dalam simulasi, menandakan peningkatan signifikan dalam efisiensi pembelajaran dan kualitas lintasan.

Meskipun HER terbukti efektif dalam meningkatkan efisiensi pembelajaran pada robot manipulator, metode ini masih memiliki beberapa keterbatasan mendasar. HER hanya melakukan *relabeling* terhadap pengalaman masa lalu agar agen dapat belajar dari kegagalannya, tetapi tidak membantu agen dalam menentukan arah eksplorasi yang lebih bermakna pada tahap awal pelatihan. Akibatnya, agen tetap harus mengumpulkan sejumlah besar pengalaman melalui interaksi acak yang sering kali tidak informatif sebelum menemukan tindakan yang menghasilkan *reward* positif. Kondisi tersebut menyebabkan proses pelatihan tetap memerlukan waktu yang lama (Sanchez dkk., 2023).

Untuk meningkatkan efisiensi pembelajaran pada *HER*, Sanchez dkk. (2023) mengusulkan algoritma *Q-Switch Mixture of Primitives Hindsight*

Experience Replay (QMP-HER). Metode ini menggabungkan prinsip *Q-Switch Mixture of Policies* (QMP) dengan HER untuk memanfaatkan kembali keterampilan dasar (*primitive behaviours*) yang telah dipelajari pada tugas-tugas yang lebih sederhana guna mempercepat pembelajaran pada tugas yang lebih kompleks. Dalam QMP-HER, agen tidak hanya mengandalkan eksplorasi acak seperti pada HER konvensional, tetapi juga mendapatkan panduan dari kebijakan primitif yang telah ada. Penelitian yang dilakukan oleh Sanchez dkk. (2023) menunjukkan peningkatan signifikan dalam efisiensi pembelajaran dan waktu konvergensi dibandingkan metode HER murni. Metode ini mencapai tingkat keberhasilan hingga 95% dengan waktu pelatihan dan jumlah pembaruan kebijakan yang lebih sedikit dibanding metode lainnya.

Meskipun QMP-HER telah menunjukkan kemampuan dalam meningkatkan efisiensi pembelajaran dibandingkan HER biasa, penerapannya pada tugas manipulasi presisi seperti penyisipan objek masih memerlukan kajian lebih lanjut. Tugas penyisipan objek memiliki karakteristik yang lebih kompleks karena keberhasilan tidak hanya ditentukan oleh kemampuan *end-effector* mencapai target, tetapi juga oleh rangkaian perilaku manipulasi yang saling bergantung, yaitu mendekati objek, menggenggam objek, mempertahankan kestabilan objek selama perpindahan, menyelaraskan posisi dan orientasi objek terhadap lubang, serta melepaskan objek pada posisi yang tepat. Kompleksitas ini menjadi semakin tinggi ketika objek yang digunakan memiliki bentuk geometri berbeda, karena setiap bentuk dapat memengaruhi kestabilan *grasp*, arah penyelarasan orientasi, dan peluang keberhasilan saat proses penyisipan.

Penelitian yang dilakukan oleh Sanchez dkk. (2023) menunjukkan bahwa pemanfaatan *primitive behaviour* dapat meningkatkan efisiensi pembelajaran HER. Namun, penelitian tersebut belum secara khusus mengevaluasi penerapan QMP-HER pada tugas penyisipan objek multi-bentuk. Selain itu, penelitian tersebut berfokus pada peningkatan efisiensi pembelajaran di lingkungan simulasi, sedangkan tantangan penerapan pada robot nyata masih memerlukan evaluasi lebih lanjut, terutama karena adanya perbedaan antara simulasi dan kondisi fisik seperti

error persepsi, ketidakstabilan *grasp*, keterbatasan aktuator, serta perbedaan karakteristik kontak.

Berdasarkan *gap* tersebut, penelitian ini berfokus pada implementasi SAC berbasis QMP-HER untuk tugas penyisipan objek dengan variasi bentuk geometri, yaitu kubus, silinder, dan prisma segitiga. Kontribusi utama penelitian ini terletak pada adaptasi QMP-HER dalam kerangka SAC untuk membentuk *target policy* yang dibantu oleh dua *primitive policy*, yaitu *grasp policy* dan *insert policy*. *Grasp policy* digunakan untuk membantu proses mendekati dan menggenggam objek, sedangkan *insert policy* digunakan untuk membantu proses membawa objek menuju lubang target dan menyelaraskan pose objek, serta menyisipkan objek ke lubang target. Kedua *primitive policy* tersebut kemudian digunakan sebagai sumber kandidat aksi selama pelatihan *target policy* melalui mekanisme *Q-Switch*.

Selain mengevaluasi performa pada lingkungan simulasi, penelitian ini juga melakukan pengujian awal pada robot nyata untuk mengamati kemampuan transfer dari simulasi ke sistem fisik. Dengan demikian, penelitian ini tidak hanya menilai apakah SAC berbasis QMP-HER mampu membantu pembelajaran pada tugas penyisipan objek di simulasi, tetapi juga mengidentifikasi keterbatasan utama ketika *policy* hasil pelatihan diterapkan pada robot nyata. Hasil penelitian diharapkan dapat memberikan dasar awal bagi pengembangan metode pembelajaran robotik yang lebih efisien pada simulasi serta menjadi acuan untuk perbaikan sistem *sim-to-real* pada tugas manipulasi presisi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah penelitian ini adalah sebagai berikut:

1. Bagaimana implementasi algoritma *Soft Actor-Critic* dengan pendekatan *Q-Switch Mixture of Primitives Hindsight Experience Replay* untuk mengendalikan robot manipulator dalam melakukan tugas penyisipan objek dengan berbagai bentuk?
2. Bagaimana performa model algoritma *Soft Actor-Critic* dengan pendekatan *Q-Switch Mixture of Primitives Hindsight Experience Replay* dalam

melaksanakan tugas penyisipan objek dengan berbagai bentuk pada lingkungan simulasi dan nyata?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah sebagai berikut:

1. Mengimplementasikan algoritma *Soft Actor-Critic* dengan pendekatan *Q-Switch Mixture of Primitives Hindsight Experience Replay* untuk mengendalikan robot manipulator dalam melakukan tugas penyisipan objek dengan berbagai bentuk.
2. Menganalisis performa dan efektivitas model yang telah dilatih menggunakan algoritma *Soft Actor-Critic* dengan pendekatan *Q-Switch Mixture of Primitives Hindsight Experience Replay* pada lingkungan simulasi dan nyata.

1.4 Manfaat Penelitian

1.4.1 Manfaat Teoretis

1. Memberikan kontribusi terhadap pengembangan algoritma *Reinforcement Learning*, khususnya pada penerapan *Soft Actor-Critic* berbasis QMP-HER.
2. Memperluas pemahaman tentang bagaimana kombinasi *primitive behaviours* dan *experience relabeling* dapat meningkatkan efisiensi pembelajaran dan stabilitas kontrol robot.
3. Menjadi referensi bagi penelitian lanjutan di bidang pembelajaran adaptif dan kendali robotik.

1.4.2 Manfaat Praktis

1. Menjadi dasar awal pengembangan metode peningkatan adaptivitas dan presisi gerak robot manipulator pada tugas penyisipan objek.
2. Menjadi dasar awal pengembangan sistem kendali adaptif berbasis *reinforcement learning* untuk tugas manipulasi presisi.
3. Menjadi dasar pengembangan sistem pembelajaran otonom untuk robot industri pada lingkungan dinamis.

1.5 Batasan Masalah

1. Penelitian ini menggunakan robot MyCobot 280pi dan simulator fisik MuJoCo.

2. Objek uji dibatasi pada tiga bentuk dasar, yaitu kubus, silinder, dan prisma segitiga, dengan dimensi karakteristik 30 mm.
3. Ukuran lubang pada media penyisipan memiliki toleransi sebesar 5 mm terhadap dimensi objek.
4. Persepsi lingkungan dibatasi pada data visual dari Kamera Logitech C270 HD Webcam yang digunakan untuk memperkirakan posisi dan orientasi objek.
5. Penelitian ini berfokus pada tahap implementasi dan evaluasi kinerja QMP-HER pada tugas penyisipan objek, tanpa melakukan optimasi mendalam terhadap arsitektur atau parameter algoritma.
6. Penelitian ini tidak melakukan analisis biaya komputasi secara mendalam, seperti pengukuran durasi pelatihan secara terstruktur, utilisasi GPU, konsumsi memori, konsumsi daya, atau efisiensi komputasi perangkat keras
7. Pengujian pada lingkungan nyata dilakukan hanya sebagai evaluasi awal untuk melihat kemampuan *transfer policy* dari simulasi ke robot fisik. Metode *sim-to-real* yang digunakan masih terbatas pada penyamaan konfigurasi lingkungan simulasi dan lingkungan nyata serta penggunaan *trajectory replay*.

BAB II
TINJAUAN PUSTAKA

2.1 Robot Manipulator

Robot manipulator, yang juga dikenal sebagai lengan robotik (*robotic arm*), adalah sebuah rantai mekanik yang dapat digerakkan, yang tediri dari serangkaian *link* (batang kaku) yang saling terhubung oleh *joint* (sendi) (Crane & Duffy, 2008). Robot ini mampu bekerja dengan kecepatan tinggi, efisiensi energi, serta tingkat ketelitian yang konsisten sehingga mampu untuk menggantikan peran manusia dalam lingkungan industri yang memiliki karakteristik pekerjaan berulang, kotor, dan berbahaya (Yudha dkk., 2018).

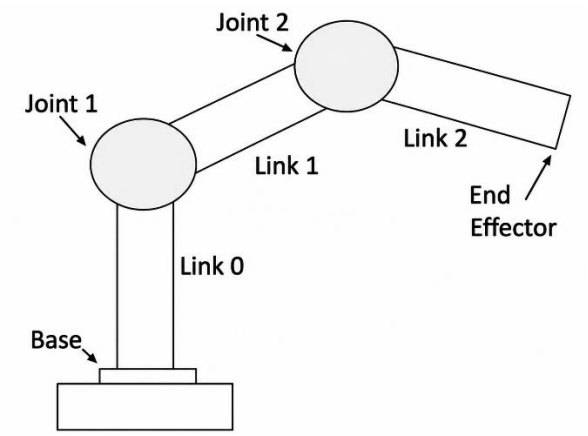
Penerapan robot manipulator telah meluas ke berbagai bidang, seperti manufaktur, pertanian, kesehatan, logistik, eksplorasi ruang angkasa, pendidikan, dan penelitian. Dalam berbagai penerapan tersebut, penggunaan manipulasi robotik terbukti mampu meningkatkan produktivitas, menekan biaya tenaga kerja manusia, meningkatkan akurasi kerja, serta memperkuat aspek keselamatan di lingkungan kerja (Han dkk., 2023).

Istilah manipulasi merujuk pada serangkaian operasi fisik yang dapat dilakukan robot untuk berinteraksi dengan objek di lingkungannya. Kegiatan manipulasi mencakup berbagai tindakan seperti mengambil dan meletakkan benda, mengenggam, melepaskan, serta menyesuaikan posisi dan orientasi objek dalam ruang kerja robot (Yudha dkk., 2018). Dengan demikian, manipulasi merupakan fungsi utama dari robot manipulator, karena melalui kemampuan ini robot dapat meniru peran tangan manusia dalam melakukan tugas-tugas fisik seperti pengangkatan, pemindahan, dan penataan objek (Calderon-Cordova dkk., 2024).

Terdapat beberapa komponen penting pada robot manipulator yang bekerja secara terintegrasi untuk menghasilkan gerakan dan kendali yang presisi. Komponen dasar tersebut meliputi *base, link*, *joint*, serta *end-effector* (Rawat dkk., 2023). Ilustrasi dari bagian – bagian dasar tersebut digambarkan pada Gambar 2.1.

Link merupakan bagian tubuh robot yang bersifat kaku dan berfungsi sebagai penghubung antar bagian dalam rantai kinematik. Setiap *link* dapat didefinisikan berdasarkan dua parameter geometrik utama, yaitu panjang *link* dan

sudut puntir, yang menentukan konfigurasi fisik dan orientasi antar *link* (Crane & Duffy, 2008).



Gambar 2.1 Ilustrasi Komponen pada Robot Manipulator
(Sumber: https://automaticaddison.com/robot-state-publisher-vs-joint-state-publisher/ (2021))

Joint atau sendi berperan sebagai mekanisme penghubung antar *link* yang memungkinkan terjadinya gerakan relatif (Murray dkk., 2017). Jenis *joint* yang paling umum digunakan adalah *revolute joint*, yang memungkinkan rotasi antar *link*, dan *prismatic joint*, yang memungkinkan gerakan translasi atau geser sepanjang sumbu tertentu (Crane & Duffy, 2008). Kombinasi dan jumlah *joint* yang digunakan menentukan derajat kebebasan (*degrees of freedom*) dari sistem, yang menggambarkan kemampuan manipulator untuk bergerak dan menjangkau titik tertentu dalam ruang kerjanya (Murray dkk., 2017; Yudha dkk., 2018).

Base merupakan bagian terbawah dari robot manipulator. Bagian ini langsung dipasang ke tanah (*ground*) dan dianggap stasioner (Yudha dkk., 2018). *Base* berfungsi sebagai kerangka basis, yaitu kerangka koordinat acuan di mana semua pergerakan robot lainnya diukur (Murray dkk., 2017). Sedangkan *end-effector* merupakan bagian ujung dari robot manipulator yang menjadi penghubung langsung antara robot dan lingkungannya. *End-effector* dapat berupa *gripper* atau perangkat lain yang disesuaikan dengan tugas spesifik robot (Murray dkk., 2017).

2.1.1 Kinematika Robot Manipulator

Kinematika pada robot manipulator menjelaskan hubungan antara gerakan pada sendi-sendi (*joints)* robot dengan gerakan yang dihasilkan oleh benda-benda

kaku (*rigid bodies*) yang membentuk robot tersebut (Murray dkk., 2017). Kinematika pada robot manipulator pada dasarnya dibagi menjadi dua masalah utama, yaitu kinematika maju (*forward kinematics*) dan kinematika balik (*inverse kinematics*).

Forward Kinematics, yang sering juga disebut sebagai *forward analysis* atau *forward displacement analysis*, merupakan proses untuk menentukan posisi dan orientasi dari *end-effector* berdasarkan sudut-sudut sendi (*joint angles*). Artinya, apabila semua nilai sudut diketahui, maka posisi dan orientasi dari *end-effector* terhadap kerangka dasar dapat ditentukan secara pasti (Crane & Duffy, 2008). Permasalahan *forward kinematics* dianggap relatif sederhana karena untuk setiap himpunan nilai variabel sendi yang diberikan, selalu terdapat satu solusi unik terhadap posisi dan orientasi *end-effector* (Crane & Duffy, 2008).

Secara matematis, hubungan antara ruang sendi dan ruang konfigurasi dapat dinyatakan sebagai suatu pemetaan yang dinyatakan pada persamaan (2.1).

$$g_{st}: Q \rightarrow SE(3) \tag{2.1}$$

di mana, Q adalah ruang sendi yang memuat semua kombinasi nilai θ dari masing-masing sendi, dan $\mathrm{SE}(3)$ atau *Special Euclidean Group* merupakan himpunan seluruh transformasi Euclidean yang menggambarkan gerakan benda kaku di ruang tiga dimensi. Elemen g_{st} menyatakan transformasi homogen konfigurasi *end-effector* (t) terhadap kerangka dasar (s), mencakup baik rotasi maupun translasi, yang merupakan output dari analisis kinematika maju (Murray dkk., 2017).

Sebaliknya, *inverse kinematics* merupakan proses untuk menentukan nilai sudut-sudut sendi (*joint angles*) untuk mencapai posisi dan orientasi *end-effector* yang diinginkan. Metode ini menjadi sangat penting ketika *end-effector* harus bergerak melalui beberapa titik atau posisi tertentu dalam menjalankan suatu tugas yang telah ditentukan. Berbeda dengan *forward kinematics*, yang relatif langsung karena memiliki satu solusi unik untuk setiap set nilai sendi, *inverse kinematics* bersifat lebih kompleks dan nonlinier (Crane & Duffy, 2008).

Tujuan utama dari *inverse kinematics* adalah menentukan nilai variabel sendi (θ) yang menghasilkan posisi dan orientasi *end-effector* sesuai dengan konfigurasi yang diinginkan (Crane & Duffy, 2008). Secara matematis, jika

konfigurasi target *end-effector* dinyatakan sebagai g_{st}, maka permasalahan *inverse kinematics* dapat dituliskan sebagai penyelesaian dari persamaan *forward kinematics* pada persamaan (2.2).

$$g_{st}(\theta) = g_d \tag{2.2}$$

di mana, g_d adalah pose yang diinginkan (*desired pose*) dari *end-effector* (Murray dkk., 2017).

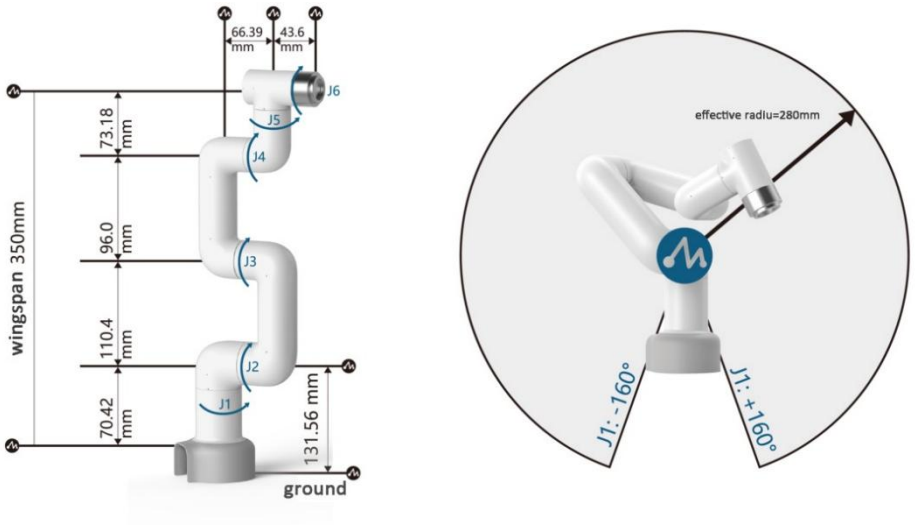
Secara umum, metode penyelesaian kinematika balik dibagi menjadi dua pendekatan utama, yaitu solusi tertutup atau solusi analitik dan solusi numerik atau solusi iterative (Murray dkk., 2017). Pendekatan analitik menghasilkan solusi eksplisit untuk setiap variabel sendi berdasarkan hubungan geometrik. Sementara itu, pendekatan numerik digunakan ketika pendekatan analitik sulit diperoleh. Pendekatan ini dilakukan secara iteratif dengan menebak nilai awal variabel sendi, menghitung konfigurasi hasil melalui *forward kinematics*, lalu meminimalkan selisih antara posisi dan orientasi aktual terhadap yang diinginkan (Crane & Duffy, 2008). Secara keseluruhan, kinematika merupakan dasar penting dalam perancangan dan pengendalian gerak robot, karena memungkinkan penentuan hubungan antara gerakan sendi dan posisi serta orientasi *end-effector*, baik melalui analisis kinematika maju maupun kinematika balik.

2.1.2 myCobot 280 PI 2023

Robot MyCobot 280 Pi 2023 merupakan robot lengan kolaboratif (*collaborative robotic arm*) berukuran kecil yang dikembangkan oleh Elephant Robotics. Robot ini termasuk dalam kategori desktop manipulator dengan enam derajat kebebasan (6-*DOF*), yang dirancang untuk kebutuhan penelitian, pendidikan, dan pengembangan aplikasi berbasis robotika (Elephant Robotics, 2023). Gambar 2.2 menjelaskan struktur dan konfigurasi dari myCobot 280 Pi 2023.

MyCobot 280 Pi menggunakan Raspberry Pi 4B sebagai pengendali utama, sehingga mendukung pemrograman berbasis Python, ROS (*Robot Operating System*), serta komunikasi serial melalui *port USB*. Dengan desain modular dan terbuka, robot ini mudah dikembangkan untuk integrasi dengan sensor, kamera, atau perangkat eksternal lainnya. Robot MyCobot 280 Pi memiliki konfigurasi 6-

DOF revolute dengan rantai terbuka (*open-chain manipulator*). Keenam sendinya dihubungkan secara serial mulai dari *base joint* hingga *end-effector*, sehingga mampu melakukan gerakan dalam tiga dimensi dengan kombinasi rotasi dan translasi (Elephant Robotics, 2023). Spesifikasi dari myCobot 280 Pi dapat dilihat pada Tabel 2.1.



Gambar 2.2 Struktur dan Konfigurasi Robot myCobot 280 Pi 2023

(Sumber: Elephant Robotics (2023))

Tabel 2.1 Spesifikasi Robot myCobot 280 Pi 2023

Parameter	Spesifikasi
Jumlah Derajat Kebebasan	6 *DOF*
Pengendali Utama	Raspberry Pi 4B
Jangkauan Kerja Maksimum	±280 mm
Kapasitas Beban	±250 g
Kepresisian	±0,5 mm
Catu Daya	8–12 Volt
Berat Robot	±1,2 kg
Komunikasi	USB, Wi-Fi, Bluetooth
Material Struktur	Aluminium alloy dan plastik ABS

2.2 *Reinforcement Learning*

Reinforcement Learning (RL) merupakan salah satu paradigma penting dan berpengaruh dalam bidang kecerdasan buatan (Ghasemi dkk., 2025). Paradigma ini memiliki pendekatan pembelajaran yang unik, yaitu berfokus pada perilaku yang

berorientasi pada tujuan (*goal-directed behaviour*) melalui proses interaksi antara agen dan lingkungannya (Sutton & Barto, 2018).

Reinforcement Learning merupakan salah satu cabang dari *Machine Learning* di mana sebuah agen belajar mengambil keputusan dengan cara berinteraksi secara berulang dengan lingkungannya (Naeem dkk., 2020). Tujuan utama dari agen ini adalah menemukan tindakan-tindakan terbaik yang seharusnya dilakukan pada kondisi tertentu agar memperoleh nilai imbalan kumulatif (*cumulative reward*) yang maksimal, sehingga agen dapat mempelajari kebijakan (*policy*) yang paling optimal untuk mencapai tujuan dalam jangka panjang (Almahamid & Grolinger, 2021).

Menurut Sutton & Barto (2018) *Reinforcement Learning* memiliki tiga karakteristik utama yang membedakannya dari bentuk pembelajaran mesin lainnya. Pertama, interaksi bersifat tertutup (*closed-loop interaction*), di mana setiap tindakan yang diambil oleh agen akan memengaruhi keadaan berikutnya.

Kedua, tidak adanya instruksi langsung (*no direct instruction*), artinya agen tidak diberi tahu tindakan apa yang benar untuk dilakukan. Sebaliknya, agen harus menemukan sendiri tindakan yang menghasilkan imbalan terbesar melalui proses *trial and error* (Naeem dkk., 2020).

Ketiga, metode ini ditandai oleh adanya konsekuensi yang tertunda (*delayed consequences*). Dampak dari suatu tindakan tidak selalu langsung terlihat, melainkan mungkin baru berpengaruh pada nilai imbalan setelah beberapa waktu. Oleh karena itu, agen harus mempertimbangkan imbalan jangka panjang (*long-term cumulative reward*), bukan hanya keuntungan sesaat.

Reinforcement Learning disebut sebagai kategori ketiga dalam bidang *Machine Learning*, yang berdiri sejajar dengan *Supervised Learning* dan *Unsupervised Learning*. Jika *Supervised Learning* mengandalkan contoh berlabel, dan *Unsupervised Learning* berfokus pada penemuan pola tersembunyi dalam data, maka *RL* beroperasi berdasarkan prinsip belajar dari interaksi dan umpan balik dinamis (Naeem dkk., 2020; Sutton & Barto, 2018).

2.2.1 *Markov Decision Process*

Markov Decision Process (MDP) adalah model matematis utama yang menjadi fondasi bagi berbagai metode dalam RL. MDP didefinisikan sebagai suatu proses pengambilan keputusan berurutan (*sequential decision-making process*) yang bersifat stokastik, di mana hasil dari suatu tindakan bergantung sebagian pada keputusan agen dan sebagian lainnya pada faktor acak (Ghasemi dkk., 2025; Sewak, 2019).

Inti dari MDP terletak pada Sifat Markov (*Markov Property*), yang sering disebut sebagai sifat "tanpa ingatan" (*memoryless*) (Sewak, 2019). Sifat ini menyatakan bahwa probabilitas keadaan berikutnya (*next state*) hanya bergantung pada keadaan saat ini dan tindakan yang diambil, bukan pada urutan keadaan sebelumnya (Cai dkk., 2025). Dengan demikian, kebijakan (*policy*) hanya perlu mempertimbangkan keadaan saat ini tanpa mengingat seluruh riwayat interaksi (Naeem dkk., 2020). MDP dapat didefinisikan sebagai tupel yang dinyatakan pada Notasi (2.3) (Ghasemi dkk., 2025).

$$(S, A, P, R, \gamma) \tag{2.3}$$

di mana:

S : himpunan dari keadaan lingkungan (*state space*)

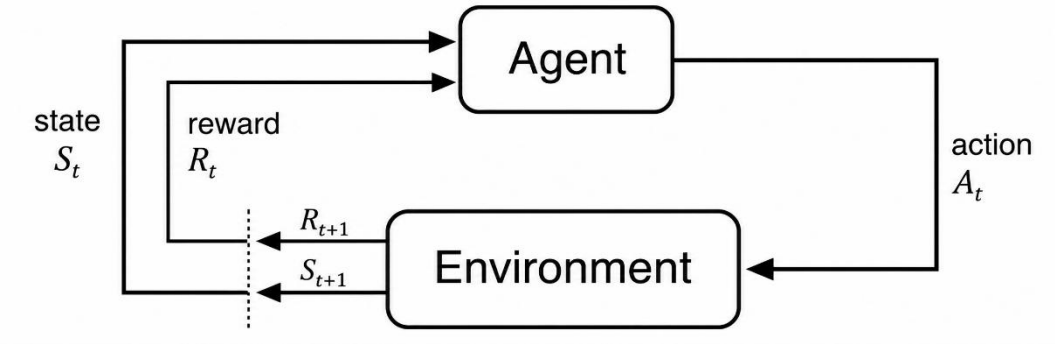
A : himpunan dari tindakan yang tersedia bagi agen (*action space*)

P : fungsi probabilitas transisi keadaan, $P(s' \mid s, a)$

R : fungsi *reward* $R(s, a)$ yang memberikan nilai umpan balik

γ : faktor diskonto (*discount factor*) dengan rentang nilai 0 dan 1

Struktur interaksi dalam *Reinforcement Learning* dengan mengikuti *Markov Decision Process* dapat dilihat pada Gambar 2.3 berikut. Gambar 2.3 memperlihatkan mekanisme dasar interaksi antara agen dan lingkungan dalam RL, di mana agen berperan sebagai pengambil keputusan yang berinteraksi secara berulang dengan lingkungannya. Pada setiap langkah waktu t, agen mengamati keadaan atau *state* S_t, kemudian berdasarkan kebijakan (*policy*) yang dimilikinya, agen memilih dan melakukan suatu aksi (*action*) A_t. Lingkungan merespons aksi tersebut dengan memberikan umpan balik berupa *reward* R_t, yang mencerminkan



Gambar 2.3 Diagram Interaksi MDP
(Sumber: D. Wang & Snooks (2021))

keberhasilan atau kegagalan tindakan yang dilakukan, serta menghasilkan *state* baru S_{t+1}. Proses ini berlangsung secara terus-menerus membentuk suatu siklus umpan balik antara agen dan lingkungan, di mana agen secara bertahap belajar dari pengalaman untuk memperbarui kebijakan pengambilan keputusannya untuk memaksimalkan *reward* kumulatif (D. Wang & Snooks, 2021).

Tujuan utama dalam MDP adalah memaksimalkan total imbalan diskonto (*discounted cumulative reward*) yang diperoleh agen sepanjang waktu. Secara matematis, hal ini dapat dinyatakan pada persamaan (2.4) (Ghasemi dkk., 2025).

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.4}$$

di mana:

G_t : *reward* hasil kumulatif terdiskonto pada waktu t
R_{t+k+1} : *reward* yang diterima pada langkah ke-t + k + 1
k : jumlah langkah ke depan dalam perhitungan total imbalan

Agen kemudian mencari kebijakan optimal (π^*) yang memaksimalkan nilai harapan dari penjumlahan *reward* masa kini hingga mendatang (Ghasemi dkk., 2025).

2.2.2 Fungsi Nilai dan Persamaan Bellman

Dalam konteks MDP, fungsi nilai (*value function*) berperan sebagai ukuran seberapa baik suatu keadaan atau pasangan keadaan–aksi bagi agen. Fungsi ini menggambarkan jumlah *reward* kumulatif yang diharapkan jika agen memulai dari keadaan tertentu dan mengikuti kebijakan tertentu (π) di masa mendatang. Dengan

demikian, fungsi nilai memberikan penilaian jangka panjang terhadap tindakan dan menjadi dasar pengambilan keputusan dalam *Reinforcement Learning* (Calderon-Cordova dkk., 2024).

Terdapat dua bentuk utama fungsi nilai. Pertama, fungsi nilai keadaan (*state-value function*), dilambangkan sebagai $V_\pi(s)$, yang menyatakan nilai ekspektasi dari *reward* kumulatif terdiskonto jika agen berada pada keadaan s dan selanjutnya mengikuti kebijakan π (H. Zhang dkk., 2023). Secara matematis, fungsi ini dapat ditulis sebagai persamaan (2.5) (Sutton & Barto, 2018).

$$V_\pi(s) = E_\pi[\, G_t \mid S_t = s \,] \tag{2.5}$$

di mana:

$V_\pi(s)$: nilai ekspektasi *return* kumulatif dari keadaan s di bawah kebijakan π

E_π : operator ekspektasi berdasarkan kebijakan π

S_t : keadaan sistem pada waktu t

π : kebijakan yang menentukan probabilitas pemilihan aksi oleh agen

Fungsi ini mencerminkan kebaikan relatif dari suatu keadaan, yaitu seberapa menguntungkan keadaan tersebut jika kebijakan tertentu diterapkan (Roldan Sanchez, 2025).

Kedua, fungsi nilai aksi (*action-value function*), dilambangkan $Q_\pi(s,a)$, yang mengukur nilai ekspektasi dari *reward* kumulatif jika agen berada pada keadaan s, mengambil tindakan a, dan kemudian mengikuti kebijakan π. Secara matematis, fungsi ini dirumuskan pada persamaan (2.6) (Sutton & Barto, 2018):

$$Q_\pi(s,a) = E_\pi[\, G_t \mid S_t = s, A_t = a \,] \tag{2.6}$$

di mana:

$Q_\pi(s,a)$: nilai ekspektasi *return* dari keadaan s dan aksi a di bawah kebijakan π

A_t : aksi yang diambil pada waktu t

Fungsi ini menilai kualitas suatu tindakan dalam keadaan tertentu dan menyediakan informasi lebih rinci dibanding fungsi nilai keadaan (Roldan Sanchez, 2025). Tujuan pembelajaran dalam *Reinforcement Learning* adalah menemukan kebijakan optimal π^* yang memaksimalkan kedua fungsi nilai tersebut, sehingga diperoleh $V^*(s)$ dan $Q^*(s,a)$ (Shao dkk., 2023; H. Zhang dkk., 2023).

Fungsi nilai tersebut bersifat rekursif, yang berarti nilai suatu keadaan dapat dinyatakan berdasarkan nilai keadaan berikutnya. Hubungan rekursif ini dijelaskan melalui Persamaan Bellman (*Bellman Equation*). Persamaan Bellman merupakan konsep fundamental dalam RL yang berperan penting untuk menghitung dan memperbarui fungsi nilai (Calderon-Cordova dkk., 2024). Persamaan Bellman menggambarkan hubungan antara nilai suatu keadaan dengan nilai keadaan-keadaan berikutnya, berdasarkan *reward* langsung yang diterima dan nilai diskonto dari masa depan (Sewak, 2019).

Terdapat dua bentuk utama Persamaan Bellman, yaitu *Bellman Expectation Equation* dan *Bellman Optimality Equation*. *Bellman Expectation Equation* digunakan untuk mengevaluasi kebijakan tertentu. Dalam konteks ini, agen diasumsikan mengikuti *policy* π yang telah ditentukan, sehingga nilai suatu keadaan merupakan ekspektasi dari *reward* yang diterima dan nilai keadaan berikutnya berdasarkan distribusi aksi dari *policy* tersebut (Ghasemi dkk., 2025). Bentuk matematisnya dapat dituliskan pada persamaan (2.7) (Sutton & Barto, 2018):

$$V_{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s',r} P(s', r \mid s, a)\ [r + \gamma V_{\pi}(s')] \tag{2.7}$$

di mana:

$\pi(a \mid s)$	: probabilitas agen memilih aksi a di keadaan s
$P(s', r \mid s, a)$	: probabilitas transisi ke keadaan s' dan mendapat *reward* r
r	: *reward* yang diterima
$V_{\pi}(s')$	: nilai keadaan berikutnya berdasarkan kebijakan π

Sedangkan untuk fungsi nilai aksi dituliskan pada persamaan (2.8).

$$Q_{\pi}(s, a) = \sum_{s',r} P(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') Q_{\pi}(s', a') \right] \tag{2.8}$$

Kedua bentuk persamaan ini berfungsi sebagai alat evaluasi yang menggambarkan hubungan rekursif antara imbal hasil langsung dan nilai masa depan. Dengan demikian, *Bellman Expectation Equation* memungkinkan agen

untuk memperkirakan seberapa baik kebijakan yang sedang diikuti dalam mencapai tujuan jangka panjang (Ghasemi dkk., 2025).

Sementara itu, formulasi kedua dikenal sebagai *Bellman Optimality Equation*, yang digunakan untuk mencari kebijakan terbaik atau kebijakan optimal π^*. Berbeda dengan bentuk sebelumnya yang menghitung nilai di bawah kebijakan tetap, persamaan optimalitas ini mencari nilai maksimum yang dapat dicapai oleh agen dengan memilih tindakan terbaik di setiap keadaan (Sutton & Barto, 2018). Bentuk matematisnya dapat dituliskan pada persamaan (2.9) (Sutton & Barto, 2018):

$$V^*(s) = \max_a \sum_{s',r} P(s', r \mid s, a)[r + \gamma V^*(s')] \tag{2.9}$$

di mana:

$V^*(s)$: nilai optimal dari suatu keadaan s

$V^*(s')$: nilai optimal keadaan berikutnya s'

Hal yang sama berlaku untuk fungsi nilai aksi optimal $Q^*(s,a)$, yang menggambarkan nilai ekspektasi maksimum yang dapat dicapai jika agen mengambil tindakan a pada keadaan s dan kemudian bertindak optimal pada langkah-langkah berikutnya. Hubungan tersebut dapat dinyatakan pada persamaan (2.10) (Sutton & Barto, 2018):

$$Q^*(s,a) = \sum_{s',r} P(s', r \mid s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right] \tag{2.10}$$

di mana:

$Q^*(s,a)$: nilai optimal dari aksi a pada keadaan s

$Q^*(s',a')$: nilai optimal dari aksi a' pada keadaan berikutnya s'

Secara keseluruhan, formulasi Persamaan Bellman baik dalam bentuk ekspektasi maupun optimalitas memberikan kerangka matematis yang komprehensif untuk menghitung dan memperbarui nilai jangka panjang dari suatu kebijakan. Persamaan ekspektasi digunakan untuk mengevaluasi kebijakan yang sedang diikuti, sedangkan persamaan optimalitas digunakan untuk mencari kebijakan terbaik yang memaksimalkan total imbal hasil kumulatif. Kedua bentuk persamaan ini menjadi fondasi utama dalam pengembangan berbagai metode *RL*.

2.3 *Soft Actor-Critic*

2.3.1 Konsep Dasar *Soft Actor-Critic*

Soft Actor-Critic (SAC) merupakan algoritma *model-free off-policy Reinforcement Learning* yang termasuk ke dalam keluarga *actor–critic*. Algoritma ini dikembangkan untuk mengatasi tantangan utama yang sering muncul dalam *Deep Reinforcement Learning*, yaitu kompleksitas sampel yang tinggi dan sensitivitas terhadap hiperparameter. Berbeda dengan pendekatan lain seperti *DDPG* atau *TD3* (*Twin Delayed* DDPG) yang hanya berfokus pada maksimisasi *reward* kumulatif, SAC menambahkan entropi ke dalam fungsi objektifnya (Haarnoja dkk., 2018; Wong dkk., 2021). Haarnoja dkk. (2018) mengusulkan kerangka *Maximum Entropy Reinforcement Learning*, yang tidak hanya memaksimalkan *expected reward*, tetapi juga memaksimalkan entropi kebijakan. Pendekatan ini mendorong eksplorasi yang lebih luas dan menghasilkan kebijakan yang lebih *robust* terhadap kesalahan estimasi model. Tujuan pembelajaran dalam kerangka ini diformulasikan pada persamaan (2.11).

$$J(\pi) = \sum_{t=0}^{T} E_{(s_t,a_t)\sim\rho^{\pi}}\left[r(s_t,a_t) + \alpha\,\mathcal{H}\left(\pi(\cdot|\, s_t)\right)\right] \tag{2.11}$$

di mana:

$r(s_t, a_t)$: nilai *reward* dengan *state* dan *action* pada waktu ke-t

$\mathcal{H}\left(\pi(\cdot|\, s_t)\right)$: entropi dari kebijakan

α : koefisien temperatur

Nilai α pada rumus tersebut berperan sebagai koefisien *temperature* yang menyeimbangkan antara eksplorasi (entropi) dan eksploitasi (*reward*). Ketika α=0, formulasi ini kembali ke objektif *RL* konvensional.

Melalui pendekatan ini, aktor dalam SAC berupaya mencapai dua sasaran utama, yaitu memperoleh *reward* yang tinggi untuk menyelesaikan tugas sekaligus mempertahankan tingkat ketidakpastian (entropi) agar kebijakan tetap eksploratif dan *robust* terhadap kesalahan estimasi nilai (Kasaura dkk., 2023). Dengan demikian, *policy* yang dihasilkan mampu menangkap berbagai mode perilaku yang

optimal dan memberikan probabilitas seimbang terhadap tindakan-tindakan yang sama baiknya (Haarnoja dkk., 2018).

Landasan teoretis dari algoritma SAC adalah konsep *Soft Policy Iteration*, yang dikembangkan untuk mencapai optimal *policy* dalam kerangka *Maximum Entropy Reinforcement Learning*. *Soft Policy Iteration* terdiri atas dua tahap utama, yaitu *soft policy evaluation* dan *soft policy improvement*. Kedua tahap ini dilakukan secara bergantian hingga kebijakan konvergen ke *soft optimal policy* (Haarnoja dkk., 2018).

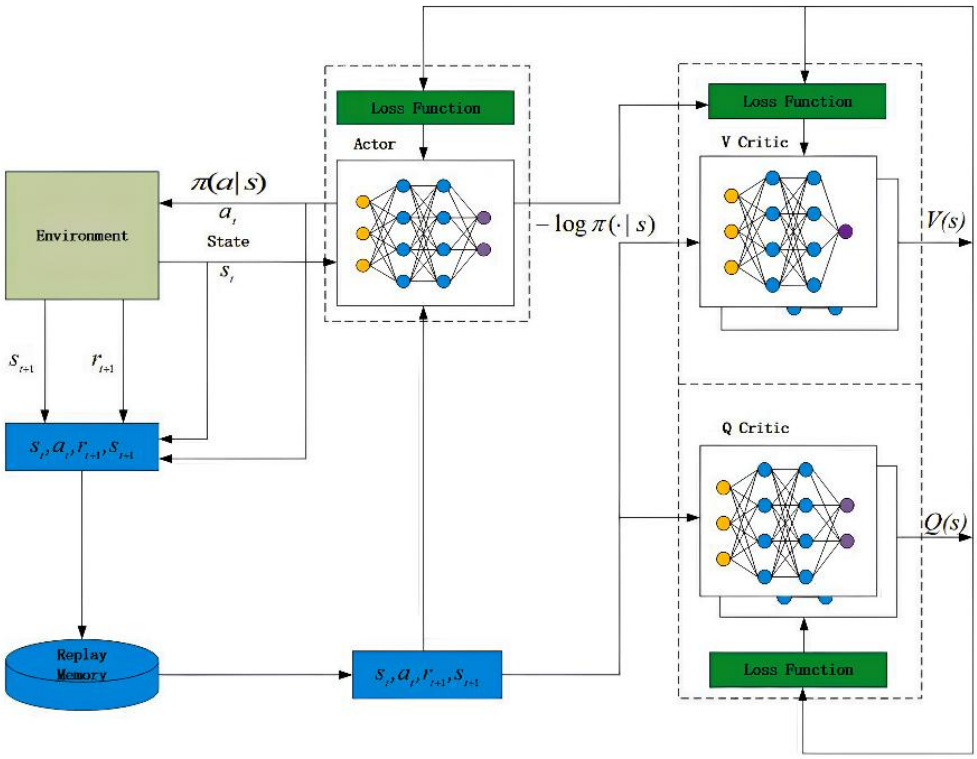
2.3.2 Arsitektur dan Komponen *Soft Actor-Critic*

Pada implementasinya, SAC menggunakan *deep neural network* untuk mengaproksimasi fungsi-fungsi nilai dan kebijakan. Pendekatan berbasis jaringan saraf ini memungkinkan SAC diterapkan pada lingkungan dengan ruang keadaan dan aksi yang kontinu, di mana perhitungan nilai eksak tidak memungkinkan secara analitik. Dengan kemampuan representasi yang kuat, jaringan-jaringan ini membantu SAC mengatasi kompleksitas dinamika lingkungan yang tinggi (Haarnoja dkk., 2018).

Arsitektur SAC terdiri atas tiga komponen utama, yaitu dua jaringan Q (*soft Q-network*) sebagai *critic*, satu jaringan nilai keadaan (*value network*), dan satu jaringan kebijakan stokastik (*policy network*) sebagai *actor*. Ketiga komponen ini saling berinteraksi dan diperbarui secara bergantian selama proses pelatihan, sehingga membentuk sistem pembelajaran yang stabil, efisien, dan eksploratif (Haarnoja dkk., 2018). Gambar 2.4 berikut menggambarkan skema keseluruhan dari algoritma SAC.

Komponen pertama dari arsitektur SAC adalah dua buah jaringan Q yang masing-masing diwakili oleh $Q_{\theta_1}(s,a)\ dan\ Q_{\theta_2}(s,a)$. Penggunaan dua jaringan Q berfungsi untuk mengurangi bias estimasi positif (*overestimation bias*). Proses pembaruan parameter jaringan Q dilakukan dengan meminimalkan fungsi kehilangan (*loss function*) pada persamaan (2.12) (Haarnoja dkk., 2018):

$$J_Q(\theta) = E_{(s_t,a_t)\sim D}\left[\frac{1}{2}\left(Q_\theta(s_t,a_t) - \hat{Q}(s_t,a_t)\right)^2\right] \qquad (2.12)$$



Gambar 2.4 Ilustrasi Skema Algoritma SAC

(Sumber: (Li dkk., 2023))

dengan target nilai yang dimodifikasi pada persamaan (2.13) berikut.

$$\hat{Q}(s_t, a_t) = r(s_t, a_t) + \gamma E_{s' \sim p}\left[V_{\bar{\psi}}(s')\right] \tag{2.13}$$

dengan gradien untuk pembaruan parameter θ dapat dinyatakan pada persamaan (2.14).

$$\nabla_\theta J_Q(\theta) = \nabla_\theta Q_\theta(s_t, a_t)\left(Q_\theta(s_t, a_t) - r(s_t, a_t) - \gamma V_{\bar{\psi}}(s')\right) \tag{2.14}$$

di mana:

$J_Q(\theta)$	: fungsi kerugian
$E_{(s_t,a_t)\sim D}$	: nilai harapan atas sampel dari *replay buffer D*
$Q_\theta(s_t, a_t)$	: nilai Q yang diprediksi oleh jaringan *critic*
$V_{\bar{\psi}}(s')$	: nilai target dari *value network*
∇_θ	: operator gradien terhadap parameter θ
γ	: faktor diskonto

Pembaruan jaringan Q dilakukan secara *off-policy*, sehingga data lama masih dapat dimanfaatkan untuk memperbarui model tanpa harus bergantung pada

pengalaman terbaru. Hal ini meningkatkan efisiensi sampel dan mempercepat konvergensi. Selain itu, nilai minimum dari dua *Q-network* digunakan dalam pembaruan *value network* dan *policy*, untuk memastikan estimasi yang konservatif (Haarnoja dkk., 2018).

Untuk menjaga stabilitas pelatihan, parameter target $\bar{\theta}$ pada jaringan Q diperbarui secara lembut menggunakan *soft target update*, dengan persamaan (2.15)

$$\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta} \tag{2.15}$$

Di mana τ bernilai kecil (biasanya 0.005). Mekanisme ini berfungsi meredam perubahan mendadak pada estimasi Q, sehingga pembelajaran lebih stabil (Haarnoja dkk., 2019).

Komponen kedua adalah *value network* $V_\psi(s)$, yang berfungsi memperkirakan nilai keadaan dengan memperhitungkan kontribusi entropi kebijakan. Nilai keadaan ini menggambarkan ekspektasi *Q-value* yang dikurangi dengan ketidakpastian kebijakan yang dinyatakan pada persamaan (2.16) (Haarnoja dkk., 2018).

$$V_\psi(s) = E_{a\sim\pi_\phi}\left[Q_{\min}(s_t, a_t) - \alpha \log \pi_\phi\,(a \mid s)\right] \tag{2.16}$$

di mana jaringan ini dilatih untuk meminimalkan *squared residual error* pada persamaan (2.17).

$$J_V(\psi) = E_{s\sim\mathbb{D}}\left[\frac{1}{2}\left(V_\psi(s_t) - E_{a\sim\pi_\phi}\left[Q_\theta(s_t, a_t) - \alpha \log \pi_\phi\,(a|s)\right]\right)^2\right] \tag{2.17}$$

dengan gradien untuk pembaruan terhadap parameter ψ dapat dinyatakan pada persamaan (2.18).

$$\nabla_\psi J_V(\psi) = \nabla_\psi V_\psi(s_t)\left(V_\psi(s) - Q_\theta(s_t, a_t) + \alpha \log \pi_\phi\,(a|s)\right) \tag{2.18}$$

di mana:

$J_V(\psi)$	: fungsi kerugian *value network*
∇_ψ	: operator gradien terhadap parameter ψ
$V_\psi(s)$	: nilai keadaan yang diprediksi oleh *value network*
$Q_{\min}(s_t, a_t)$	: nilai minimum dari dua *Q-network*
$\log \pi_\phi\,(a \mid s)$	: *log-likelihood* aksi yang dihasilkan oleh kebijakan

Dengan memperbarui ψ sesuai gradien ini, *value network* belajar merepresentasikan nilai keadaan yang selaras dengan kebijakan saat ini (Haarnoja dkk., 2018).

Pada versi SAC awal (Haarnoja dkk., 2018), arsitektur mencakup sebuah *value network* terpisah untuk memperkirakan nilai keadaan. Namun, penelitian lanjutan (Haarnoja dkk., 2019) menemukan bahwa fungsi nilai ini dapat diestimasi langsung dari *Q-network*, sehingga komponen *value network* tidak lagi digunakan dalam implementasi SAC modern. Penghapusan komponen ini menyederhanakan arsitektur dan mengurangi kompleksitas pelatihan tanpa menurunkan stabilitas.

Komponen ketiga dalam arsitektur SAC adalah *policy network* $J_\pi(\phi)$, yang berperan menghasilkan distribusi probabilitas tindakan berdasarkan keadaan tertentu. Berbeda dengan algoritma deterministik seperti DDPG, SAC menggunakan kebijakan stokastik berbasis distribusi Gaussian, yang memungkinkan agen untuk melakukan eksplorasi alami. Jaringan ini menghasilkan dua keluaran, yaitu mean $\mu_\phi(s)$ dan standar deviasi $\sigma_\phi(s)$, yang digunakan untuk menentukan aksi melalui proses *sampling* (Haarnoja dkk., 2018). Untuk mengoptimalkan kebijakan, *SAC* meminimalkan fungsi pada persamaan (2.19).

$$J_\pi(\phi) = E_{s\sim D, a\sim\pi_\phi}\left[\alpha \log \pi_\phi\,(a|s) - Q_\theta(s_t, a_t)\right] \tag{2.19}$$

Di mana, $\alpha \log \pi_\phi\,(a|s)$ berperan sebagai regularisasi entropi yang menjaga tingkat eksplorasi, $-Q_\theta(s_t, a_t)$ berperan memastikan kebijakan tetap memprioritaskan tindakan dengan nilai tinggi, dan parameter α berfungsi sebagai koefisien *temperature* yang menyeimbangkan eksplorasi dan eksploitasi (Haarnoja dkk., 2018).

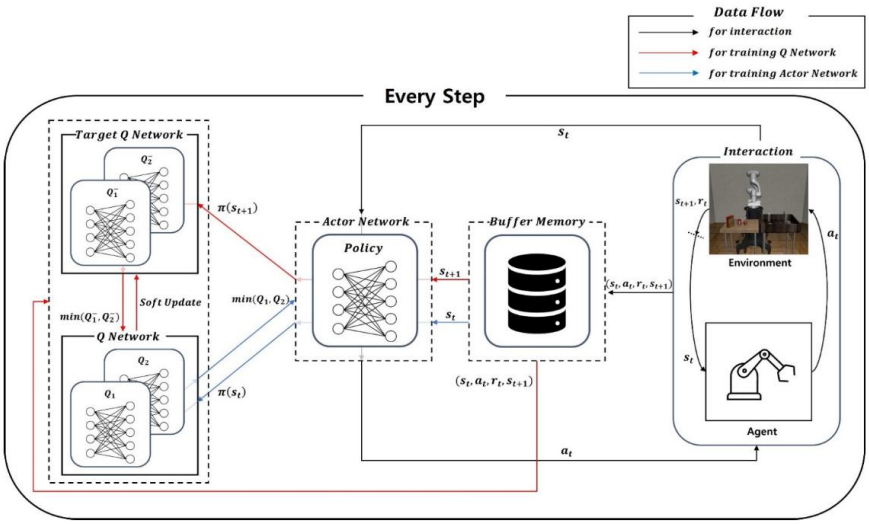
Ketiga jaringan bekerja dalam satu siklus pembelajaran. *Critic* $(Q_{\theta_1}, Q_{\theta_2})$ memperkirakan nilai aksi yang diambil oleh kebijakan, *Value network* (V_ψ) menilai keadaan saat ini dengan memperhitungkan entropi kebijakan, *Policy network* (π_ϕ) diperbarui untuk memaksimalkan *Q-value* sambil tetap mempertahankan kebijakan yang acak. Seluruh proses dilakukan secara *off-policy*, menggunakan data dari *replay buffer*. Berikut langkah-langkah algoritma SAC yang dijelaskan dalam *pseudocode* pada Gambar 2.5 (Haarnoja dkk., 2018).

Algoritma Soft Actor-Critic (SAC)

Inisialisasi: parameter vektor $\theta_1, \theta_2, \phi, \alpha$
Inisialisasi target Q: $\bar{\theta}_1 \leftarrow \theta_1, \quad \bar{\theta}_2 \leftarrow \theta_2$
Inisialisasi replay buffer: $\mathcal{D} \leftarrow \emptyset$
for iterasi $= 1$ hingga N **do**
  for langkah lingkungan $t = 0$ hingga $S - 1$ **do**
    Sampel aksi: $a_t \sim \pi_\phi(a \mid s_t)$
    Langkah lingkungan: $s_{t+1} \sim p(s_{t+1} \mid s_t, a_t)$
    Simpan transisi: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r(s_t, a_t), s_{t+1})\}$
  end for
  for langkah gradien $g = 1$ hingga G **do**
    Perbarui dua *Q-Network*:
      $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i), \quad i \in \{1, 2\}$
    Perbarui *Policy Network*:
      $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    Sesuaikan temperatur:
      $\alpha \leftarrow \alpha - \lambda_\alpha \hat{\nabla}_\alpha J(\alpha)$
    Soft update target Q:
      $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i, \quad i \in \{1, 2\}$
  end for
end for
Keluaran: θ_1, θ_2, ϕ *(parameter teroptimasi)*

Gambar 2.5 Algoritma *Soft Actor-Critic*

Gambar 2.6 menujukkan arsitektur algoritma SAC yang digunakan dalam penelitian Kim dkk. (2023) sebagai algoritma utama dalam menyelesaikan tugas *Pick and Place* dengan robot manipulator.



Gambar 2.6 Arsitektur *Soft Actor-Critic*

(Sumber: Kim dkk., 2023)

Berdasarkan hasil eksperimen pada berbagai *continuous control benchmark* seperti Hopper-v1, Walker2d-v1, HalfCheetah-v1, Ant-v1, dan Humanoid-v1, SAC menunjukkan performa yang lebih cepat konvergen, lebih efisien dalam

penggunaan sampel, serta lebih stabil dibandingkan DDPG, PPO, dan TD3. Haarnoja dkk. (2018) melaporkan bahwa SAC mampu mencapai performa optimal dengan variansi yang rendah antar *random seeds*, hal ini menunjukkan konsistensi hasil yang tinggi.

2.4 *Hindsight Experience Replay*

Sebelum diperkenalkannya metode *Hindsight Experience Replay* (HER) oleh Andrychowicz dkk. (2018), algoritma RL konvensional menghadapi berbagai kendala mendasar yang menghambat efektivitas pembelajaran, terutama pada lingkungan yang kompleks dan bersifat *sparse reward*, di mana sinyal umpan balik (*reward*) jarang diberikan dan hanya muncul ketika agen benar-benar mencapai tujuan akhir. Secara umum, permasalahan utama yang dihadapi RL sebelum HER mencakup tiga aspek utama, yaitu masalah kelangkaan sinyal *reward* (*sparse reward*), ketergantungan pada rekayasa *reward* (*reward shaping*), dan ketidakmampuan agen untuk belajar dari kegagalan (Andrychowicz dkk., 2018; Ibrahim dkk., 2024).

Masalah *sparse reward* merupakan masalah paling mendasar dalam RL. Kondisi ini merupakan kondisi di mana agen hanya menerima sinyal positif (*reward*) ketika berhasil menyelesaikan tugas secara sempurna, dan tidak menerima *reward* atau mendapat penalti pada kondisi lainnya (He & Lv, 2024). Contohnya pada tugas manipulasi robot untuk memindahkan objek ke posisi target, agen hanya mendapat *reward* +1 jika objek tepat di posisi yang diinginkan, dan 0 di kondisi lain. Hal ini mengakibatkan mayoritas episode tidak memberikan sinyal pembelajaran dan agen tidak tahu apakah tindakannya mendekati atau menjauhi tujuan, sehingga pembelajaran menjadi sangat lambat atau bahkan tidak terjadi sama sekali.

Untuk mengatasi masalah *sparse reward*, banyak penelitian terdahulu mencoba melakukan rekayasa fungsi *reward* (*reward engineering*) dengan menambahkan sinyal tambahan yang memberikan umpan balik lebih sering. Namun, praktik *reward engineering* ini memiliki beberapa kelemahan sehingga menimbulkan permasalahan baru. Pertama, desain fungsi *reward* yang efektif membutuhkan keahlian domain yang tinggi dan pemahaman mendalam terhadap

dinamika lingkungan (Yang dkk., 2021). Kedua, *reward* yang direkayasa sering kali tidak sejalan dengan metrik keberhasilan sebenarnya, karena pembentukannya lebih berorientasi pada kemudahan pembelajaran dibandingkan makna dari tujuan yang diinginkan. Ketiga, penggunaan *reward shaping* yang tidak tepat dapat menimbulkan bias eksplorasi, di mana agen justru terjebak pada perilaku suboptimal akibat terlalu terarah pada sinyal yang salah (Andrychowicz dkk., 2018).

Masalah berikutnya adalah ketidakmampuan agen untuk belajar dari pengalaman gagal. Dalam pendekatan *model-free* konvensional, setiap episode yang tidak mencapai tujuan dianggap tidak berguna karena hanya menghasilkan *reward* nol atau negatif tanpa memberikan petunjuk tentang bagaimana memperbaiki kebijakannya. Hal ini berarti sebagian besar data yang dikumpulkan agen tidak dimanfaatkan dalam proses pembaruan kebijakan. Akibatnya, efisiensi sampel (*sample efficiency*) menjadi sangat rendah karena algoritma hanya belajar dari sebagian kecil episode yang kebetulan berhasil (Andrychowicz dkk., 2018).

Pendekatan HER diperkenalkan oleh Andrychowicz dkk. (2018) sebagai solusi terhadap permasalahan yang sudah disebutkan. HER mengubah paradigma pembelajaran RL dengan memungkinkan agen untuk belajar dari pengalaman gagal melalui proses yang disebut *goal relabeling*. Dengan memanfaatkan kembali setiap pengalaman, HER meningkatkan efisiensi sampel (*sample efficiency*) dan mengurangi kebutuhan akan rekayasa fungsi *reward* (*reward engineering*) (Andrychowicz dkk., 2018).

Hindsight Experience Replay bekerja dengan prinsip *hindsight reasoning*, yaitu kemampuan agen untuk meninjau kembali pengalaman yang gagal dan menafsirkan ulang hasilnya seolah-olah merupakan keberhasilan terhadap tujuan lain. Dalam algoritma RL konvensional, episode yang tidak mencapai tujuan akan dianggap gagal dan tidak memberikan kontribusi terhadap pembaruan kebijakan. Sebaliknya, HER menganggap setiap episode sebagai sumber informasi potensial dengan cara mengganti tujuan awal (*original goal*) dengan tujuan baru (*hindsight goal*) yang sesuai dengan hasil yang sebenarnya dicapai (Andrychowicz dkk., 2018).

Hindsight Experience Replay bekerja dengan memodifikasi mekanisme *experience replay* yang digunakan oleh algoritma *off-policy*. Karena dalam *off-policy learning* distribusi data tidak harus berasal dari kebijakan saat ini, HER dapat mengganti tujuan pada pengalaman yang sama tanpa mengubah dinamika lingkungan. Langkah-langkah utama algoritma HER dapat dijabarkan sebagai berikut (Andrychowicz dkk., 2018).

1. *Episode Generation*

   Agen menjalankan episode menggunakan kebijakan perilaku $\pi(\cdot \mid g)$, yang dikondisikan pada tujuan asli g, menghasilkan urutan keadaan $s_0, s_1, \ldots, s_T$.

2. *Standard Replay*

   Setiap transisi (s_t, a_t, s_{t+1}) dicatat ke dalam *replay buffer* dengan pasangan tujuan asli dan *reward* terkait $(s_t \| g, a_t, r_t, s_{t+1} \| g)$

3. *Hindsight Replay*

   Untuk setiap episode, sekumpulan tujuan tambahan G' dipilih berdasarkan strategi pengambilan sampel tertentu.

4. *New Transition Recording*

   Untuk setiap tujuan pengganti $g' \in G'$, *reward* baru r' dihitung berdasarkan apakah tujuan g' tercapai dalam transisi tersebut. Transisi baru kemudian disimpan sebagai $(s_t \| g', a_t, r'_t, s_{t+1} \| g')$

5. *Optimization*

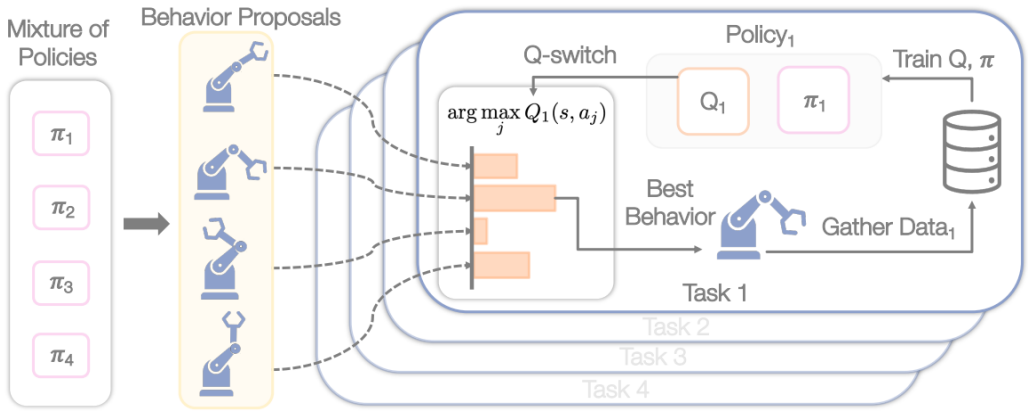
   Algoritma RL yang digunakan kemudian mengekstraksi *minibatch* acak dari buffer R, yang kini berisi campuran pengalaman asli dan *hindsight*, untuk memperbarui parameter kebijakan dan fungsi nilai.

2.5 *Q-Switch Mixture of Primitives Hindsight Experience Replay*

Metode *Q-Switch Mixture of Primitives Hindsight Experience Replay* (QMP-HER) merupakan hasil pengembangan dari metode Q-*Switch Mixture of Policies* (QMP) yang diperkenalkan oleh Zhang dkk. (2023) dalam penelitian berjudul "*Efficient Multi-Task Reinforcement Learning via Selective Behavior Sharing*". QMP dikembangkan untuk mengatasi keterbatasan efisiensi pembelajaran pada *multi-task Reinforcement Learning* dengan memperkenalkan

mekanisme berbagi perilaku (*behavior sharing*) antar-tugas melalui proses seleksi tindakan berbasis nilai Q (*Q-value*).

Dalam metode tersebut, agen memiliki beberapa kebijakan dari berbagai tugas yang berbeda, dan pada setiap langkah waktu (*timestep*), agen dapat memilih tindakan dari kebijakan mana yang akan digunakan berdasarkan nilai Q tertinggi yang dihasilkan oleh *critic network* pada tugas yang sedang dijalankan. Proses pemilihan ini disebut sebagai *Q-Switch*, karena kebijakan aktif yang digunakan dapat berganti-ganti (*switch*) secara adaptif sesuai evaluasi nilai Q. Pendekatan ini memungkinkan suatu kebijakan memanfaatkan perilaku yang telah terbukti efektif dalam tugas lain, sehingga meningkatkan efisiensi pembelajaran dan mempercepat konvergensi pada lingkungan multi-tugas (G. Zhang dkk., 2023). Berikut ilustrasi cara kerja *QMP* yang disajikan pada Gambar 2.7.



Gambar 2.7 Ilustrasi Cara Kerja Metode QMP

(Sumber: G. Zhang dkk., 2023)

Prinsip dasar *QMP* tersebut kemudian diadaptasi oleh Sanchez dkk. (2023) dalam penelitian berjudul "*Learning and Reusing Primitive Behaviours to Improve Hindsight Experience Replay Sample Efficiency*", yang menghasilkan metode baru bernama *Q-Switch Mixture of Primitives Hindsight Experience Replay* (QMP-HER). Metode ini bertujuan untuk meningkatkan efisiensi sampel dan mempercepat proses pembelajaran yang menggunakan pendekatan *Hindsight Experience Replay* (HER). Jika QMP digunakan untuk berbagi perilaku antar tugas dalam konteks *multi-task learning*, maka QMP-HER menggunakan konsep serupa untuk berbagi perilaku antar *primitive* dalam satu tugas yang kompleks. QMP-HER

tidak membagi perilaku antar-tugas yang berbeda, melainkan memanfaatkan kembali keterampilan dasar yang telah dipelajari sebelumnya (*primitive policies*) untuk membantu pembelajaran tugas yang lebih sulit (Sanchez dkk., 2023).

QMP-HER mengombinasikan tiga komponen utama, yaitu kebijakan target (*target policy*), sekumpulan kebijakan primitif (*primitive policies*), dan *critic network* yang bertugas menilai kualitas tindakan berdasarkan nilai Q. Pada setiap langkah pelatihan, baik kebijakan target maupun kebijakan primitif menghasilkan sejumlah tindakan kandidat (*candidate actions*) untuk kondisi lingkungan saat itu. Semua kandidat tindakan tersebut kemudian dievaluasi oleh *critic network* dari kebijakan target berdasarkan nilai Q yang dihasilkan. Tindakan dengan nilai Q tertinggi dipilih dan dieksekusi di lingkungan (Sanchez dkk., 2023). Mekanisme pemilihan tindakan ini dirumuskan pada persamaan (2.20).

$$a^* = \arg\max_{a \in A_t} Q^{\pi}(s_t, a) \tag{2.20}$$

Persamaan di atas menunjukkan bahwa tindakan optimal a^* pada langkah waktu t adalah tindakan yang memberikan nilai Q tertinggi terhadap keadaan saat ini s_t, berdasarkan fungsi *critic* dari kebijakan target Q^{π}.

Setelah tindakan terbaik dipilih dan dijalankan, transisi yang terjadi $(s_t, a_t, s_{t+1}, g, r_t)$ disimpan ke dalam *replay buffer*. Selanjutnya, logika HER tetap digunakan untuk memodifikasi pengalaman gagal menjadi pengalaman sukses melalui proses *goal relabeling*, sehingga agen tetap dapat belajar dari seluruh pengalaman yang diperoleh. Pada fase awal pelatihan, karena nilai Q yang diprediksi masih acak, agen diberi kesempatan untuk mengeksplorasi dengan mengikuti saran kebijakan primitif melalui pendekatan *ε-greedy*. Seiring meningkatnya kemampuan kebijakan target, tindakan yang dihasilkan oleh kebijakan target sendiri akan semakin sering memiliki nilai Q tertinggi, sehingga sistem secara alami kembali berperilaku seperti HER murni pada tahap akhir pelatihan. Berikut algoritma QMP-HER dalam bentuk *pseudocode* berdasarkan sumber penelitian Sanchez dkk. (2023) yang disajikan pada Gambar 2.8. Pada baris 1 – 25 merupakan algoritma QMP biasa, lalu baris setelahnya merupakan kombinasi dengan HER.

Algoritma QMP-HER

Diberikan:
  Agen off-policy RL $A \sim (\pi(s,g), Q^{\pi}(s,g,a))$
  K kebijakan primitif: $P = \{\pi_{b_1}(s,g), \pi_{b_2}(s,g), \ldots, \pi_{b_K}(s,g)\}$
  Replay buffer $\mathcal{R}$
Inisialisasi agen A
Muat kebijakan primitif P
Tentukan himpunan L tujuan $O = \{o_1, o_2, \ldots, o_L\}$ untuk P
for epoch $= 0$ hingga N **do**
  for episode $= 1$ hingga M **do**
    Ambil goal g dan keadaan awal s_0
    for $t = 0$ hingga $T - 1$ **do**
      for $k = 1$ hingga K **do**
        for $l = 1$ hingga L **do**
          $(s', g') \leftarrow$ sesuaikan state dan/atau goal berdasarkan o_l
          Ambil aksi kandidat $a_t^{k,l} \sim \pi_{b_k}(s', g')$
          Tambahkan $a_t^{k,l}$ ke dalam himpunan kandidat A_t
        end for
      end for
      Ambil aksi kandidat $a_t^{\pi} \sim \pi(s_t, g)$
      Tambahkan a_t^{π} ke dalam A_t
      Pilih aksi terbaik a_t^*, dengan penerapan ϵ-greedy
      Eksekusi a_t^* dan amati s_{t+1} serta r_t
      Simpan transisi $(s_t, a_t^*, s_{t+1}, g, r_t)$ ke dalam $\mathcal{R}$
    end for
  end for
  Terapkan logika HER dan perbarui $\pi(s,g)$ serta $Q^{\pi}(s,g,a)$
end for

Gambar 2.8 *Pseudocode* Algoritma QMP-HER

Melalui kombinasi ini, QMP-HER berhasil mengarahkan eksplorasi agen ke tindakan yang lebih bermakna, mempercepat konvergensi, dan menghemat waktu komputasi secara signifikan. QMP-HER menghadirkan paradigma baru dalam pembelajaran robotik berbasis RL, di mana pengetahuan yang telah dimiliki oleh agen pada tugas-tugas sederhana dapat dimanfaatkan kembali secara adaptif untuk mempercepat proses pembelajaran pada tugas yang lebih kompleks.

2.6 *Sim-to-Real Transfer*

Sim-to-Real Transfer mengacu pada metode yang digunakan untuk mentransfer model atau kebijakan (*policy*), yang biasanya dipelajari menggunakan algoritma *Reinforcement Learning* di lingkungan simulasi, untuk kemudian diterapkan dengan sukses pada sistem robotik di dunia nyata (W. Liu dkk., 2023). Tujuan utama dari pendekatan ini adalah mentransfer pengetahuan yang telah diperoleh selama pelatihan di lingkungan simulasi serta menyesuaikan kebijakan

yang dihasilkan agar mampu berfungsi secara efektif dan efisien di kondisi dunia nyata (Chen dkk., 2021).

Penggunaan simulasi dalam pelatihan, khususnya robot dengan algoritma RL, didorong oleh beberapa alasan. Pertama, efisiensi biaya dan waktu menjadi alasan utama, karena proses pelatihan berbasis *trial and error* di dunia nyata sering kali memerlukan sumber daya besar dan waktu yang panjang. Lingkungan simulasi memungkinkan pengumpulan data dalam jumlah besar dengan biaya yang relatif rendah (Shakerimov dkk., 2023). Kedua, ketersediaan data yang hampir tak terbatas menjadikan simulasi sebagai platform ideal untuk eksperimen berskala besar. Agen dapat mengeksplorasi berbagai kondisi tanpa batasan fisik maupun risiko kerusakan perangkat keras (Zhao dkk., 2020). Ketiga, keamanan menjadi aspek penting. Melalui simulasi, pengembang dapat menghindari risiko kecelakaan atau kerusakan akibat tindakan robot yang belum stabil selama proses pembelajaran (Zhao dkk., 2020).Dengan demikian, simulasi berperan sebagai laboratorium virtual yang aman dan efisien untuk melatih agen cerdas sebelum diterapkan di dunia nyata.

2.6.1 *Domain Randomization*

Domain Randomization merupakan salah satu pendekatan penting dalam bidang *Sim-to-Real transfer* yang berakar pada teori *robust control under uncertainty* atau kendali tangguh terhadap ketidakpastian (Shakerimov dkk., 2023). Metode ini didasarkan pada gagasan bahwa kebijakan yang dilatih dalam simulasi akan lebih tangguh apabila dilatih di bawah berbagai kondisi yang bervariasi secara acak (Shakerimov dkk., 2023).

Secara konsep, tujuan utama pendekatan ini adalah meningkatkan ketahanan (*robustness*) dan kemampuan generalisasi kebijakan terhadap perbedaan antara lingkungan simulasi dan dunia nyata (Zhao dkk., 2020). Alih-alih berusaha memodelkan setiap parameter dunia nyata dengan presisi tinggi, *Domain Randomization* memanfaatkan variasi acak terhadap parameter-parameter tertentu dalam simulasi untuk memperluas cakupan distribusi pelatihan (Zhao dkk., 2020). Pendekatan ini secara efektif mengurangi dampak *distributional shift* (ketidaksesuaian antara distribusi data *training* dan data *test*) yang sering menjadi

penyebab utama kegagalan kebijakan saat berpindah dari simulasi ke realitas (Shakerimov dkk., 2023).

Dalam implementasinya, metode ini dilakukan dengan mengacak sejumlah parameter fisik maupun visual di dalam simulator. Randomisasi dinamika dilakukan pada aspek-aspek yang berkaitan dengan model fisik, seperti massa objek, koefisien gesekan, atau gaya aktuator. Sementara itu, randomisasi visual diterapkan pada aspek persepsi, seperti pencahayaan, tekstur, warna, posisi kamera, serta gangguan sensorik. Selama proses pelatihan, setiap episode dimulai dengan konfigurasi parameter yang berbeda secara acak, sehingga agen belajar beradaptasi terhadap berbagai variasi yang mungkin ditemui di dunia nyata (Shakerimov dkk., 2023).

2.7 Kalibrasi Kamera

Kalibrasi kamera merupakan proses untuk memperoleh parameter kamera yang digunakan untuk membangun hubungan geometris antara titik pada ruang tiga dimensi dan titik hasil proyeksinya pada citra dua dimensi. Dalam sistem visi komputer, kamera tidak secara langsung menghasilkan koordinat tiga dimensi, melainkan menangkap objek dalam bentuk proyeksi piksel pada bidang citra (Zhou dkk., 2024). Oleh karena itu, kalibrasi kamera diperlukan agar informasi visual yang diperoleh dari kamera dapat digunakan untuk melakukan pengukuran posisi dan orientasi secara lebih akurat (Genovese, 2024).

Dalam aplikasi robotika, kalibrasi kamera menjadi tahap penting karena banyak sistem robot menggunakan informasi visual sebagai masukan untuk mengenali posisi objek, memperkirakan pose, atau mengarahkan gerakan robot. Kalibrasi intrinsik dan ekstrinsik yang akurat diperlukan pada aplikasi robotika yang bergantung pada kamera sebagai sumber persepsi visual (Linder dkk., 2025). Pada sistem robot yang dipandu kamera, kesalahan kalibrasi dapat menyebabkan informasi visual tidak sesuai dengan sistem koordinat robot. Oleh karena itu, kalibrasi kamera tidak hanya berfungsi untuk memperbaiki hasil pengamatan citra, tetapi juga menjadi dasar agar hasil pengukuran visual dapat digunakan dalam proses kendali robot (Ulrich dkk., 2024).

Salah satu pendekatan yang umum digunakan dalam kalibrasi kamera adalah menggunakan target kalibrasi dengan geometri yang telah diketahui, seperti *checkerboard* atau AprilTag *grid*. Target kalibrasi digunakan karena titik-titik pada pola tersebut memiliki susunan geometris yang diketahui, sehingga dapat dicocokkan dengan titik-titik yang terdeteksi pada citra kamera (Linder dkk., 2025). Dalam kalibrasi berbasis *chessboard*, titik sudut pada pola kalibrasi dapat diperlakukan sebagai titik tiga dimensi pada bidang datar, sedangkan titik hasil deteksinya pada citra digunakan sebagai titik dua dimensi (OpenCV, 2016).

Pada pola *chessboard* planar, titik kontrol dapat dinyatakan pada bidang $\boldsymbol{XY}$ dengan nilai $\boldsymbol{Z} = \boldsymbol{0}$ (OpenCV, 2016). Secara umum, titik sudut ke-i, j pada pola *chessboard* dapat dituliskan pada persamaan (2.21) berikut.

$$\boldsymbol{P}_{ij} = \begin{bmatrix} ij \\ jL \\ \boldsymbol{0} \end{bmatrix} \tag{2.22}$$

Pada persamaan tersebut, $\boldsymbol{P}_{ij}$ merupakan titik kontrol pada pola *chessboard* dalam ruang tiga dimensi. Indeks i dan j menunjukkan posisi titik sudut pada arah horizontal dan vertikal pola *chessboard*. Variabel L menunjukkan ukuran sisi kotak pada pola *chessboard* yang digunakan sebagai skala pengukuran dunia nyata. Nilai $\boldsymbol{Z} = \boldsymbol{0}$ digunakan karena pola *chessboard* diasumsikan berada pada bidang datar (OpenCV, 2016).

Secara umum, hubungan antara titik pada ruang tiga dimensi dan titik hasil proyeksinya pada citra dapat dinyatakan menggunakan model proyeksi kamera (Song dkk., 2024). Model proyeksi kamera menunjukkan bahwa titik tiga dimensi diproyeksikan ke bidang citra melalui parameter intrinsik kamera dan parameter ekstrinsik yang menyatakan posisi serta orientasi kamera terhadap sistem koordinat luar (Song dkk., 2024). Hubungan tersebut dapat dituliskan pada persamaan (2.22) berikut.

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{K} \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{2.22}$$

Pada persamaan tersebut, u dan v menunjukkan koordinat titik pada citra dalam satuan piksel. Variabel X, Y, dan Z menunjukkan koordinat titik pada ruang tiga

dimensi. Matriks $\boldsymbol{K}$ menunjukkan parameter intrinsik kamera, sedangkan $\boldsymbol{R}$ dan $\boldsymbol{t}$ menunjukkan parameter ekstrinsik berupa rotasi dan translasi kamera terhadap sistem koordinat luar. Faktor s merupakan faktor skala pada koordinat homogen yang digunakan untuk menghubungkan koordinat ruang dengan koordinat citra (Song dkk., 2024).

Parameter intrinsik kamera menggambarkan karakteristik internal kamera, seperti panjang fokus dan titik pusat optik kamera. Parameter intrinsik berperan penting karena menentukan bagaimana titik pada ruang tiga dimensi diproyeksikan ke bidang citra (Hagemann dkk., 2023). Akurasi parameter intrinsik juga memengaruhi akurasi estimasi pose kamera maupun objek, sehingga parameter ini penting untuk aplikasi seperti *vision servoing*, navigasi robot, dan perakitan otomatis (Genovese, 2024). Matriks intrinsik kamera dapat dinyatakan dalam bentuk matriks $\boldsymbol{K}$ yang memuat panjang fokus dan titik pusat optik kamera (Song dkk., 2024). Bentuk matriks intrinsik kamera dapat dituliskan pada persamaan (2.23) berikut.

$$\boldsymbol{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{2.23}$$

Pada persamaan tersebut, f_x dan f_y menunjukkan panjang fokus kamera pada sumbu horizontal dan vertikal dalam satuan piksel (Song dkk., 2024). Variabel c_x dan c_y menunjukkan koordinat titik pusat optik atau *principal point* pada citra (Song dkk., 2024). Matriks intrinsik tersebut merupakan salah satu hasil utama dari proses kalibrasi kamera dan dapat digunakan kembali selama konfigurasi kamera tidak berubah (OpenCV, 2016).

Sedangkan parameter ekstrinsik menjelaskan hubungan posisi dan orientasi kamera terhadap sistem koordinat lain, seperti sistem koordinat dunia atau sistem koordinat robot. Dalam konteks robotika, parameter ekstrinsik diperlukan agar data visual dari kamera dapat dihubungkan dengan posisi fisik robot atau objek pada lingkungan nyata. Dengan demikian, parameter ekstrinsik memiliki peran penting ketika hasil deteksi kamera perlu digunakan untuk menentukan posisi objek terhadap robot (Ulrich dkk., 2024).

Selain parameter intrinsik dan ekstrinsik, kalibrasi kamera juga berkaitan dengan estimasi koefisien distorsi lensa. Distorsi lensa dapat menyebabkan posisi titik pada citra tidak sepenuhnya sesuai dengan model proyeksi. Kesalahan pada model distorsi dapat memengaruhi hasil estimasi parameter intrinsik maupun hasil estimasi pose. Oleh karena itu, koreksi distorsi diperlukan agar posisi titik pada citra dapat digunakan sebagai dasar pengukuran visual yang lebih akurat (Genovese, 2024).

Dalam model distorsi yang umum digunakan pada OpenCV, koefisien distorsi terdiri atas komponen distorsi radial dan distorsi tangensial (OpenCV, 2016). Koefisien distorsi tersebut dapat dinyatakan pada persamaan (2.24) berikut.

$$\boldsymbol{D} = [k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3] \tag{2.24}$$

Pada persamaan tersebut, k_1, k_2, dan k_3 merupakan koefisien distorsi radial. Sementara itu, p_1 dan p_2 merupakan koefisien distorsi tangensial. Koefisien distorsi ini digunakan untuk mengoreksi penyimpangan posisi titik pada citra akibat karakteristik lensa kamera (OpenCV, 2016).

Proses kalibrasi kamera dilakukan dengan menggunakan pasangan titik tiga dimensi pada pola kalibrasi dan titik dua dimensi hasil deteksi pada citra. Setelah titik-titik sudut pola *chessboard* terdeteksi, posisi sudut tersebut dapat diperhalus untuk meningkatkan akurasi titik citra yang digunakan dalam proses kalibrasi. Selanjutnya, parameter kamera dihitung untuk memperoleh matriks intrinsik kamera, koefisien distorsi, vektor rotasi, dan vektor translasi (OpenCV, 2016).

Kualitas parameter kamera hasil kalibrasi dapat dievaluasi menggunakan *reprojection error* (Linder dkk., 2025). *Reprojection error* menunjukkan selisih antara titik sudut yang terdeteksi pada citra dan titik hasil proyeksi ulang berdasarkan parameter kamera hasil kalibrasi. Semakin kecil nilai *reprojection error*, semakin baik kesesuaian antara model kamera dan hasil pengamatan pada citra (OpenCV, 2016). Secara matematis, kesalahan proyeksi ulang untuk titik ke- i dapat dituliskan pada persamaan (2.25) berikut.

$$e_i = ||\boldsymbol{m_i} - \widehat{\boldsymbol{m_i}}|| \tag{2.25}$$

Pada persamaan tersebut, $\boldsymbol{m_i}$ merupakan titik hasil deteksi pada citra, sedangkan $\widehat{\boldsymbol{m_i}}$ merupakan titik hasil proyeksi ulang menggunakan parameter kamera hasil

kalibrasi. Nilai e_i menunjukkan besar selisih antara titik hasil deteksi dan titik hasil proyeksi ulang pada citra (OpenCV, 2016).

Jika terdapat N titik sudut yang digunakan dalam proses kalibrasi, maka nilai *root mean square error* atau RMSE dari *reprojection error* dapat dinyatakan sebagai persamaan (2.26) berikut.

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} {e_i}^2} \tag{2.26}$$

Pada persamaan tersebut, N menunjukkan jumlah seluruh titik sudut yang digunakan dalam evaluasi *reprojection error*. Nilai RMSE digunakan untuk menunjukkan rata-rata kesalahan proyeksi ulang dalam satuan piksel. Dengan demikian, RMSE dapat digunakan sebagai indikator kualitas hasil kalibrasi kamera (OpenCV, 2016).

Berdasarkan uraian tersebut, kalibrasi kamera dapat dipahami sebagai tahap dasar dalam sistem visi robot karena menjadi penghubung antara data citra dan koordinat ruang yang dibutuhkan oleh robot. Dalam penelitian ini, hasil kalibrasi kamera mendukung proses estimasi pose objek yang digunakan sebagai masukan bagi sistem kendali bagi robot.

2.8 AprilTag

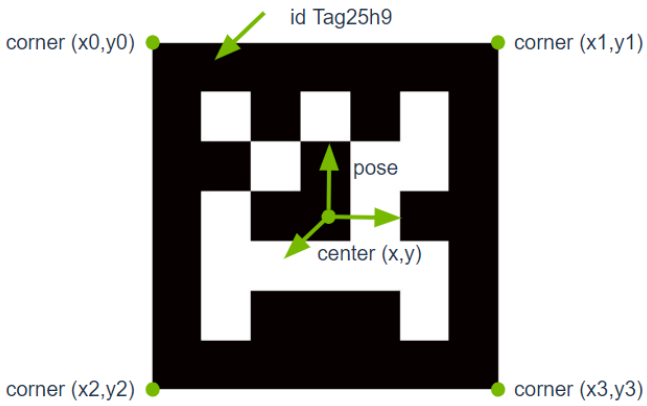
Fiducial marker merupakan fitur visual buatan yang dirancang secara khusus untuk mendukung proses deteksi otomatis dan berfungsi sebagai titik acuan atau alat ukur dalam suatu citra (Olson, 2011). Dalam robotika, *Fiducial marker* berguna karena mampu memberikan pose relatif terhadap kamera dengan akurasi tinggi (Olson, 2011). Dari satu *marker* dapat diperoleh estimasi 6 derajat kebebasan (*DOF*), yang memungkinkan penerapan dalam sistem yang memerlukan data lokalisasi presisi (J. Wang & Olson, 2016).

Penelitian oleh Olson (2011) memperkenalkan AprilTag, sebuah sistem *Fiducial marker* yang tangguh dan fleksibel karena memiliki ketahanan terhadap rotasi, pencahayaan buruk, dan distorsi perspektif. Secara fisik, AprilTag berbentuk persegi hitam-putih dengan muatan biner terenkode, mirip dengan konsep sistem kode batang dua dimensi (*2D barcode*). Berbeda dari *QR code* yang mampu

mengirimkan sejumlah besar data, AprilTag hanya membawa muatan informasi kecil, sekitar 12 bit (Olson, 2011).

Kerangka kerja AprilTag terdiri atas dua komponen utama, yakni pendeteksi *tag* (*tag detector*) yang berfungsi mengidentifikasi kandidat *tag* serta memperkirakan posisinya, dan sistem pengkodean (*coding system*) yang bertugas mendekode identitas *tag* dan memvalidasinya (Olson, 2011). Kedua komponen ini bekerja secara terpadu untuk memungkinkan deteksi, lokalisasi, serta identifikasi penanda fidusial dengan tingkat ketelitian tinggi.

Sistem AprilTag terus dikembangkan untuk meningkatkan efisiensi dan ketahanannya dalam berbagai kondisi lingkungan. Versi lanjutannya, AprilTag 2, memperkenalkan algoritma deteksi baru yang mampu meningkatkan kecepatan pemrosesan serta mengurangi tingkat kesalahan deteksi (*false positive*) tanpa mengorbankan akurasi (J. Wang & Olson, 2016). Dengan demikian, sistem ini semakin relevan untuk aplikasi dunia nyata seperti robotika, *augmented reality*, dan navigasi visual yang memerlukan ketepatan serta kecepatan tinggi dalam pelacakan posisi objek. Berikut ini ilustrasi AprilTag yang disajikan pada Gambar 2.9.



Gambar 2.9 Ilustrasi AprilTag

(Sumber: https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_apriltag (2025))

2.9 Metrik Evaluasi

Evaluasi kinerja kebijakan dalam *Reinforcement Learning* bertujuan untuk menilai seberapa baik agen dalam mencapai tujuan yang diharapkan, baik dari sisi efektivitas, efisiensi, maupun stabilitas performa. Beragam metrik digunakan untuk menggambarkan kualitas kebijakan hasil pelatihan, mulai dari tingkat keberhasilan

hingga kemampuan adaptasi terhadap lingkungan nyata. Bagian berikut menguraikan beberapa metrik utama yang umum digunakan dalam penelitian *Reinforcement Learning*, khususnya pada konteks robotika dan manipulasi.

2.9.1 Tingkat Keberhasilan

Metrik tingkat keberhasilan digunakan untuk menilai efektivitas kebijakan dalam mencapai kondisi tujuan yang telah ditentukan (Xu dkk., 2025). Metrik ini banyak digunakan pada berbagai studi *Reinforcement Learning*. Sebagai contoh, penelitian yang dilakukan oleh X. Liu dkk. (2025) menggunakan metrik tingkat keberhasilan (*success rate*) untuk mengevaluasi performa kebijakan pembelajaran dalam tugas *peg-in-hole assembly*, dengan cara mengukur persentase keberhasilan penyisipan peg ke dalam lubang. Dalam penelitian yang dilakukan penulis, tingkat keberhasilan mengindikasikan seberapa sering robot berhasil memasukkan objek ke lubang target. Nilainya dihitung berdasarkan rasio jumlah episode yang berhasil mencapai *goal state* terhadap total episode yang dijalankan, sebagaimana diformulasikan dalam persamaan (2.27) (X. Liu dkk., 2025).

$$\text{Tingkat Keberhasilan} = \frac{N_s}{N_t} \times 100\% \tag{2.27}$$

di mana N_s adalah jumlah episode yang berhasil mencapai tujuan, dan N_t adalah jumlah total episode evaluasi.

Nilai tingkat keberhasilan yang tinggi menunjukkan bahwa kebijakan mampu menghasilkan aksi yang konsisten dan relevan terhadap kondisi lingkungan yang bervariasi. Oleh karena itu, metrik ini menjadi indikator utama dalam menilai tingkat keberhasilan pelatihan kebijakan *Reinforcement Learning*.

2.9.2 Waktu Penyelesaian

Metrik waktu penyelesaian digunakan untuk menilai tingkat kecepatan kebijakan dalam mengeksekusi suatu tugas hingga selesai. Metrik ini mengukur durasi total yang dibutuhkan untuk menyelesaikan suatu episode hingga mencapai kondisi tujuan. Semakin singkat waktu penyelesaian, semakin baik lintasan dan keputusan aksi yang dihasilkan oleh kebijakan.

Sebagai contoh, pada penelitian yang dilakukan oleh Tang dkk. (2025), waktu penyelesaian digunakan sebagai salah satu indikator utama untuk

mengevaluasi kinerja kebijakan *Reinforcement Learning* berbasis *Soft Actor–Critic* (*SAC*). Dalam penelitian tersebut, waktu penyelesaian tugas *peg-in-hole* diukur berdasarkan jumlah langkah (*step*) penyesuaian yang dilakukan robot hingga proses perakitan berhasil diselesaikan.

Dalam penelitian yang dilakukan penulis, metrik waktu penyelesaian digunakan untuk mengukur seberapa cepat kebijakan yang dihasilkan mampu menuntaskan satu episode tugas hingga kondisi tujuan tercapai berdasarkan jumlah langkah yang diambil.

2.9.3 Efisiensi Sampel

Metrik efisiensi sampel digunakan untuk menilai seberapa cepat model kebijakan mencapai performa optimal dengan jumlah data pelatihan atau interaksi yang terbatas (Sanchez dkk., 2023). Metrik ini menjadi penting karena setiap episode pelatihan umumnya memerlukan waktu dan sumber daya fisik yang signifikan, terutama ketika diterapkan pada sistem nyata.

Pada penelitian yang dilakukan oleh Andrychowicz dkk. (2018), efisiensi sampel dievaluasi berdasarkan kurva tingkat keberhasilan terhadap jumlah episode atau *epoch* pelatihan. Sementara itu, penelitian yang dilakukan oleh Sanchez dkk. (2023) efisiensi sampel dievaluasi menggunakan kurva tingkat keberhasilan terhadap jumlah pembaruan kebijakan (*policy updates*), serta memperhitungkan waktu komputasi per pembaruan.

Dalam penelitian ini, metrik efisiensi sampel akan diukur dengan mengamati seberapa cepat model mencapai *reward* tertentu terhadap jumlah episode pelatihan yang telah dijalankan. Penilaian dilakukan dengan memantau grafik *reward* dari model pada setiap fase pelatihan. Semakin sedikit jumlah episode atau interaksi yang diperlukan untuk mendapatkan model yang dapat menyelesaikan tugasnya, maka semakin tinggi tingkat efisiensi sampel dari model yang dikembangkan.

2.9.4 Analisis Trajektori *End-Effector*

Analisis trajektori *end-effector* digunakan sebagai metrik evaluasi untuk menilai stabilitas pergerakan robot manipulator selama melakukan tugas. Pada penelitian yang dilakukan oleh X. Liu dkk. (2025) evaluasi trajektori digunakan

untuk menilai kemampuan kebijakan hasil pembelajaran penguatan dalam menghasilkan lintasan gerak yang stabil dan presisi pada tugas *peg-in-hole assembly*. Penelitian tersebut melakukan evaluasi dengan membandingkan lintasan hasil simulasi dan lintasan aktual pada robot UR10e berdasarkan grafik posisi tiap *joint* terhadap waktu.

Pada penelitian yang dilakukan penulis, evaluasi ini berfokus pada pola lintasan yang dihasilkan oleh *end-effector* dalam ruang koordinat (x, y, z) sehingga dapat diketahui seberapa baik model mampu mengikuti jalur target, serta meminimalkan penyimpangan selama pergerakan.

2.10 Penelitian Terdahulu

Pada bagian ini ditampilkan beberapa penelitian terdahulu yang relevan yang dapat dilihat pada Tabel 2.2. Penelitian-penelitian tersebut digunakan sebagai bahan acuan untuk memahami perkembangan penelitian yang telah dilakukan pada topik yang serupa,

Tabel 2.2 Penelitian Terdahulu

No	Penulis	Judul	Fokus Penelitian	Metode	Hasil
1	X. Liu dkk. (2025)	*A General Peg-in-Hole Assembly Policy Based on Domain Randomized Reinforcement Learning*	Meningkatkan kemampuan generalisasi kebijakan (*policy*) tugas *peg-in-hole* hingga 6 derajat kebebasan.	Algoritma *Proximal Policy Optimization* (*PPO*) dan *Domain Randomization* dalam simulasi Isaac Lab.	~100% tingkat keberhasilan dan kapabilitas *Zero-shot Sim2Real* transfer tanpa memerlukan *fine-tuning* tambahan.
2	Xu dkk. (2025)	*Accelerating Visual Reinforcement Learning with Separate Primitive Policy for Peg-in-Hole Tasks*	Percepatan pembelajaran visual *RL* melalui untuk tugas *peg-in-hole*	*Separate Primitive Policy* (*S2P*) berbasis DrQ-v2 dengan dua *policy,* yaitu *location* dan *insertion*.	Tingkat keberhasilan ~100%, serta efisiensi sampel meningkat 20-30%.

(bersambung)

(sambungan)

No	Penulis	Judul	Fokus Penelitian	Metode	Hasil
3	Xia dkk. (2023)	*Application and Evaluation of Soft-Actor Critic Reinforcement Learning in Constrained Trajectory Planning for 6DOF Robotic Manipulators*	Perencanaan lintasan untuk robot manipulator 6-*DOF*.	Menggunakan algoritma *Soft Actor-Critic* (*SAC*) yang dipadukan dengan *Hindsight Experience Replay* (*HER*).	Mencapai 97% keberhasilan mencapai target posisi dan orientasi tanpa tabrakan.
4	Sanchez dkk. (2023)	*Learning and reusing primitive behaviours to improve Hindsight Experience Replay sample efficiency*	Peningkatan efisiensi pembelajaran *RL* melalui perilaku primitif dan mekanisme *HER*.	Mengombinasikan *Q-switch Mixture of Primitives* (*QMP*) dengan *HER* berbasis *DDPG*.	Mempercepat konvergensi hingga 50%, meningkatkan efisiensi sampel 2 kali dari metode biasa, dan mencapai >95% tingkat keberhasilan

Berdasarkan penelitian terdahulu pada Tabel 2.2, tugas manipulasi presisi seperti *peg-in-hole* dan *object insertion* telah banyak dikaji menggunakan pendekatan *Reinforcement Learning*. Penelitian X. Liu dkk. (2025) menunjukkan bahwa PPO dapat digunakan untuk menghasilkan *policy* yang mampu melakukan tugas *peg-in-hole* dengan tingkat keberhasilan tinggi, terutama ketika dikombinasikan dengan strategi peningkatan generalisasi pada simulasi. Namun, PPO merupakan algoritma *on-policy*, sehingga data pengalaman yang diperoleh umumnya tidak digunakan ulang seefisien algoritma *off-policy*. Berbeda dengan itu, penelitian ini menggunakan SAC karena SAC bersifat *off-policy*, dapat memanfaatkan *replay buffer*, dan lebih sesuai untuk dikombinasikan dengan HER pada lingkungan *sparse reward* serta ruang aksi kontinu robot manipulator.

Penelitian Xia dkk. (2023) memperlihatkan bahwa kombinasi SAC dan HER relevan untuk tugas robot manipulator 6-DOF karena mampu membantu pembelajaran pada ruang aksi kontinu dan tujuan berbasis posisi serta orientasi. Akan tetapi, pendekatan SAC+HER masih bergantung pada eksplorasi dari *policy* utama tanpa bantuan *primitive policy*. Di sisi lain, penelitian Xu dkk. (2025) dan Sanchez dkk. (2023) menunjukkan bahwa pemanfaatan *primitive policy* dapat membantu mempercepat pembelajaran pada tugas manipulasi yang kompleks. Sanchez dkk. (2023) secara khusus memperkenalkan QMP-HER untuk meningkatkan efisiensi HER melalui mekanisme *Q-Switch* dan penggunaan *primitive behaviour*.

Berdasarkan sintesis tersebut, penelitian ini mengambil posisi pada adaptasi SAC berbasis QMP-HER untuk tugas penyisipan objek multi-bentuk menggunakan robot manipulator. Berbeda dari penelitian sebelumnya, penelitian ini merancang dua *primitive policy*, yaitu *grasp policy* dan *insert policy*, untuk membantu pelatihan *target policy* pada objek kubus, silinder, dan prisma segitiga.

BAB III
METODE PENELITIAN

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilaksanakan di Laboratorium Teknik Robotika dan Kecerdasan Buatan yang terletak di Gedung Kuliah Bersama Kampus C Universitas Airlangga, yang berlokasi di Jalan Dr. Ir. H. Soekarno, Kecamatan Mulyorejo Kota Surabaya, Provinsi Jawa Timur. Penelitian berlangsung selama 10 bulan mulai dari September 2025 hingga Juni 2026.

3.2 Alat dan Bahan Penelitian

Alat dan bahan yang digunakan antara lain adalah sebagai berikut.

1. Komputer Laboratorium Teknik Robotika dan Kecerdasan Buatan

   Komputer laboratorium digunakan sebagai perangkat utama untuk menjalankan simulasi, melakukan pelatihan algoritma serta memproses data hasil eksperimen. Komputer ini dilengkapi dengan RAM 16 GB dan GPU Nvidia GeForce RTX 3050.
2. Mycobot 280 pi 2023

   Robot manipulator dengan 6 *DOF* ini digunakan sebagai platform fisik untuk menguji hasil pelatihan model di dunia nyata.
3. Python versi 3.14

   Bahasa pemrograman utama yang digunakan dalam penelitian ini. Python dipilih karena memiliki ekosistem pustaka yang luas untuk pengembangan algoritma *Reinforcement Learning*.
4. Kamera Logitech C270 HD Webcam

   Kamera ini digunakan untuk menangkap data visual yang digunakan untuk mendeteksi posisi dan orientasi objek uji dan target pada lingkungan nyata.
5. MuJoCo

   Platform simulasi berbasis fisika realistis yang digunakan untuk melatih dan menguji algoritma dalam lingkungan virtual.
6. Objek Uji berbentuk kubus, silinder, dan prisma segitiga

   Objek-objek ini digunakan sebagai target manipulasi dalam pengujian fisik. Setiap bentuk objek memiliki karakteristik geometri berbeda yang

menantang agen untuk menyesuaikan strategi *grasping* dan *insertion*. Variasi bentuk ini membantu menguji kemampuan generalisasi model terhadap bentuk dan orientasi yang berbeda. Masing-masing objek memiliki dimensi karakteristik 30 mm yang disesuaikan dengan bentuk objek, seperti sisi pada kubus, diameter pada silinder, serta ukuran dasar pada prisma segitiga.

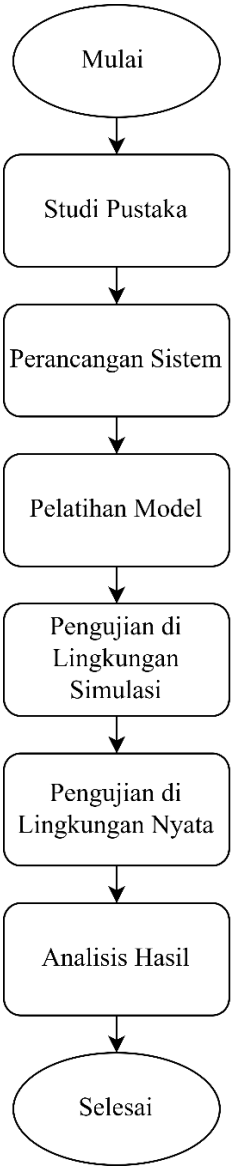
7. *Gripper*

   Komponen yang berfungsi sebagai alat untuk mengambil dan memegang objek uji pada ujung manipulator.

3.3 Metode Penelitian

Penelitian ini menggunakan metode eksperimen berbasis simulasi dan implementasi nyata (*simulation-to-real*). Penelitian ini dimulai dengan tahap studi pustaka yang berfokus pada pengumpulan referensi dan kajian literatur terkait. Selanjutnya, pada tahap perancangan sistem, dilakukan perumusan arsitektur penelitian yang mencakup desain arsitektur sistem, konfigurasi lingkungan eksperimen, serta desain algoritma. Implementasi algoritma kemudian dilakukan dengan menerapkan metode yang telah dirancang ke dalam lingkungan pemrograman yang sesuai.

Setelah itu, dilakukan pelatihan agen di lingkungan simulasi guna memastikan algoritma dapat beradaptasi terhadap berbagai kondisi dan skenario yang mungkin terjadi. Tahap berikutnya adalah pengujian di lingkungan simulasi, yang bertujuan untuk mengevaluasi performa awal sistem sebelum diterapkan pada robot nyata. Selanjutnya, dilakukan pengujian dan validasi pada robot manipulator nyata untuk membandingkan hasil implementasi di dunia nyata dengan hasil yang diperoleh di simulasi. Terakhir, dilakukan analisis hasil guna menilai tingkat keberhasilan metode yang diusulkan serta mengidentifikasi faktor-faktor yang memengaruhi kinerja sistem secara keseluruhan. Secara keseluruhan, alur proses penelitian ditunjukkan pada Gambar 3.1.



Gambar 3.1 Alur Prosedur Penelitian

3.3.1 Studi Pustaka

Tahap ini mencakup kajian literatur, jurnal, dan penelitian terkait *Reinforcement Learning*, algoritma *Soft Actor-Critic*, metode *Hindsight Experience Replay,* serta QMP-HER. Tujuannya adalah memperoleh dasar teori dan pemahaman tentang bagaimana integrasi antara SAC dengan QMP-HER dapat meningkatkan efisiensi pembelajaran robot. Landasan teori yang mendukung tahap ini telah dibahas secara rinci pada Bab 2.

3.3.2 Perancangan Sistem

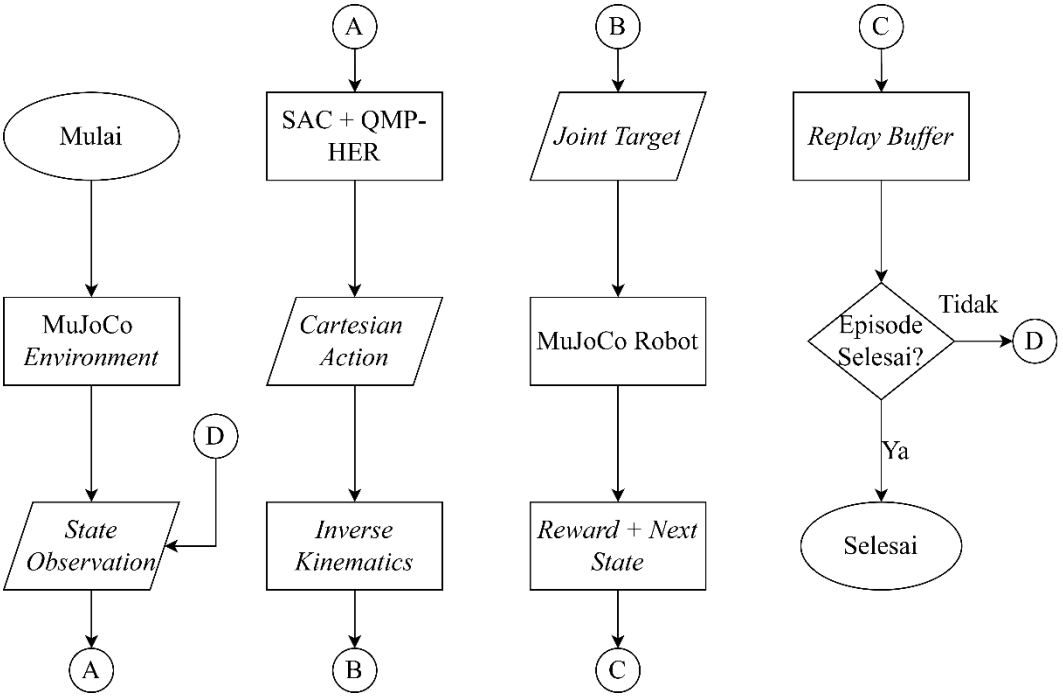
3.3.2.1 Desain Arsitektur Sistem

Arsitektur sistem pada penelitian ini dibedakan menjadi dua alur utama, yaitu arsitektur pelatihan pada lingkungan simulasi dan arsitektur pengujian pada lingkungan nyata. Pemisahan ini dilakukan karena sumber data keadaan, mekanisme eksekusi aksi, dan bentuk umpan balik pada kedua lingkungan memiliki perbedaan. Pada lingkungan simulasi, seluruh informasi keadaan diperoleh langsung dari simulator MuJoCo. Sementara itu, pada lingkungan nyata, informasi posisi dan orientasi awal objek serta target diperoleh melalui kamera dan AprilTag sebelum *trajectory* dijalankan pada robot fisik.

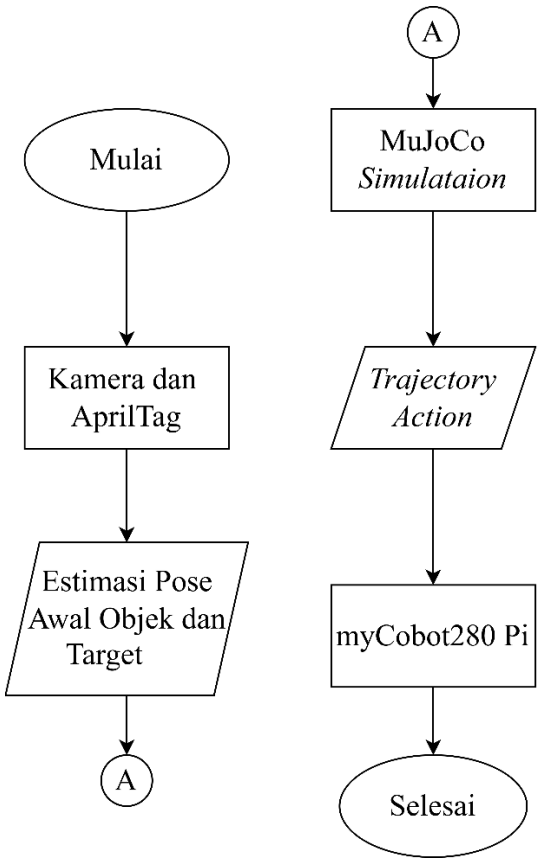
Gambar 3.2 menunjukkan arsitektur pelatihan model pada lingkungan simulasi. Proses dimulai dari MuJoCo Environment yang menghasilkan *state observation* (ruang keadaan) berupa kondisi robot, objek, dan target. *State* tersebut menjadi masukan bagi agen SAC berbasis QMP-HER untuk menghasilkan *Cartesian action*. Aksi ini kemudian dikonversi menjadi *joint target* melalui *inverse kinematics* dan dieksekusi oleh robot simulasi. Setelah aksi dijalankan, lingkungan menghasilkan *reward* dan *next state* yang disimpan ke *replay buffer* untuk proses pembelajaran.

Gambar 3.3 menunjukkan arsitektur pengujian pada lingkungan nyata. Pada tahap ini, kamera dan AprilTag digunakan untuk memperoleh estimasi pose awal objek dan target. Pose tersebut kemudian digunakan pada MuJoCo Simulation untuk menghasilkan *trajectory* sendi menggunakan *target policy* yang telah dilatih. *Trajectory* tersebut selanjutnya dijalankan pada robot myCobot 280 Pi.

Dengan demikian, pengujian nyata pada penelitian ini menggunakan pendekatan *trajectory replay* yang bersifat *open-loop*. Robot belum melakukan koreksi posisi dan orientasi secara *real-time* selama eksekusi, sehingga kesalahan pose awal, *error* transformasi koordinat, pergeseran objek, atau ketidakstabilan *grasp* dapat memengaruhi keberhasilan penyisipan objek.

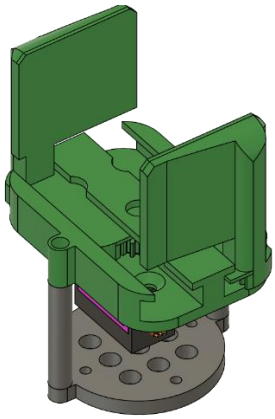


Gambar 3.2 Arsitektur Pelatihan Model pada Lingkungan Simulasi

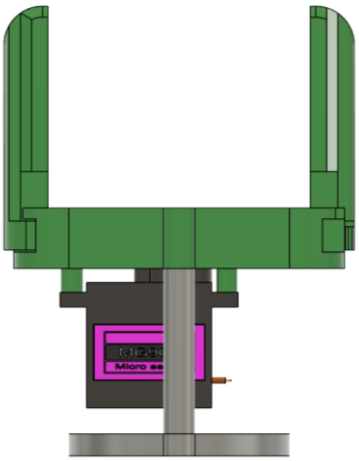


Gambar 3.3 Arsitektur Pengujian Model pada Lingkungan Nyata

Untuk merealisasikan proses pengambilan dan penyisipan objek, sistem menggunakan desain *gripper* yang disesuaikan dengan kompatibilitas MyCobot 280 Pi dan deteksi target berbasis AprilTag. Detail desain ditunjukkan pada Gambar 3.4 dan 3.5 berikut.



Gambar 3.4 Desain Gripper Tampak Isometri

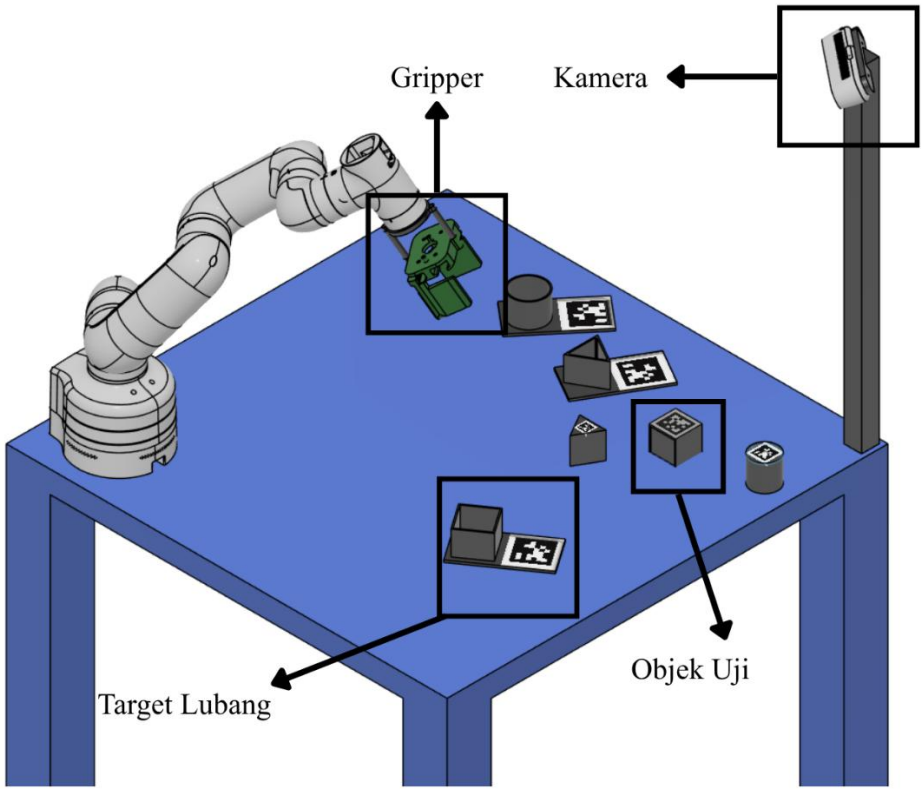


Gambar 3.5 Desain Gripper Tampak Atas

3.3.2.2 Konfigurasi Lingkungan Eksperimen

Lingkungan ini terdiri atas dua bagian yang saling merepresentasikan satu sama lain, yaitu lingkungan simulasi dan lingkungan nyata, yang keduanya memiliki konfigurasi, ukuran, dan tata letak yang sama agar memudahkan proses transfer kebijakan (*sim-to-real transfer*). Pada lingkungan simulasi, sistem dikembangkan menggunakan MuJoCo yang memodelkan robot MyCobot 280. Objek uji berbentuk kubus, silinder, dan prisma segitiga ditempatkan secara acak

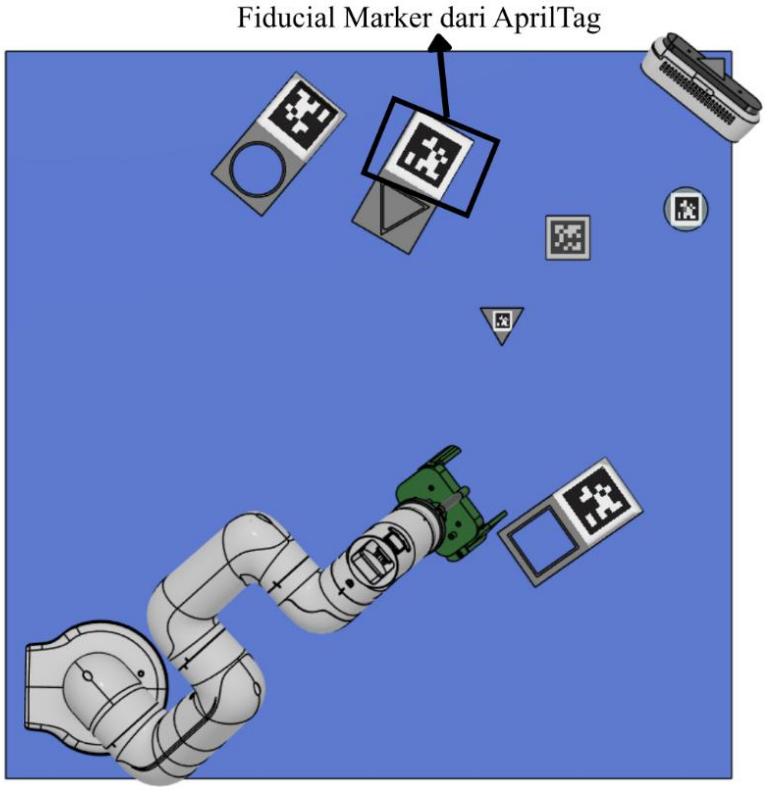
dalam jangkauan kerja *end-effector*, bersama dengan target yang sesuai untuk setiap bentuk. Posisi dan orientasi dari objek dan lubang diperoleh langsung dari simulator melalui API internal tanpa perlu deteksi visual, sehingga memberikan estimasi 6 DOF yang akurat. Lingkungan simulasi ini digunakan sebagai tempat utama untuk pelatihan agen, karena memungkinkan pengumpulan data dalam jumlah besar secara efisien dan aman.



Gambar 3.6 Ilustrasi Isometrik Konfigurasi Lingkungan Eksperimen

Pada lingkungan nyata, konfigurasi fisik dirancang agar menyerupai kondisi simulasi. Robot MyCobot 280 Pi ditempatkan di atas meja, sementara kamera Logitech C270 HD Webcam diletakkan pada posisi yang dapat menangkap objek uji serta lubang target. Setiap objek uji dan lubang dilengkapi dengan AprilTag untuk memperoleh estimasi pose awal objek dan target sebelum *trajectory replay* dijalankan. Hasil deteksi dari kamera kemudian ditransformasikan ke sistem koordinat robot, dalam penelitian ini *base* robot akan ditempelkan juga AprilTag untuk mengestimasi jarak antara objek serta target terhadap *base* robot, sehingga sistem dapat mengenali posisi dan orientasi objek maupun target dengan akurat.

Ilustrasi pada Gambar 3.6 dan 3.7 akan menjelaskan gambaran konfigurasi lingkungan eksperimen yang akan digunakan.

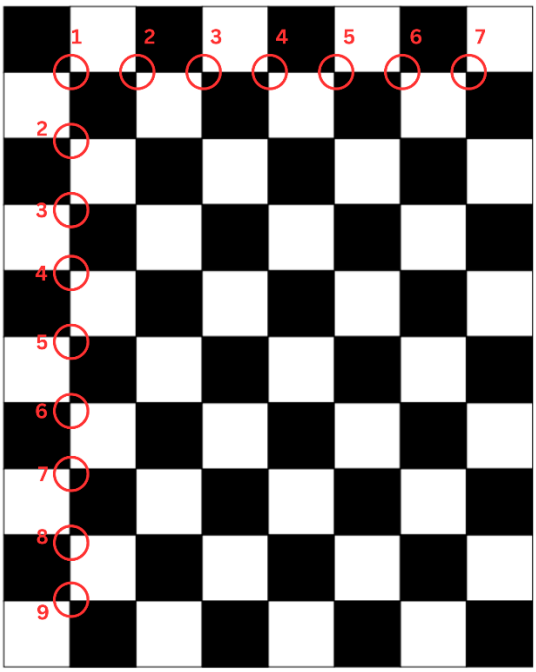


Gambar 3.7 Ilustrasi Tampak Atas Konfigurasi Lingkungan Eksperimen

3.3.2.3 Kalibrasi Kamera dan Transformasi Koordinat AprilTag

Pada pengujian lingkungan nyata, kamera digunakan untuk memperoleh posisi dan orientasi awal objek serta lubang target. Informasi pose tersebut diperlukan agar kondisi awal pada lingkungan nyata dapat direpresentasikan pada lingkungan simulasi MuJoCo sebelum *trajectory replay* dijalankan pada robot. Oleh karena itu, sebelum digunakan untuk deteksi AprilTag, kamera terlebih dahulu dikalibrasi untuk memperoleh parameter intrinsik dan koefisien distorsi.

Kalibrasi dilakukan menggunakan pola *chessboard* dengan 7 x 9 titik sudut dalam (*inner corners*). Ukuran setiap kotak pada pola *chessboard* adalah 20 mm. Pada proses kalibrasi, citra *chessboard* diambil dari beberapa posisi dan sudut pandang. Variasi posisi dan sudut pandang diperlukan agar proses kalibrasi dapat menangkap perbedaan perspektif kamera terhadap pola kalibrasi. Contoh pola *chessboard* yang digunakan dalam penelitian ini ditunjukkan pada Gambar 3.8.



Gambar 3.8 Pola *chessboard* 7x9 yang digunakan untuk kalibrasi kamera

Proses kalibrasi dilakukan dengan mengambil 30 citra *chessboard* dari berbagai posisi dan sudut pandang. Pengambilan citra dari berbagai posisi dilakukan agar proses kalibrasi dapat menangkap variasi perspektif pada bidang gambar. Dengan demikian, parameter kamera yang diperoleh dapat merepresentasikan karakteristik kamera secara lebih baik ketika digunakan untuk mendeteksi pose objek dan lubang target. Hasil dari proses kalibrasi berupa matriks intrinsik kamera dan koefisien distorsi. Matriks intrinsik kamera memuat parameter utama berupa panjang fokus pada sumbu horizontal dan vertikal serta titik pusat optik kamera. Sementara itu, koefisien distorsi digunakan untuk mengoreksi penyimpangan citra yang disebabkan oleh karakteristik lensa kamera. Parameter hasil kalibrasi tersebut disimpan dan digunakan kembali pada proses deteksi AprilTag.

Sebelum citra diproses oleh pendeteksi AprilTag, citra kamera terlebih dahulu dikoreksi menggunakan parameter kalibrasi tersebut untuk mengurangi pengaruh distorsi lensa. Setelah citra dikoreksi, citra kemudian dikonversi ke dalam format *grayscale* dan diproses menggunakan pendeteksi AprilTag. AprilTag yang digunakan pada penelitian ini berasal dari keluarga *tag36h11* dengan ukuran fisik

tag sebesar 0,022 m. Dengan memanfaatkan parameter intrinsik kamera dan ukuran fisik *tag*, sistem dapat mengestimasi pose setiap *tag* yang terdeteksi terhadap kamera.

Agar posisi objek dan lubang target dapat dinyatakan terhadap sistem koordinat robot, penelitian ini menggunakan satu AprilTag sebagai *base tag*. *Base tag* berfungsi sebagai acuan koordinat dunia pada lingkungan nyata. *Tag* ini ditempatkan pada posisi referensi di sekitar basis robot, sehingga pose *tag* lain dapat dihitung relatif terhadap *base tag*, bukan hanya terhadap kamera. Penggunaan *base tag* diperlukan karena sistem koordinat kamera tidak selalu sama dengan sistem koordinat robot.

Secara keseluruhan, tahapan kalibrasi kamera dan transformasi koordinat berbasis AprilTag memungkinkan sistem memperoleh pose awal objek dan lubang target dari kamera, mengubahnya ke sistem koordinat acuan robot, lalu menggunakan pose tersebut sebagai kondisi awal pada simulasi MuJoCo. Pose hasil transformasi ini kemudian digunakan untuk menghasilkan *trajectory joint* yang akan dijalankan pada robot nyata melalui pendekatan *trajectory replay*. Dengan demikian, proses ini menjadi penghubung antara persepsi visual di lingkungan nyata dan eksekusi gerakan robot berbasis hasil simulasi.

3.3.2.4 Desain Algoritma

Algoritma pembelajaran yang digunakan dalam penelitian ini merupakan penggabungan dari dua pendekatan utama, yaitu SAC dan QMP-HER. Kombinasi keduanya dirancang untuk meningkatkan efisiensi pembelajaran dan stabilitas kontrol pada tugas penyisipan objek menggunakan robot manipulator. Pendekatan ini berfokus pada pelatihan kebijakan *end-to-end* yang mampu meniru dan menggabungkan keunggulan dari beberapa kebijakan dasar (*primitive policies*) yang telah dilatih sebelumnya.

Algoritma SAC yang digunakan berperan sebagai fondasi utama yang menyediakan kerangka *Reinforcement Learning* berbasis *maximum entropy*, sehingga agen dapat belajar dengan stabil sekaligus mempertahankan kemampuan eksploratif yang tinggi. Sementara itu, QMP-HER digunakan untuk meningkatkan efisiensi pembelajaran pada tugas yang bersifat kompleks dan memiliki *sparse*

reward. Pada tugas penyisipan objek, keberhasilan hanya dapat diperoleh ketika robot menyisipkan objek dengan benar. Proses ini sulit dipelajari secara langsung apabila agen hanya mengandalkan eksplorasi acak. Oleh karena itu, QMP-HER digunakan untuk memanfaatkan *primitive policy* yang telah dilatih sebelumnya agar eksplorasi *target policy* menjadi lebih terarah.

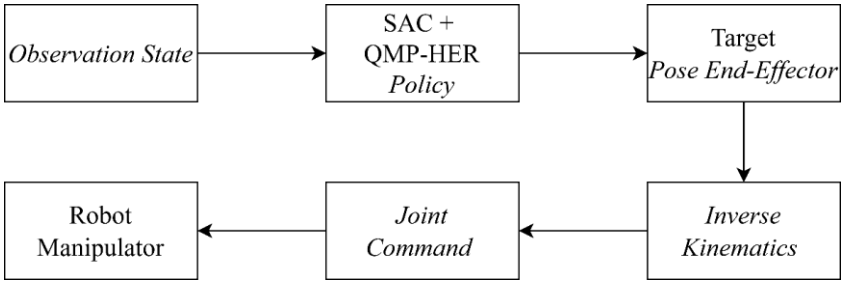
Dalam penelitian ini, dua *primitive policy* dilatih terlebih dahulu menggunakan SAC tanpa mekanisme QMP-HER untuk menangani subtugas yang lebih sederhana, yaitu:

1. *Grasp Policy*, yaitu kebijakan dasar yang bertugas mengarahkan *end-effector* menuju objek, menyesuaikan pose *end-effector* terhadap objek, dan menghasilkan kondisi yang memungkinkan objek dapat digenggam dengan stabil.
2. *Insert Policy*, yaitu kebijakan dasar yang bertugas membawa objek yang telah tergenggam menuju target, menyelaraskan posisi dan orientasi objek terhadap lubang, serta melakukan proses penyisipan objek secara presisi.

Setelah kedua *primitive policy* mencapai konvergensi, dilakukan pelatihan *target policy* menggunakan mekanisme *Q-Switch*. Pada tahap ini, *target policy* dilatih untuk menjadi kebijakan utama yang mampu menggabungkan kemampuan dari *grasp policy* dan *insert policy* secara adaptif. Selama pelatihan, *primitive policy* digunakan sebagai sumber kandidat aksi untuk membantu *target policy* mengeksplorasi tindakan yang lebih bermakna. Namun, pada akhir proses pelatihan, *target policy* menjadi satu-satunya kebijakan yang digunakan untuk mengontrol robot secara langsung tanpa bantuan *primitive policy*.

Pada penelitian ini, ruang aksi tidak dirancang dalam bentuk perubahan sudut sendi secara langsung, melainkan dalam bentuk perubahan pose *end-effector* pada ruang Cartesian. Dengan demikian, *policy* menghasilkan aksi berupa perubahan posisi dan orientasi *end-effector*, kemudian aksi tersebut dikonversi menjadi perintah sudut sendi menggunakan metode *inverse kinematics* sebelum dikirimkan ke aktuator robot. Pendekatan ini dipilih karena tugas penyisipan objek sangat bergantung pada ketepatan posisi dan orientasi *end-effector* terhadap objek

dan lubang target. Secara umum, alur kendali yang digunakan dalam penelitian ini dijelaskan dalam diagram blok yang tertera pada Gambar 3.9



Gambar 3.9 Alur Kendali Cartesian Berbasis *Inverse Kinematics*

Dengan pendekatan ini, agen tidak perlu mempelajari hubungan langsung antara aksi dan sudut sendi robot. Agen cukup mempelajari perubahan pose *end-effector* yang relevan terhadap penyelesaian tugas, sedangkan proses konversi ke ruang sendi dilakukan oleh modul *inverse kinematics*.

Setiap *primitive policy* memiliki ruang keadaan, ruang aksi, dan fungsi *reward* yang dirancang khusus sesuai dengan tujuannya. Perancangan masing – masing *primitive policy* juga *target policy* dijelaskan pada Tabel 3.1 hingga 3.3 berikut.

Tabel 3.1 Perancangan Ruang Keadaan pada *Grasp Policy*

Komponen Keadaan	Dimensi	Keterangan
Posisi Sendi ($\boldsymbol{q}$)	6	Posisi sudut setiap sendi pada robot
Kecepatan Sendi ($\dot{\boldsymbol{q}}$)	6	Kecepatan sudut masing-masing sendi robot
Posisi *End-Effector* ($\mathbf{x}_{ee}$)	3	Posisi kartesian *end-effector* dalam koordinat dunia (x, y, z)
Orientasi *End-Effector* ($\boldsymbol{\theta}_{ee}$)	4	Orientasi *end-effector* dalam bentuk quaternion
Posisi Objek ($\mathbf{x}_{o}$)	3	Posisi objek yang terdeteksi
Orientasi Objek ($\boldsymbol{\theta}_{o}$)	4	Orientasi objek hasil deteksi
Vektor Relatif *End-Effector* dengan Objek ($\mathbf{r}_{eo}$)	3	Selisih posisi antara *end-effector* dan objek ($\mathbf{r}_{eo} = \mathbf{x}_{ee} - \mathbf{x}_{o}$)
Error Orientasi *End-Effector* terhadap Objek ($\boldsymbol{e}_{\boldsymbol{ori}}^{\boldsymbol{eo}}$)	3	Selisih orientasi antara *end-effector* dan objek dalam tiga sumbu rotasi
Status Gripper ($\boldsymbol{g}_{s}$)	1	Status gripper (0 = terbuka, 1 = tertutup)
Total Dimensi	**33**	-

Tabel 3.2 Perancangan Ruang Keadaan pada *Insert* dan *Target Policy*

Komponen Keadaan	Dimensi	Keterangan
Posisi Sendi $(\boldsymbol{q})$	6	Posisi sudut setiap sendi pada robot
Kecepatan Sendi $(\dot{\boldsymbol{q}})$	6	Kecepatan sudut masing-masing sendi robot
Posisi *End-Effector* $(\boldsymbol{x_{ee}})$	3	Posisi Cartesian *end-effector* dalam koordinat dunia x, y, z.
Orientasi *End-Effector* $(\boldsymbol{\theta_{ee}})$	4	Orientasi *end-effector* dalam bentuk quaternion
Posisi Objek $(\boldsymbol{x_o})$	3	Posisi objek yang terdeteksi atau sedang digenggam
Orientasi Objek $(\boldsymbol{\theta_o})$	4	Orientasi objek dalam bentuk quaternion
Posisi *Release* 4 cm di atas Lubang Target $(\boldsymbol{x_h})$	3	Posisi *Release* yang berada 4 cm di atas lubang target
Orientasi Lubang Target $(\boldsymbol{\theta_h})$	4	Orientasi lubang target dalam bentuk quaternion
Vektor Relatif *End-Effector* terhadap Objek $(\boldsymbol{r_{eo}})$	3	Selisih posisi antara *end-effector* dan objek $(\boldsymbol{r_{eo} = x_{ee} - x_o})$
Vektor Relatif Objek terhadap Target $(\boldsymbol{r_{oh}})$	3	Selisih posisi objek terhadap lubang target $(\boldsymbol{r_{oh} = x_o - x_h})$
Error Orientasi *End-Effector* terhadap Objek $(\boldsymbol{e^{eo}_{ori}})$	3	Selisih orientasi antara *end-effector* dan objek dalam tiga sumbu rotasi
Error Orientasi Objek terhadap Target $\left(\boldsymbol{e^{oh}_{ori}}\right)$	3	Selisih orientasi antara objek dan lubang target dalam tiga sumbu rotasi
Status Gripper $(\boldsymbol{g_s})$	1	Status gripper 0 = terbuka, 1 = tertutup.
Total Dimensi	**46**	-

Pada penelitian ini, *insert policy* dan *target policy* menggunakan ruang keadaan yang sama. Hal ini dilakukan karena kedua *policy* membutuhkan informasi lengkap mengenai posisi dan orientasi *end-effector*, objek, serta lubang target. Perbedaan antara *insert policy* dan *target policy* tidak terletak pada ruang keadaan, melainkan pada fungsi *reward* dan tujuan pelatihannya. *insert policy* menggunakan *reward* yang lebih informatif untuk membantu agen membawa objek menuju lubang target, sedangkan *target policy* menggunakan *sparse reward* untuk mempelajari penyelesaian tugas utama dengan bantuan mekanisme QMP-HER.

Tabel 3.3 Perancangan Ruang Aksi pada Semua *Policy*

Komponen Aksi	Dimensi	Keterangan
Δx	1	Perubahan posisi *end-effector* pada sumbu X
Δy	1	Perubahan posisi *end-effector* pada sumbu Y
Δz	1	Perubahan posisi *end-effector* pada sumbu Z
Δroll	1	Perubahan orientasi *end-effector* pada sumbu roll
Δpitch	1	Perubahan orientasi *end-effector* pada sumbu pitch
Δyaw	1	Perubahan orientasi *end-effector* pada sumbu yaw
Total Dimensi	**6**	Aksi Cartesian 6-DOF

Ruang aksi pada semua *policy* dirancang dalam bentuk perubahan pose *end-effector* pada ruang Cartesian 6-DOF. Aksi tersebut digunakan untuk membentuk target pose *end-effector* berikutnya. Target posisi *end-effector* dihitung dengan menambahkan perubahan posisi terhadap pose saat ini, sedangkan target orientasi dihitung berdasarkan perubahan orientasi yang dihasilkan oleh *policy*. Selanjutnya, target pose tersebut dikonversi menjadi sudut sendi menggunakan *inverse kinematics*.

Untuk tiap fungsi *reward* dirancang untuk mengarahkan proses pembelajaran *Reinforcement Learning* agar setiap *policy* dapat mencapai tujuan spesifiknya secara optimal. Setiap *primitive policy* memiliki bentuk fungsi *reward* yang berbeda, menyesuaikan dengan objektif tugas robot. Pada *grasp policy*, tujuan utama agen adalah mengarahkan *end-effector* menuju objek, menyesuaikan orientasi *end-effector* terhadap objek, dan menghasilkan kondisi yang memungkinkan objek dapat digenggam dengan stabil. Karena *gripper* tidak termasuk dalam ruang aksi, proses penutupan *gripper* dilakukan menggunakan aturan terpisah ketika posisi dan orientasi *end-effector* telah memenuhi ambang batas tertentu terhadap objek.

Reward pada *grasp policy* dirancang berdasarkan beberapa komponen utama, yaitu penalti jarak *end-effector* terhadap objek, *reward shaping* untuk memperhalus sinyal pembelajaran jarak, penalti *error* orientasi *end-effector* terhadap objek, bonus keberhasilan, serta penalti kontrol. Dengan demikian, *reward* pada *grasp policy* tidak hanya mendorong *end-effector* agar mendekati objek, tetapi

juga mendorong kesesuaian orientasi dan kestabilan gerakan. Komponen *reward shaping* digunakan untuk memperhalus sinyal pembelajaran, sedangkan bonus keberhasilan digunakan untuk memperkuat kondisi ketika *end-effector* telah memenuhi kriteria *grasping*.

Pada *insert policy*, tujuan utama agen adalah membawa objek yang telah tergenggam menuju posisi *release*, lalu menyisipkan objek ke lubang target. Berbeda dengan *grasp policy* yang berfokus pada hubungan antara *end-effector* dan objek, *insert policy* berfokus pada hubungan antara objek dan target. Setelah objek mencapai kondisi target yang ditentukan, *gripper* dibuka menggunakan aturan berbasis kondisi, bukan melalui aksi langsung dari *policy*. *Reward* pada *insert policy* dirancang berdasarkan penalti jarak objek terhadap target, *reward shaping* untuk memperhalus sinyal pembelajaran jarak, penalti *error* orientasi objek terhadap target, bonus keberhasilan, serta penalti kontrol. Dengan demikian, *reward* pada *insert policy* mendorong objek agar semakin dekat dengan target, orientasinya semakin selaras terhadap lubang target, dan pergerakan yang dihasilkan tetap stabil. Komponen *reward shaping* digunakan untuk memperhalus sinyal pembelajaran, sedangkan bonus keberhasilan diberikan ketika objek telah memenuhi kriteria posisi dan orientasi yang ditentukan.

Terakhir, *target policy* yang dilatih sebagai *policy* utama untuk menyelesaikan tugas secara *end-to-end* dengan bantuan kandidat aksi dari *grasp policy* dan *insert policy* selama proses pelatihan. Berbeda dengan *reward* pada *primitive policy* yang dapat dirancang lebih informatif untuk membantu pembelajaran masing-masing subtugas, *reward* pada *target policy* dirancang dalam bentuk *sparse reward*. Hal ini dilakukan agar *target policy* belajar menyelesaikan tujuan utama secara langsung, yaitu membawa objek menuju target. *Reward* pada *target policy* dirancang secara sederhana dengan memberikan nilai 0 ketika objek berhasil mencapai target, dan nilai -1 ketika objek belum mencapai target.

Dengan rancangan ini, *target policy* tidak memperoleh *reward* bertahap berdasarkan jarak atau orientasi, melainkan hanya memperoleh sinyal keberhasilan ketika objek benar-benar mencapai target yang telah ditentukan. Kondisi ini merepresentasikan permasalahan *sparse reward*, karena agen tidak mendapatkan

informasi *reward* positif secara langsung pada sebagian besar langkah interaksi. Untuk mengatasi keterbatasan tersebut, penelitian ini menggunakan *Hindsight Experience Replay*. HER memungkinkan pengalaman yang awalnya dianggap gagal untuk digunakan kembali dengan mengganti tujuan awal menjadi tujuan lain yang berhasil dicapai oleh agen selama episode berlangsung. Dengan demikian, meskipun *reward target policy* bersifat *sparse*, agen tetap dapat belajar dari pengalaman yang tidak mencapai target utama.

Algoritma SAC-QMP-HER

Diberikan:
  Agen SAC target $(\pi_{\text{target}}, Q_{\text{target}})$
  Kebijakan primitif terlatih $P = \{\pi_{\text{grasp}}, \pi_{\text{insert}}\}$
  Replay buffer $\mathcal{R}$
Inisialisasi parameter SAC target dan replay buffer $\mathcal{R}$
Muat parameter kebijakan primitif π_{grasp} dan π_{insert}
for episode $= 1$ hingga M **do**
  Ambil keadaan awal s_0 dan goal g
  Inisialisasi buffer episode $\tau \leftarrow \emptyset$
  for $t = 0$ hingga $T - 1$ **do**
    Bentuk himpunan kandidat aksi dari kebijakan primitif:

$$\mathcal{A}_t \leftarrow \{\pi_i(s_t, g) \mid \pi_i \in P\}$$

    if $t \geq T_{\text{target}}$ **then**
      Tambahkan aksi target policy ke kandidat:

$$\mathcal{A}_t \leftarrow \mathcal{A}_t \cup \{\pi_{\text{target}}(s_t, g)\}$$

    end if
    Pilih aksi terbaik menggunakan mekanisme Q-switch:

$$a_t^* = \arg\max_{a \in \mathcal{A}_t} Q(s_t, g, a)$$

    Dengan probabilitas ϵ, ganti a_t^* dengan aksi eksplorasi dari $\mathcal{A}_t$
    Eksekusi a_t^*, lalu amati s_{t+1} dan reward r_t
    Simpan transisi $(s_t, a_t^*, s_{t+1}, g, r_t)$ ke buffer episode τ
    $s_t \leftarrow s_{t+1}$
  end for
  Terapkan HER pada τ dan simpan transisi hasil relabeling ke $\mathcal{R}$
  Perbarui actor dan critic SAC target menggunakan sampel dari $\mathcal{R}$
  Lakukan soft update pada target network
end for

Gambar 3.10 *Pseudocode* Algoritma *SAC* dengan pendekatan *QMP–HER*

Secara keseluruhan, proses pelatihan SAC dengan pendekatan QMP–HER berlangsung iteratif. Agen menerima keadaan dari lingkungan simulasi, menghasilkan kandidat aksi dari seluruh *primitive policy* yang sudah dilatih juga target *policy* yang sedang dilatih, memilih aksi dengan nilai Q tertinggi melalui *Q-Switch,* mengeksekusi aksi tersebut, menerima *reward*, dan menyimpan transisi ke

dalam *replay buffer*. Data pada *replay buffer* kemudian diolah oleh mekanisme HER untuk memperkaya variasi pengalaman. Parameter jaringan *actor* dan *critic* diperbarui secara periodik menggunakan optimasi gradien stokastik hingga konvergensi tercapai.

Algoritma integrasi SAC dengan pendekatan QMP–HER disajikan dalam bentuk *pseudocode* pada Gambar 3.10 di atas berdasarkan acuan dari tinjauan pustaka pada Subbab 2.5, yang sebelumnya menjelaskan penggabungan antara DDPG dengan QMP-HER. Perlu dicatat bahwa algoritma integrasi yang ditampilkan pada tahap ini belum melalui proses pengujian, sehingga masih bersifat konseptual dan dapat mengalami penyesuaian atau perubahan pada saat tahap implementasi dilakukan.

3.3.3 Pelatihan Model

Proses pelatihan model dilakukan di lingkungan simulasi menggunakan MuJoCo, dengan robot MyCobot 280 Pi sebagai agen utama. Pelatihan model dilakukan secara bertahap untuk memastikan stabilitas pembelajaran dan kemampuan agen dalam menyelesaikan tugas. Proses pelatihan dibagi menjadi dua tahap, yaitu pelatihan masing – masing *primitive policy* menggunakan algoritma SAC tanpa mekanisme QMP-HER, setelah itu pelatihan *target policy* dengan pendekatan *Q-Switch Mixture of Primitives Hindsight Experience Replay* yang mengombinasikan seluruh *primitive policy* yang sudah dilatih. Seluruh proses pelatihan diimplementasikan dengan gymnasium, yaitu *framework Reinforcement Learning* yang terintegrasi langsung dengan fisika simulasi MuJoCo.

Pada penelitian ini, istilah *timestep* merujuk pada satu langkah interaksi antara agen dan lingkungan, yaitu ketika agen mengamati *state*, memilih *action*, menerima *reward*, dan memperoleh *next state*. Sementara itu, *episode* merujuk pada satu percobaan lengkap yang terdiri atas beberapa *timestep*, dimulai dari kondisi awal lingkungan hingga tugas berhasil diselesaikan atau mencapai batas maksimum *step*.

Pada tahap pertama, dua *primitive policy* dilatih secara terpisah untuk menyelesaikan subtugas spesifik. Hasil pelatihan berupa parameter jaringan yang disimpan sebagai kebijakan tetap (*frozen policy*) yang akan digunakan pada tahap

berikutnya. Pada tahap kedua, dilakukan pelatihan *target policy*, yaitu kebijakan utama yang secara *end-to-end* belajar meniru dan menggabungkan perilaku seluruh *primitive policies* menjadi satu kebijakan tunggal yang efisien. Pada setiap langkah interaksi (*step*) dengan lingkungan, semua *primitive policies* menghasilkan kandidat aksi berdasarkan keadaan sistem saat ini. Seluruh kandidat aksi tersebut kemudian dievaluasi oleh jaringan *critic* dari target *policy* untuk menghitung nilai $Q(s_t, a_i)$. Aksi dengan nilai Q tertinggi dipilih melalui mekanisme *Q-Switch* menggunakan formula pada persamaan (2.20).

Aksi terpilih kemudian dijalankan untuk menghasilkan *next state* dan *reward*. Setiap transisi yang terjadi disimpan ke dalam *replay buffer* sebagai pengalaman pembelajaran. Untuk mengatasi permasalahan *sparse reward*, data dalam *replay buffer* dimodifikasi menggunakan metode HER, yaitu dengan melabel ulang pengalaman yang awalnya gagal menjadi berhasil terhadap *goal* alternatif yang benar-benar tercapai dalam episode tersebut. Setelah tahap ini, *target policy* telah menjadi kebijakan tunggal (*single end-to-end policy*) yang mampu melakukan seluruh tahapan tugas, tanpa perlu bantuan *primitive policies* lagi. Model *target policy* inilah yang akan dievaluasi dalam tahap pengujian simulasi dan diimplementasikan pada robot MyCobot 280 Pi di dunia nyata.

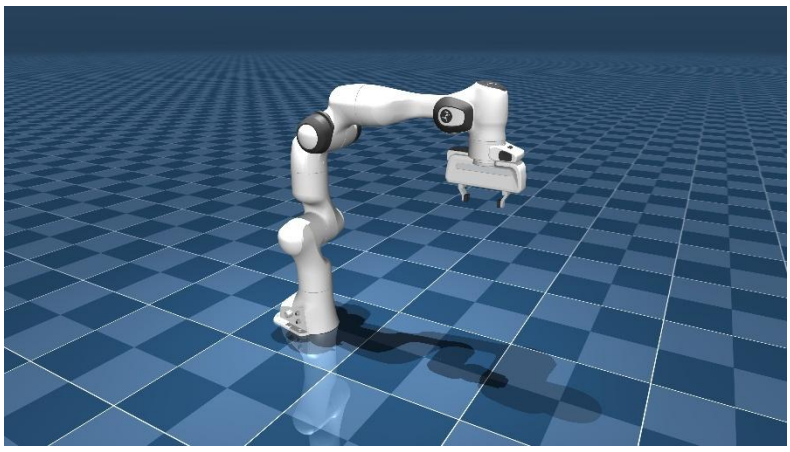
3.3.4 Pengujian di Lingkungan Simulasi

Pengujian simulasi bertujuan untuk mengevaluasi kinerja model *policy* hasil pelatihan, yaitu *target policy*, pada lingkungan simulasi yang juga merupakan lingkungan pelatihan model. Tahapan ini dimaksudkan untuk model untuk menguji dan memverifikasi model pada lingkungan simulasi sebelum diuji pada tahap selanjutnya yaitu lingkungan nyata. Seluruh pengujian dilakukan pada lingkungan simulasi MuJoCo, yang memodelkan sistem robot MyCobot 280 Pi beserta meja kerja, kamera Logitech C270 HD Webcam, dan berbagai objek uji berupa kubus, silinder, serta prisma segitiga dengan dimensi dan sifat material yang identik dengan kondisi nyata. Contoh lingkungan simulasi MuJoCo yang disajikan pada Gambar 3.11.

Secara khusus, tahap pengujian simulasi ini memiliki beberapa tujuan utama, yaitu:

1. Mengukur tingkat keberhasilan kebijakan dalam menyelesaikan tugas penyisipan objek pada berbagai variasi posisi dan orientasi target.
2. Mengevaluasi kelancaran serta kestabilan lintasan *end-effector*.
3. Menilai ketahanan kebijakan terhadap variasi parameter lingkungan (*domain randomization*) sebelum dilakukan transfer ke sistem fisik (*sim-to-real transfer*).

Setiap skenario pengujian dilaksanakan sebanyak 50 episode pada tiap objek dengan total 150 episode dengan kondisi awal seperti posisi dan orientasi dari objek dan lubang target yang diacak. Variasi tersebut dirancang untuk merepresentasikan berbagai kondisi lingkungan yang mungkin dihadapi oleh sistem robot dalam penerapan nyata. Posisi dan orientasi objek serta lubang target ditentukan secara acak dalam area jangkauan *end-effector*. Rentang posisi yang digunakan adalah 15–27 cm pada sumbu X dan -10 hingga 10 cm pada sumbu Y, dengan orientasi sebesar ±45°.



Gambar 3.11 Ilustrasi lingkungan simulasi Mujoco
(Sumber: https://mujoco.readthedocs.io/en/stable/models.html (2023))

Selain randomisasi posisi dan orientasi objek, penelitian ini juga menerapkan *domain randomization* pada parameter gesek di lingkungan simulasi MuJoCo untuk meningkatkan kemampuan generalisasi *policy* terhadap variasi kondisi kontak antara *gripper* dan objek. Parameter koefisien gesek pada *geom gripper* dan *geom objek* diacak pada setiap awal episode pelatihan, meliputi *sliding friction*, *torsional friction*, dan *rolling friction*. Pada *gripper*, nilai *sliding friction*

diacak pada rentang 0,8 hingga 2,0, *torsional friction* pada rentang 0,003 hingga 0,03, dan *rolling friction* pada rentang 1×10^{-5} hingga 5×10^{-4}. Sementara itu, pada objek, nilai *sliding friction* diacak pada rentang 0,3 hingga 1,2, *torsional friction* pada rentang 0,001 hingga 0,015, dan *rolling friction* pada rentang 1×10^{-6} hingga 2×10^{-4}. Gaya gesek tidak diacak secara langsung, melainkan tetap dihitung oleh *engine* MuJoCo berdasarkan kondisi kontak, gaya normal, dan parameter gesek yang digunakan pada episode tersebut. Dengan demikian, agen dilatih pada berbagai variasi kontak sehingga tidak terlalu bergantung pada satu konfigurasi parameter fisik tertentu.

Selama pengujian, kebijakan dijalankan dalam mode deterministik tanpa penambahan *exploration noise* agar performa murni hasil pelatihan dapat diukur secara objektif. Sistem secara otomatis merekam data posisi, orientasi *end-effector* dengan memanfaatkan sensor virtual yang tersedia pada simulator. Sementara itu, status keberhasilan setiap episode dicatat setelah eksekusi selesai, berdasarkan apakah objek berhasil dimasukkan ke lubang target atau tidak.

Secara keseluruhan, hasil pengujian simulasi digunakan sebagai dasar untuk menilai apakah *target policy* telah menunjukkan kemampuan yang cukup untuk dianalisis lebih lanjut pada lingkungan nyata. Penilaian dilakukan berdasarkan tiga aspek utama. Pertama, tingkat keberhasilan atau *success rate* digunakan untuk mengetahui kemampuan *target policy* dalam menyelesaikan tugas penyisipan objek pada lingkungan simulasi. Kedua, lintasan *end-effector* selama eksekusi diamati untuk menilai kelancaran dan kestabilan gerakan, terutama apakah *policy* mampu menghasilkan trajektori yang terarah tanpa gerakan yang tidak wajar. Ketiga, kemampuan *policy* dalam menghadapi variasi kondisi awal diuji melalui perubahan posisi dan orientasi awal objek serta lubang target pada setiap episode pengujian.

Berdasarkan hasil tersebut, pengujian pada lingkungan nyata tidak diposisikan sebagai bukti bahwa *policy* telah sepenuhnya siap diterapkan pada robot fisik, melainkan sebagai evaluasi awal untuk melihat kemampuan transfer dari simulasi ke dunia nyata. Pengujian ini juga digunakan untuk mengidentifikasi faktor-faktor yang menyebabkan penurunan performa, seperti *error* persepsi,

perbedaan kondisi fisik antara simulasi dan robot nyata, serta keterbatasan *trajectory replay* yang belum melakukan koreksi secara *real-time* selama eksekusi.

3.3.5 Pengujian di Lingkungan Nyata

Pengujian di lingkungan nyata bertujuan untuk mengevaluasi kemampuan transfer kebijakan hasil pelatihan dari lingkungan simulasi ke robot fisik. Pengujian ini tidak diposisikan sebagai validasi bahwa sistem telah siap diterapkan secara praktis, melainkan sebagai evaluasi awal *sim-to-real* untuk melihat sejauh mana *trajectory* yang berhasil dihasilkan pada simulasi dapat dieksekusi pada robot myCobot 280 Pi.

Pengujian di lingkungan nyata dilakukan sebanyak 10 episode untuk setiap jenis objek, sehingga total terdapat 30 episode pengujian fisik. Jumlah episode ini lebih sedikit dibandingkan pengujian simulasi karena pengujian pada robot nyata membutuhkan proses *reset* manual pada setiap episode. Proses tersebut meliputi penempatan ulang objek, pengecekan posisi lubang target, pemeriksaan deteksi AprilTag, serta pengawasan terhadap kestabilan *grasp* sebelum dan selama eksekusi *trajectory*.

Selain itu, pengujian pada robot fisik memiliki keterbatasan waktu dan risiko mekanik yang lebih tinggi dibandingkan simulasi. Pada beberapa kondisi, objek dapat bergeser, terlepas dari *gripper*, atau tidak berada pada pose awal yang sama setelah proses reset. Karena penelitian ini menggunakan pendekatan *trajectory replay* yang masih bersifat *open-loop*, kesalahan pose awal atau pergeseran objek selama eksekusi dapat langsung memengaruhi keberhasilan pengujian. Oleh karena itu, jumlah 10 episode per objek digunakan sebagai evaluasi awal untuk melihat kemampuan transfer dari simulasi ke robot fisik, bukan sebagai validasi final terhadap performa sistem pada lingkungan nyata.

Pada pengujian ini, pose awal objek dan lubang target diperoleh menggunakan kamera dan AprilTag. Pose tersebut kemudian ditransformasikan ke sistem koordinat robot dan dikirim ke lingkungan simulasi MuJoCo. *Target policy* dijalankan di simulasi untuk menghasilkan *trajectory joint*, kemudian *trajectory* tersebut dieksekusi pada robot nyata. Dengan demikian, pengujian lingkungan

nyata pada penelitian ini menggunakan pendekatan *trajectory replay* yang bersifat *open-loop*.

Pendekatan ini memiliki keterbatasan karena robot tidak memperbarui aksi berdasarkan kondisi aktual objek selama proses eksekusi. Apabila objek bergeser, orientasi objek berubah, *grasp* tidak stabil, atau estimasi pose awal memiliki *error*, robot tetap mengikuti *trajectory* yang telah dihasilkan sebelumnya. Oleh karena itu, hasil pengujian nyata digunakan untuk mengidentifikasi keterbatasan transfer dari simulasi ke dunia nyata, terutama pada aspek akurasi persepsi, transformasi koordinat, kestabilan *grasp*, dan kesesuaian eksekusi robot fisik terhadap *trajectory* simulasi.

3.3.6 Analisis Hasil

Analisis data dilakukan untuk menilai performa kebijakan hasil pelatihan algoritma *SAC* berbasis *QMP–HER* pada tugas penyisipan objek menggunakan robot manipulator. Tujuan utama analisis ini adalah untuk mengukur tingkat keberhasilan, waktu penyelesaian, efisiensi sampel serta kelancaran lintasan. Dengan demikian, bagian ini memberikan dasar evaluasi performa bagi penarikan kesimpulan terhadap keberhasilan penerapan metode yang dikembangkan.

Data yang digunakan dalam analisis berasal dari dua tahap pengujian, yaitu pengujian simulasi pada lingkungan MuJoCo dan pengujian di lingkungan fisik menggunakan robot MyCobot 280 Pi. Data ini dimanfaatkan untuk menghitung dan mengevaluasi kinerja kebijakan hasil pelatihan menggunakan beberapa metrik utama yang telah dijelaskan pada Subbab 2.8 mengenai berbagai aspek performa model *Reinforcement Learning*.

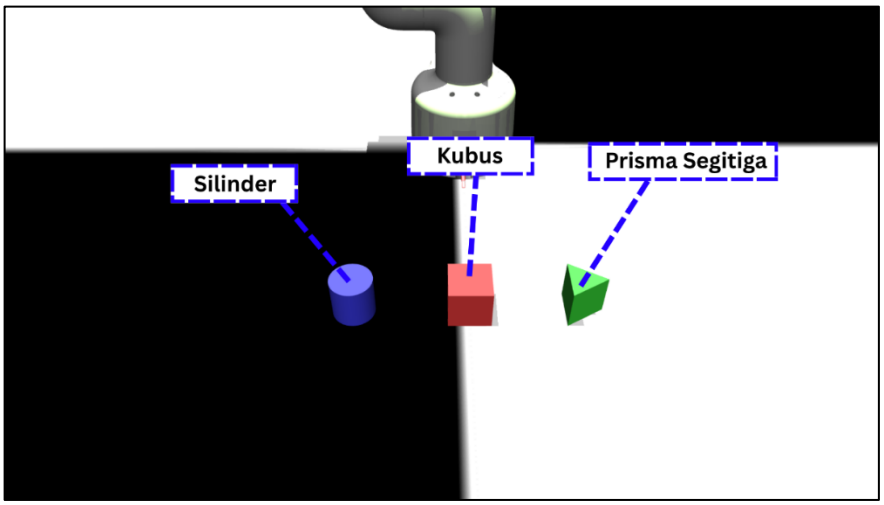
Metrik tersebut meliputi tingkat keberhasilan *(success rate*) untuk mengukur efektivitas kebijakan dalam menyelesaikan tugas, waktu penyelesaian (*completion time*) untuk menilai durasi eksekusi, serta efisiensi sampel (*sample efficiency*) yang digunakan untuk mengevaluasi seberapa cepat kebijakan mampu mencapai performa optimal dengan jumlah data atau interaksi yang minimal dengan membandingkannya dengan SAC tanpa mekanisme QMP-HER. Selain itu terdapat metrik analisis stabilitas pergerakan yang digunakan untuk mengukur kelancaran dan konsistensi lintasan *end-effector* selama proses manipulasi.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

4.1.1 Implementasi Lingkungan Simulasi

Lingkungan simulasi diimplementasikan menggunakan MuJoCo untuk merepresentasikan robot manipulator, objek uji, dan target. Robot yang digunakan pada simulasi disesuaikan dengan konfigurasi myCobot 280 Pi yang memiliki enam derajat kebebasan. Objek uji yang digunakan terdiri atas kubus, silinder, dan prisma segitiga. Setiap objek ditempatkan pada area kerja robot dengan variasi posisi dan orientasi awal untuk melatih kemampuan generalisasi *policy.* Gambar 4.1 menunjukkan bentuk dari tiap objek uji.

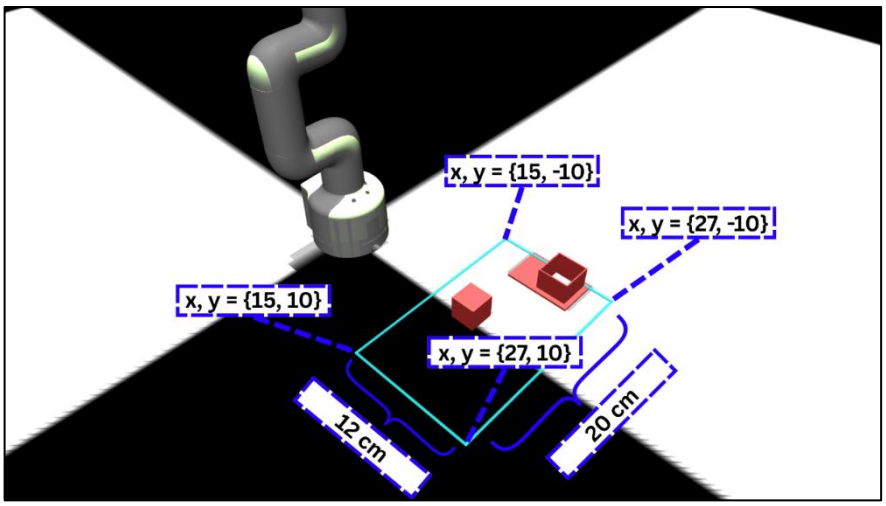


Gambar 4.1 Tiga Bentuk Objek Uji

Pada setiap episode, posisi objek dan target diinisialisasi dalam batas *workspace* robot dapat dilihat di Gambar 4.2. Target utama pada penelitian ini didefinisikan sebagai posisi lubang target. Posisi ini digunakan sebagai tujuan akhir pada *insert policy* dan *target policy*. Dengan menggunakan lingkungan simulasi, proses pelatihan dapat dilakukan secara berulang tanpa risiko kerusakan pada robot fisik. Gambar 4.2 menunjukkan batas *workspace* robot.

Selain variasi posisi dan orientasi awal, lingkungan simulasi juga menerapkan *domain randomization* parameter gesek pada *gripper* dan objek selama proses pelatihan. Randomisasi ini membuat agen mengalami kondisi kontak yang berbeda-beda, misalnya kondisi objek yang lebih mudah tergelincir ketika koefisien

gesek rendah atau lebih stabil ketika koefisien gesek tinggi. Dengan demikian, *policy* tidak hanya belajar pada satu konfigurasi kontak tertentu, tetapi dilatih agar lebih *robust* terhadap variasi parameter fisik.



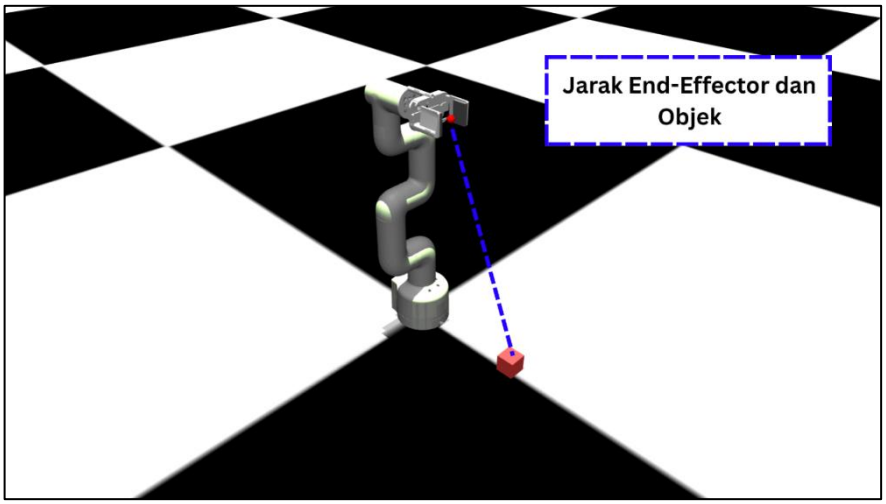
Gambar 4.2 Batas Ruang Kerja Robot

4.1.2 Implementasi Ruang Keadaan dan Ruang Aksi

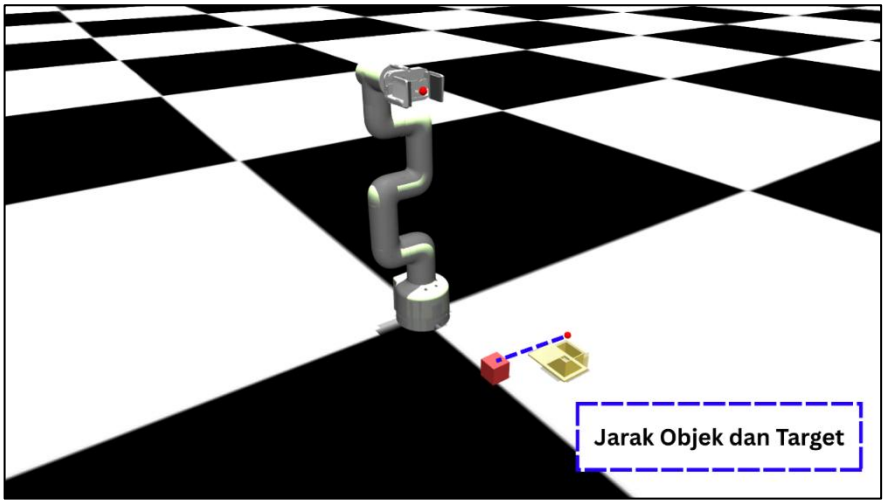
Implementasi ruang keadaan dan ruang aksi dilakukan berdasarkan rancangan yang telah dijelaskan pada Bab 3. Pada penelitian ini, setiap *policy* menggunakan ruang keadaan yang disesuaikan dengan tujuan pembelajarannya. *grasp policy* menggunakan ruang keadaan sebesar 33 dimensi yang berfokus pada hubungan antara *end-effector* dan objek. Informasi yang digunakan *grasp policy* meliputi kondisi sendi robot, pose *end-effector*, pose objek, hubungan relatif antara *end-effector* dan objek, *error* orientasi *end-effector* terhadap objek, serta status *gripper*. Ruang keadaan ini digunakan agar *policy* dapat mempelajari gerakan untuk mendekati objek dan menyesuaikan orientasi sebelum proses *grasping* dilakukan.

Insert policy dan *target policy* menggunakan ruang keadaan yang sama, yaitu sebesar 46 dimensi. Hal ini dilakukan karena kedua *policy* membutuhkan informasi yang lebih lengkap mengenai kondisi robot, objek, dan target. Informasi yang digunakan mencakup kondisi sendi robot, pose *end-effector*, pose objek, posisi lubang target, orientasi lubang target, hubungan relatif antara *end-effector* dan objek, hubungan relatif antara objek dan target, *error* orientasi *end-effector* terhadap objek, *error* orientasi objek terhadap lubang target, serta status *gripper*.

Gambar 4.3 dan 4.4 mengilustrasikan jarak dari *end-effector* ke objek lalu jarak dari objek ke target.



Gambar 4.3 Ilustrasi Jarak *End-Effector* dan Objek



Gambar 4.4 Ilustrasi Jarak Objek dan Target

Ruang aksi pada seluruh *policy* diimplementasikan dalam bentuk aksi Cartesian 6-DOF, yang terdiri atas perubahan posisi dan orientasi *end-effector*. Aksi tersebut tidak langsung dikirimkan sebagai perintah sudut sendi, tetapi terlebih dahulu dikonversi menjadi target pose *end-effector* dan kemudian dihitung menggunakan *inverse kinematics*. Pada penelitian ini, *gripper* tidak termasuk dalam ruang aksi *policy*. Mekanisme buka dan tutup *gripper* dijalankan secara *rule-based* berdasarkan kondisi posisi dan orientasi objek.

4.1.3 Implementasi Alur Pelatihan

Implementasi algoritma pembelajaran dilakukan secara bertahap sesuai dengan rancangan yang telah dijelaskan pada Bab 3. Tahapan pelatihan dibagi menjadi dua bagian utama, yaitu pelatihan *primitive policy* dan pelatihan *target policy*. *Primitive policy* terdiri atas *grasp policy* dan *insert policy*, sedangkan *target policy* merupakan kebijakan utama yang dilatih dengan pendekatan QMP-HER.

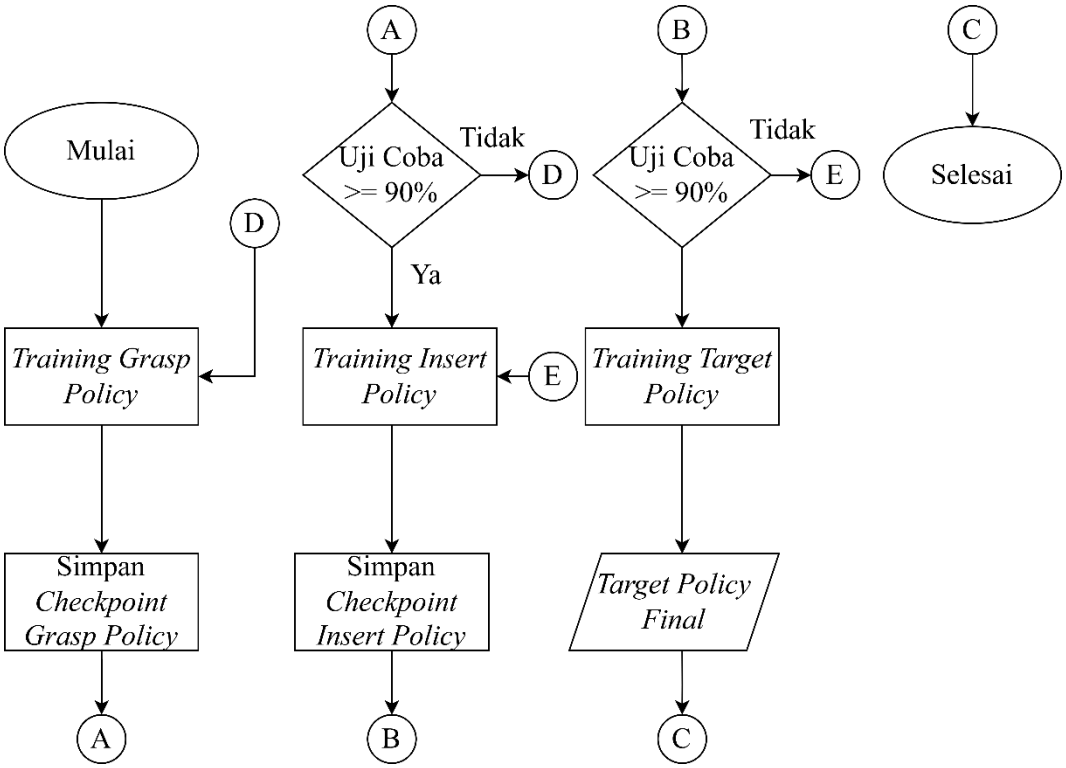
Tahap pertama adalah pelatihan *grasp policy* menggunakan algoritma *Soft Actor-Critic*. *Policy* ini dilatih untuk menghasilkan gerakan *end-effector* yang mampu mendekati objek dan membentuk kondisi *grasping* yang stabil. Setelah proses pelatihan selesai, *grasp policy* dievaluasi melalui 100 kali uji coba pada lingkungan simulasi dengan variasi posisi dan orientasi objek. *Policy* dinyatakan memadai apabila mencapai tingkat keberhasilan minimal 90%. Jika kriteria tersebut terpenuhi, parameter jaringan *grasp policy* disimpan sebagai *primitive policy*.

Tahap kedua adalah pelatihan *insert policy*. *Policy* ini dilatih untuk membawa objek yang telah tergenggam menuju posisi *release* dan menyisipkan objek ke dalam lubang. Sama seperti *grasp policy*, *insert policy* dilatih menggunakan SAC dengan *reward* yang lebih informatif agar proses pembelajaran pada subtugas ini dapat berlangsung lebih stabil. Setelah proses pelatihan selesai, *insert policy* dievaluasi melalui 100 kali uji coba pada lingkungan simulasi dengan variasi posisi dan orientasi objek serta target. *Policy* dinyatakan memadai apabila mencapai tingkat keberhasilan minimal 90%. Jika kriteria tersebut terpenuhi, parameter jaringan *insert policy* disimpan sebagai *primitive policy*.

Setelah kedua *primitive policy* selesai dilatih, tahap berikutnya adalah pelatihan *target policy* menggunakan mekanisme QMP-HER. Pada tahap ini, parameter *grasp policy* dan *insert policy* tidak lagi diperbarui, melainkan digunakan sebagai sumber kandidat aksi. Dengan demikian, kedua *primitive policy* berperan sebagai panduan eksplorasi bagi *target policy* selama proses pelatihan.

Target policy dilatih menggunakan *sparse reward*, yaitu *reward* 0 ketika objek berhasil masuk ke dalam lubang target dan *reward* -1 ketika kondisi tersebut belum tercapai. Setelah proses pelatihan selesai, *target policy* digunakan sebagai kebijakan utama pada tahap pengujian. Pada tahap ini, *primitive policy* tidak lagi

digunakan untuk mengontrol robot secara langsung. Dengan demikian, *grasp policy* dan *insert policy* hanya berperan sebagai pendukung selama proses pelatihan, sedangkan pengujian akhir dilakukan menggunakan *target policy* sebagai *policy* tunggal. Alur proses pelatihan ditunjukkan pada diagram yang ada di Gambar 4.5.



Gambar 4.5 Diagram Alur Pelatihan

4.2 Hasil Pelatihan *Policy*

Pelatihan *primitive policy* dilakukan untuk memperoleh model yang mampu menyelesaikan subtugas tertentu sebelum digunakan dalam pelatihan *target policy*. Pada penelitian ini, terdapat dua *primitive policy* yang dilatih secara terpisah, yaitu *grasp policy* dan *insert policy.* Kedua primitive *policy* dilatih menggunakan algoritma SAC tanpa mekanisme QMP-HER dengan *reward* yang informatif untuk memudahkan pelatihan. Lalu dilatih *target policy* menggunakan algoritma SAC dengan mekanisme QMP-HER dengan menggunakan bantuan *primitive policy*. Konfigurasi *hyperparameter* pada Tabel 4.1, 4.2, dan 4.3 tiap *policy* merupakan nilai yang diperoleh setelah eksperimen awal secara *trial and error* dengan mempertimbangkan kestabilan *reward*, *actor loss*, *critic loss*, dan keberhasilan *policy* pada evaluasi simulasi.

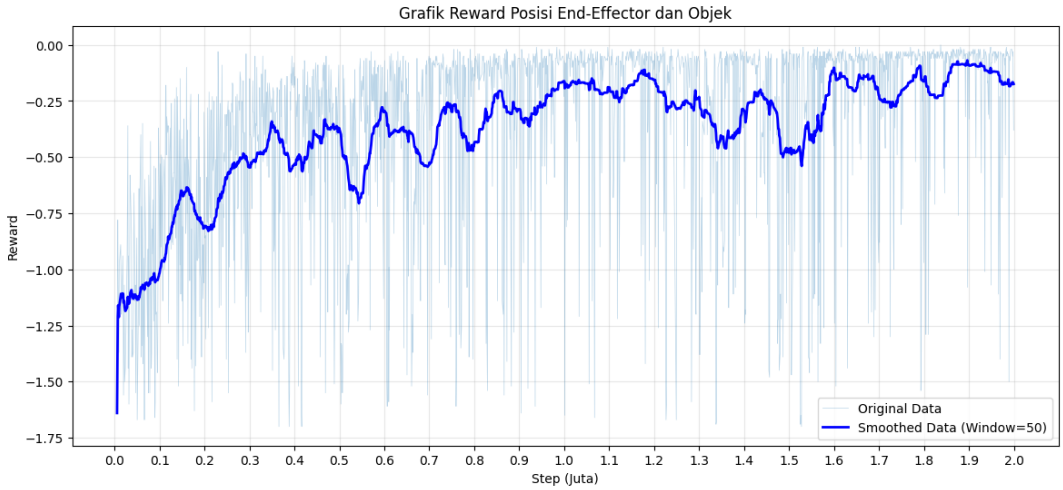
4.2.1 Hasil Pelatihan *Grasp Policy*

Grasp policy dilatih untuk mengarahkan *end-effector* menuju objek dan menghasilkan kondisi yang memungkinkan objek dapat digenggam dengan stabil. Pada tahap ini, *policy* berfokus pada hubungan antara *end-effector* dan objek, sehingga performa pelatihan dianalisis berdasarkan *reward* posisi, *reward* orientasi, total *reward*, serta hasil evaluasi akhir melalui 100 episode pengujian. Konfigurasi *hyperparameter* yang digunakan pada pelatihan *grasp policy* terdapat pada Tabel 4.1.

Tabel 4.1 Konfigurasi *Hyperparameter Grasp Policy*

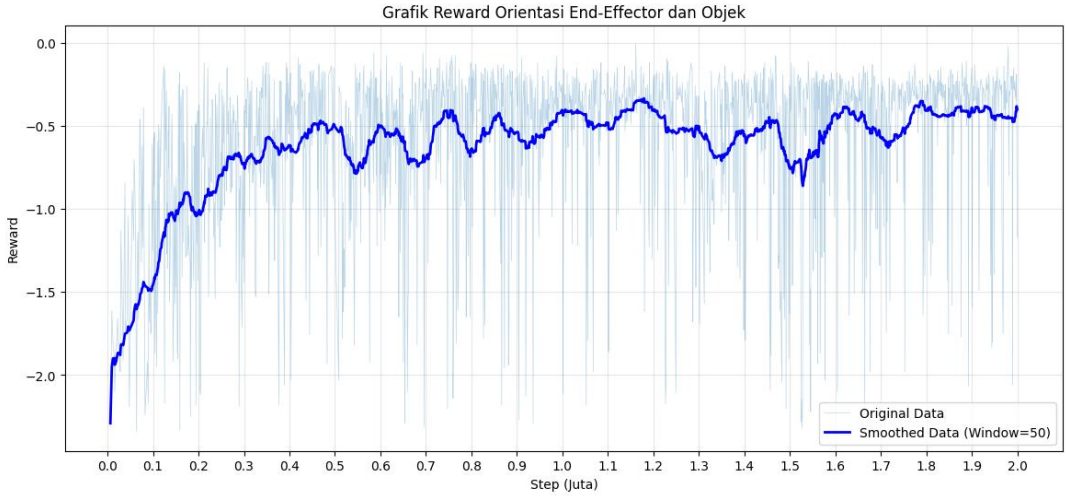
Hyperparameter	**Nilai**
Total *timesteps*	2.000.000
Learning rate	3×10^{-5}
Learning starts	5.000
Buffer size	1.000.000
Batch size	256
Discount factor (γ)	0,99
Soft update coefficient (τ)	0,005
Entropy coefficient	0,01
Actor / policy network	[512, 512, 256]
Critic / Q-function	[512, 512, 256]
Activation function	ReLU

Kurva *reward* posisi pada Gambar 4.6 memperlihatkan bahwa kemampuan *grasp policy* dalam mendekatkan *end-effector* ke objek meningkat secara bertahap selama pelatihan. Berdasarkan grafik tersebut, *reward* posisi mengalami peningkatan secara bertahap. Pada fase awal pelatihan, nilai *reward* posisi berada pada kisaran sekitar -1,2. Nilai tersebut menunjukkan bahwa jarak antara *end-effector* dan objek masih relatif besar karena *policy* masih berada pada tahap eksplorasi. Seiring bertambahnya jumlah *step* pelatihan, *reward* posisi bergerak semakin mendekati nol dan berada pada kisaran sekitar -0,15 pada fase akhir pelatihan. Karena *reward* posisi dirancang sebagai penalti terhadap jarak, nilai *reward* yang semakin mendekati nol menunjukkan bahwa *end-effector* semakin mampu mendekati objek dengan lebih baik.



Gambar 4.6 Grafik *Reward* Posisi *End-Effector* dan Objek

Meskipun data asli pada grafik menunjukkan fluktuasi yang cukup tinggi, tren hasil *smoothing* memperlihatkan peningkatan performa yang jelas. Fluktuasi tersebut dapat disebabkan oleh proses eksplorasi *policy*, variasi posisi awal objek, serta perbedaan kondisi awal pada setiap episode. Namun, secara umum, peningkatan *reward* posisi menunjukkan bahwa *grasp policy* berhasil mempelajari hubungan antara *end-effector* dan objek.



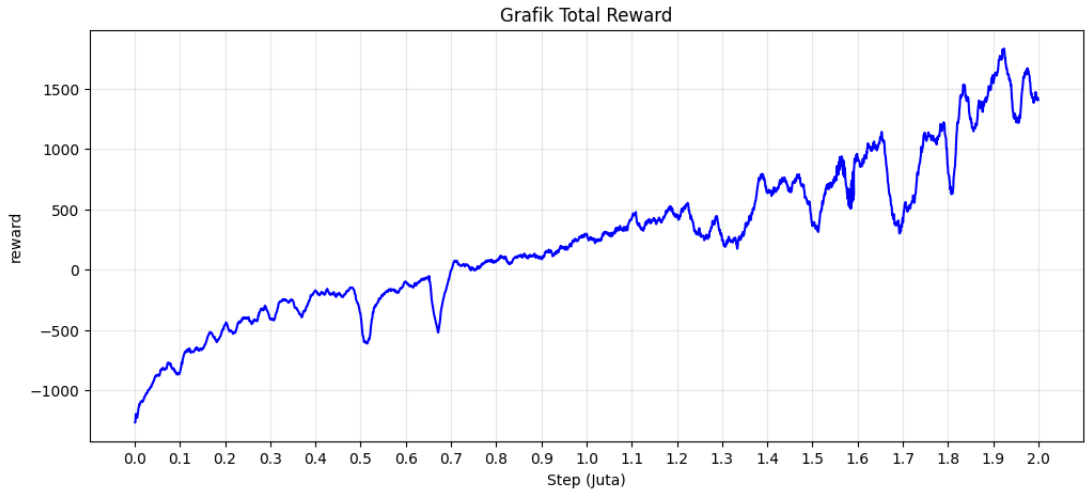
Gambar 4.7 Grafik *Reward* Orientasi *End-Effector* dan Objek

Gambar 4.7 menunjukkan grafik *reward* orientasi antara *end-effector* dan objek. Pada awal pelatihan, *reward* orientasi berada pada kisaran sekitar -2,0. Nilai ini menunjukkan bahwa orientasi *end-effector* masih memiliki perbedaan yang cukup besar terhadap orientasi objek. Setelah proses pelatihan berlangsung, *reward*

orientasi meningkat dan cenderung berada pada kisaran sekitar -0,4 hingga -0,5 pada fase akhir pelatihan. Peningkatan ini menunjukkan bahwa *policy* tidak hanya belajar mendekati objek dari sisi posisi, tetapi juga mulai mampu menyesuaikan orientasi *end-effector* terhadap objek.

Pada grafik *reward* orientasi, masih terlihat adanya fluktuasi selama proses pelatihan. Fluktuasi tersebut menunjukkan bahwa penyesuaian orientasi merupakan bagian yang lebih kompleks dibandingkan pengurangan jarak posisi. Hal ini wajar karena *grasp policy* harus menghasilkan perubahan pose *end-effector* dalam ruang Cartesian 6-DOF, sehingga *policy* perlu mempertimbangkan posisi dan orientasi secara bersamaan. Meskipun demikian, tren *reward* orientasi yang meningkat menunjukkan bahwa kemampuan *policy* dalam mengurangi *error* orientasi mengalami perbaikan.

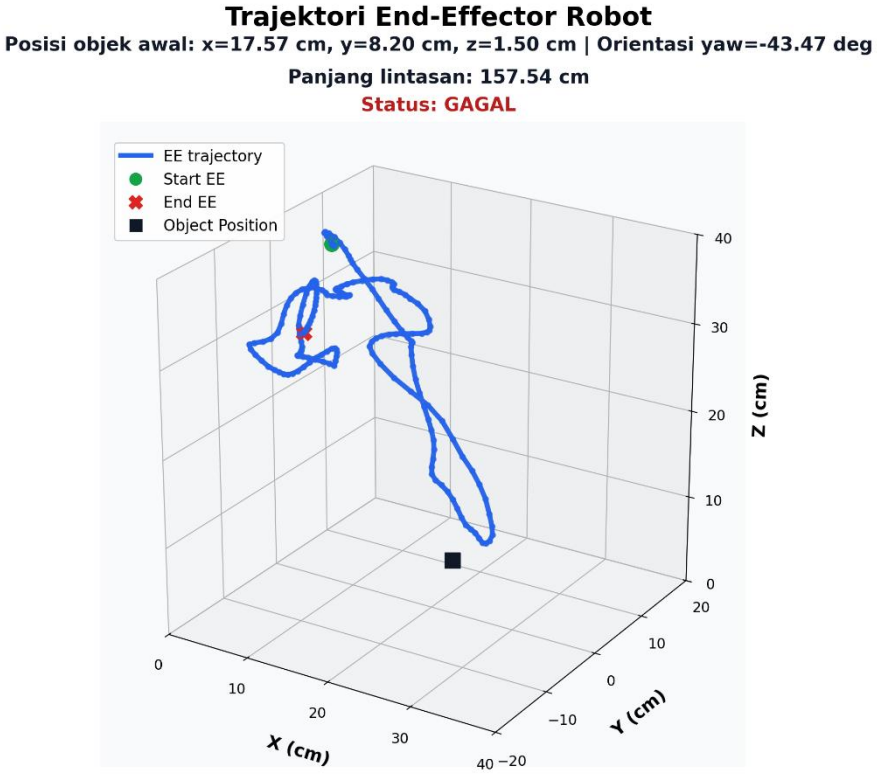
Gambar 4.8 menunjukkan grafik total *reward* selama pelatihan *grasp policy*. Total *reward* mengalami peningkatan yang signifikan dari nilai negatif pada awal pelatihan hingga mencapai nilai positif pada fase akhir pelatihan. Pada awal pelatihan, total *reward* berada pada kisaran sekitar -1200. Nilai tersebut menunjukkan bahwa *policy* belum mampu menghasilkan aksi yang efektif untuk mendekati objek dan menyesuaikan orientasi *end-effector*. Setelah proses pelatihan berlangsung, total *reward* meningkat secara bertahap hingga mencapai lebih dari 1400 pada fase akhir pelatihan. Peningkatan ini menunjukkan bahwa *policy* semakin mampu menghasilkan perilaku yang sesuai dengan tujuan *grasp policy*.



Gambar 4.8 Grafik Total *Reward Grasp Policy*

Peningkatan total *reward* menunjukkan bahwa *grasp policy* berhasil mempelajari perilaku dasar untuk mendekati objek dan membentuk kondisi *grasping* yang lebih stabil. Walaupun terdapat beberapa penurunan *reward* pada fase tertentu, tren keseluruhan tetap menunjukkan peningkatan. Penurunan sementara tersebut dapat terjadi karena eksplorasi *policy* atau kondisi awal episode yang lebih sulit, misalnya posisi objek yang lebih jauh atau orientasi objek yang lebih menantang.

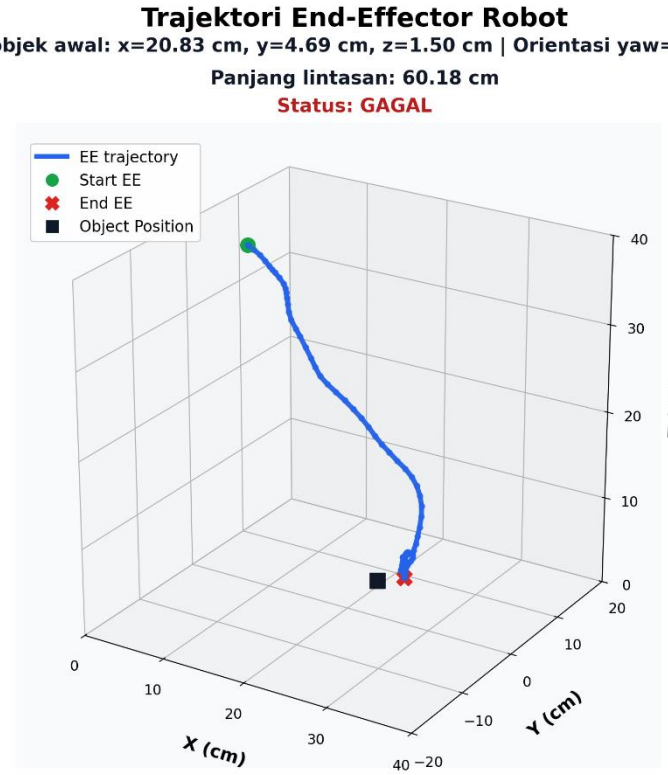
Secara keseluruhan, hasil pelatihan *grasp policy* menunjukkan bahwa *policy* mampu belajar secara progresif. *Reward* posisi yang semakin mendekati nol menunjukkan bahwa *end-effector* semakin mampu mendekati objek. *Reward* orientasi yang meningkat menunjukkan bahwa *end-effector* semakin mampu menyesuaikan orientasinya terhadap objek. Selain itu, peningkatan total *reward* menunjukkan bahwa kombinasi antara pengurangan jarak, pengurangan *error* orientasi, dan keberhasilan *grasping* memberikan kontribusi positif terhadap proses pembelajaran.



Gambar 4.9 Trajektori *End-Effector* Robot pada *Step* ke-5000

Setelah menganalisis *reward* posisi, *reward* orientasi, dan total *reward*, hasil pelatihan *grasp policy* juga dianalisis melalui contoh trajektori *end-effector*. Analisis trajektori digunakan untuk melihat perubahan perilaku *policy* secara visual, seperti kestabilan lintasan, dan kemampuan *end-effector* mendekati objek. Dengan melihat trajektori, proses pembelajaran tidak hanya dapat dinilai dari nilai *reward*, tetapi juga dari pola gerakan yang dihasilkan oleh *policy*.

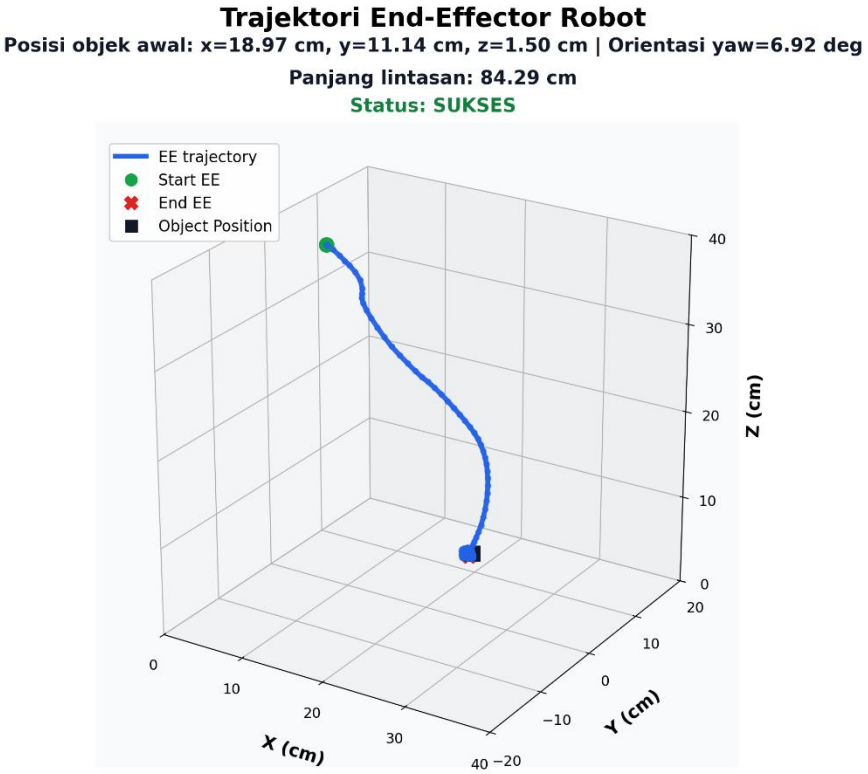
Gambar 4.9 di atas menunjukkan contoh trajektori *end-effector* pada *timestep* ke-5.000. Pada fase awal pelatihan ini, trajektori *end-effector* masih menunjukkan pola gerakan yang belum terarah. Kondisi tersebut wajar karena pada fase awal, *policy* masih berada pada tahap eksplorasi dan belum memiliki representasi tindakan yang stabil untuk mendekati objek.



Gambar 4.10 Trajektori *End-Effector* Robot pada *Step* ke-500.000

Gambar 4.10 menunjukkan contoh trajektori *end-effector* pada *timestep* ke-500.000. Pada fase ini, lintasan *end-effector* mulai menunjukkan perbaikan dibandingkan fase awal. Gerakan yang dihasilkan mulai mengarah ke area objek, tetapi masih terdapat pola lintasan yang berfluktuasi dan belum sepenuhnya stabil. Meskipun *end-effector* sudah lebih dekat terhadap objek dibandingkan pada fase

awal, episode masih berstatus gagal. Hal ini menunjukkan bahwa *policy* sudah mulai mempelajari arah gerakan yang relevan, tetapi belum cukup konsisten untuk memenuhi seluruh kriteria keberhasilan, terutama dari sisi ketepatan posisi akhir, orientasi, atau kondisi *grasping*.



Gambar 4.11 Trajektori *End-Effector* Robot pada *Step* ke-2.000.000

Gambar 4.11 menunjukkan contoh trajektori *end-effector* pada *timestep* ke-2.000.000. Pada fase akhir pelatihan, lintasan *end-effector* terlihat lebih stabil dan lebih terarah menuju objek. *End-effector* mampu bergerak dari posisi awal menuju objek dengan penyimpangan yang lebih kecil dibandingkan fase awal dan fase menengah. Status episode yang berhasil menunjukkan bahwa *policy* telah mampu menghasilkan aksi yang sesuai untuk mencapai kondisi *grasping*. Perubahan ini memperlihatkan bahwa *grasp policy* mengalami peningkatan kemampuan dalam memahami hubungan antara pose *end-effector* dan posisi objek.

Secara keseluruhan, analisis trajektori menunjukkan adanya perkembangan perilaku *policy* selama proses pelatihan. Pada *timestep* awal, *policy* masih menghasilkan gerakan yang belum terarah dan belum berhasil menyelesaikan tugas. Pada *timestep* menengah, *policy* mulai menghasilkan gerakan yang lebih mendekati

objek, tetapi belum cukup stabil untuk mencapai keberhasilan. Pada *timestep* akhir, *policy* mampu menghasilkan lintasan yang lebih terarah, dan berhasil mencapai kondisi *grasping*. Hasil ini memperkuat analisis *reward* sebelumnya, yaitu bahwa peningkatan *reward* posisi, *reward* orientasi, dan total *reward* sejalan dengan peningkatan kualitas gerakan *end-effector*.

Setelah analisis trajektori dilakukan, *grasp policy* dievaluasi melalui 100 episode pengujian pada lingkungan simulasi. Evaluasi dilakukan dengan variasi posisi dan orientasi awal objek untuk mengukur kemampuan *policy* dalam menghasilkan kondisi *grasping* yang stabil. Berdasarkan hasil evaluasi, *Grasp Policy* berhasil menyelesaikan 90 dari 100 episode pengujian, sehingga memperoleh *success rate* sebesar 90%. Rata-rata jumlah *step* yang dibutuhkan untuk mencapai kondisi berhasil adalah 110 *step*.

Berdasarkan hasil pelatihan dan evaluasi akhir, *grasp policy* menunjukkan kemampuan yang memadai dalam menyelesaikan subtugas *grasping*. *Reward* posisi yang semakin mendekati nol menunjukkan bahwa *end-effector* semakin mampu mendekati objek. *Reward* orientasi yang meningkat menunjukkan bahwa *end-effector* semakin mampu menyesuaikan orientasinya terhadap objek. Selain itu, total *reward* yang meningkat dan trajektori yang lebih stabil menunjukkan bahwa *policy* mampu menghasilkan perilaku gerakan yang semakin sesuai dengan tujuan *grasping*. Dengan demikian, parameter jaringan *grasp policy* disimpan sebagai *primitive policy* dan digunakan sebagai salah satu sumber kandidat aksi pada pelatihan *target policy*.

4.2.2 Hasil Pelatihan *Insert Policy*

Insert policy dilatih untuk membawa objek yang telah tergenggam menuju posisi *release* lalu melepaskan objek menuju lubang target. Berbeda dengan *grasp policy* yang berfokus pada hubungan antara *end-effector* dan objek, *Insert policy* berfokus pada hubungan antara objek dan target. Oleh karena itu, performa *insert policy* dianalisis berdasarkan *reward* posisi objek terhadap target, *reward* orientasi objek terhadap lubang target, total *reward*, trajektori pergerakan, serta hasil evaluasi akhir melalui 100 episode pengujian. Konfigurasi *hyperparameter* yang digunakan pada pelatihan *insert policy* terdapat pada tabel 4.2.

Tabel 4.2 Konfigurasi *Hyperparameter Insert Policy*

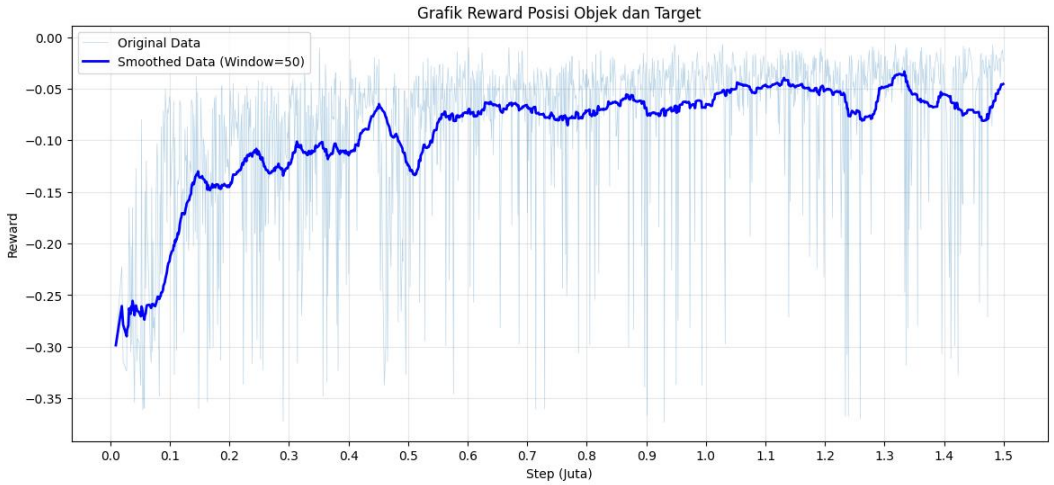
Hyperparameter	Nilai
Total *timesteps*	1.500.000
Learning rate	3×10^{-5}
Learning starts	5.000
Buffer size	1.000.000
Batch size	256
Discount factor (γ)	0,99
Soft update coefficient (τ)	0,005
Entropy coefficient	0,01
Actor / policy network	[512, 512, 256]
Critic / Q-function	[512, 512, 256]
Activation function	ReLU

Gambar 4.12 menunjukkan grafik *reward* posisi antara objek dan posisi target selama proses pelatihan *insert policy*. Berdasarkan grafik tersebut, *reward* posisi mengalami peningkatan secara bertahap dari fase awal hingga fase akhir pelatihan. Pada awal pelatihan, nilai *reward* posisi berada pada kisaran sekitar -0,30. Nilai ini menunjukkan bahwa objek masih relatif jauh dari posisi target karena *policy* masih berada pada tahap eksplorasi dan belum mampu menghasilkan aksi yang konsisten untuk membawa objek menuju posisi target.

Seiring bertambahnya jumlah *step* pelatihan, *reward* posisi meningkat dan bergerak semakin mendekati nol. Pada fase menengah pelatihan, *reward* posisi mulai berada pada kisaran sekitar -0,10 hingga -0,07. Selanjutnya, pada fase akhir pelatihan, *reward* posisi cenderung berada pada kisaran sekitar -0,05 sampai -0,07. Karena *reward* posisi dirancang sebagai penalti terhadap jarak antara objek dan target, peningkatan *reward* yang semakin mendekati nol menunjukkan bahwa jarak objek terhadap posisi target semakin kecil.

Meskipun data asli pada grafik masih menunjukkan fluktuasi dan beberapa penurunan tajam, tren hasil *smoothing* memperlihatkan peningkatan performa yang jelas. Fluktuasi tersebut dapat disebabkan oleh variasi kondisi awal episode, perbedaan posisi dan orientasi target, serta proses eksplorasi *policy* selama pelatihan. Namun, secara umum, grafik ini menunjukkan bahwa *insert policy*

berhasil mempelajari hubungan posisi antara objek dan target, sehingga mampu menghasilkan aksi yang semakin baik untuk membawa objek menuju posisi target.



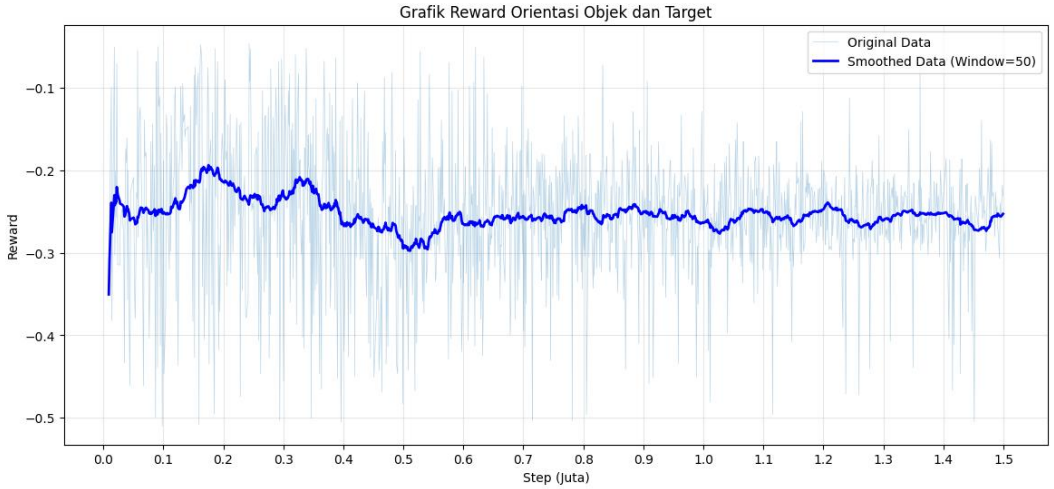
Gambar 4.12 Grafik *Reward* Posisi Objek dan Target

Gambar 4.13 menunjukkan grafik *reward* orientasi antara objek dan target selama proses pelatihan. Berdasarkan grafik tersebut, *reward* orientasi menunjukkan perubahan yang cukup fluktuatif selama pelatihan. Pada fase awal pelatihan, nilai *reward* orientasi berada pada kisaran sekitar -0,35, kemudian meningkat cukup cepat menuju kisaran sekitar -0,25. Peningkatan awal ini menunjukkan bahwa *policy* mulai mampu mengurangi *error* orientasi antara objek dan lubang target pada tahap awal pembelajaran.

Namun, setelah fase awal tersebut, *reward* orientasi tidak menunjukkan peningkatan yang sebesar *reward* posisi. Nilai *reward* orientasi cenderung berfluktuasi pada kisaran sekitar -0,20 hingga -0,30 sepanjang proses pelatihan. Pada beberapa bagian, *reward* orientasi sempat meningkat mendekati -0,20, tetapi kembali turun pada fase berikutnya. Kondisi ini menunjukkan bahwa proses penyelarasan orientasi objek terhadap lubang target lebih sulit dan lebih tidak stabil dibandingkan proses membawa objek mendekati posisi target. Hal ini disebabkan juga oleh nilai penalti *error* orientasi tidak sebesar penalti jarak objek.

Fluktuasi pada *reward* orientasi dapat disebabkan oleh beberapa faktor, seperti variasi orientasi awal objek, perubahan orientasi target, serta kompleksitas aksi Cartesian 6-DOF yang mengharuskan *policy* mengatur translasi dan rotasi secara bersamaan. Selain itu, karena objek berada dalam kondisi tergenggam,

perubahan kecil pada gerakan *end-effector* dapat memengaruhi orientasi objek terhadap target. Meskipun demikian, hasil *smoothing* menunjukkan bahwa *reward* orientasi tetap berada pada kisaran yang relatif lebih stabil setelah fase awal pelatihan. Hal ini menunjukkan bahwa *insert policy* telah mulai mempelajari strategi untuk menjaga orientasi objek terhadap lubang target, meskipun peningkatannya tidak sekuat *reward* posisi.



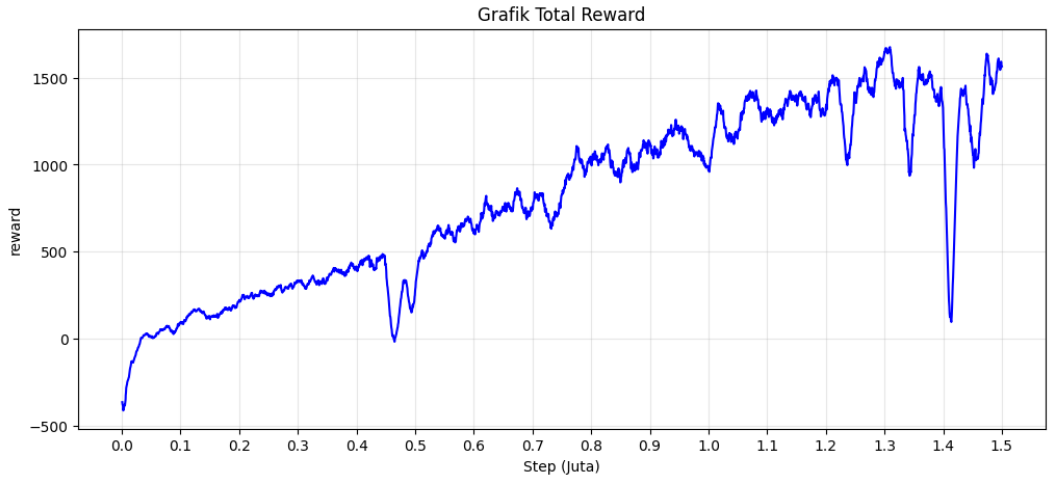
Gambar 4.13 Grafik *Reward* Orientasi Objek dan Target

Dengan demikian, grafik *reward* orientasi menunjukkan bahwa *insert policy* mampu memperoleh kemampuan dasar dalam menyelaraskan orientasi objek terhadap target. Hasil ini sejalan dengan karakteristik tugas *insert*, di mana keberhasilan tidak hanya bergantung pada kedekatan posisi objek terhadap target, tetapi juga pada kestabilan orientasi objek selama proses pergerakan.

Gambar 4.14 menunjukkan grafik total *reward* selama proses pelatihan *insert policy*. Berdasarkan grafik tersebut, total *reward* mengalami peningkatan yang cukup signifikan dari fase awal hingga fase akhir pelatihan. Pada awal pelatihan, total *reward* berada pada kisaran sekitar -400. Nilai tersebut menunjukkan bahwa *policy* belum mampu menghasilkan aksi yang efektif untuk membawa objek menuju posisi target maupun menjaga kesesuaian orientasi objek terhadap lubang target.

Seiring bertambahnya jumlah *step* pelatihan, total *reward* meningkat secara bertahap dan mulai memasuki nilai positif. Pada fase menengah pelatihan, total *reward* berada pada kisaran sekitar 500 hingga 1000. Selanjutnya, pada fase akhir

pelatihan, total *reward* meningkat hingga berada pada kisaran sekitar 1500 sampai 1600. Peningkatan ini menunjukkan bahwa *insert policy* semakin mampu menghasilkan aksi yang sesuai dengan tujuan subtugas, yaitu membawa objek mendekati posisi target dan mengurangi *error* orientasi objek terhadap lubang target. Meskipun tren utama total *reward* menunjukkan peningkatan, terdapat beberapa penurunan tajam pada titik tertentu selama proses pelatihan. Penurunan tersebut dapat disebabkan oleh eksplorasi *policy*, variasi kondisi awal episode, posisi target yang lebih menantang, atau kondisi orientasi objek yang lebih sulit untuk diselaraskan.



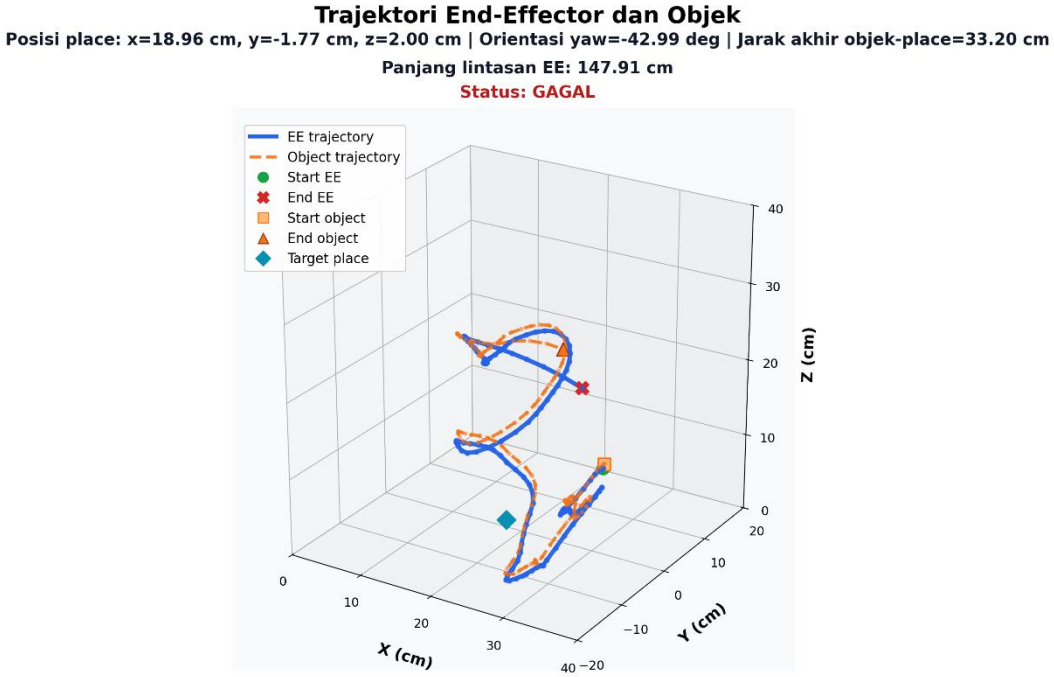
Gambar 4.14 Grafik Total *Reward Insert Policy*

Secara umum, peningkatan total *reward* menunjukkan bahwa *insert policy* berhasil mempelajari perilaku dasar untuk membawa objek menuju target. Hasil ini juga mendukung analisis pada *reward* posisi dan *reward* orientasi sebelumnya. *Reward* posisi menunjukkan bahwa objek semakin mampu diarahkan mendekati posisi target, sedangkan *reward* orientasi menunjukkan bahwa *policy* mulai mampu menjaga kesesuaian orientasi objek terhadap lubang target. Dengan demikian, peningkatan total *reward* menunjukkan bahwa kombinasi antara pengurangan jarak, pengurangan *error* orientasi, dan keberhasilan mencapai target lubang memberikan kontribusi positif terhadap proses pembelajaran *insert policy*.

Setelah menganalisis *reward* posisi, *reward* orientasi, dan total *reward*, hasil pelatihan *insert policy* juga dianalisis melalui contoh trajektori objek atau *end-effector* pada beberapa *checkpoint* pelatihan. Analisis trajektori digunakan untuk

melihat perubahan perilaku *policy* secara visual, terutama dalam hal arah gerakan, kestabilan lintasan, serta kemampuan objek bergerak menuju target. Dengan melihat trajektori, proses pembelajaran tidak hanya dinilai dari nilai *reward*, tetapi juga dari pola gerakan yang dihasilkan selama pelatihan.

Gambar 4.15 menunjukkan contoh trajektori *insert policy* pada *timestep* ke-5.000. Pada fase awal pelatihan ini, lintasan *end-effector* dan objek masih menunjukkan pola gerakan yang belum stabil. *Policy* belum mampu menghasilkan aksi yang konsisten untuk membawa objek menuju posisi target. Hal ini ditunjukkan oleh status episode yang masih gagal.



Gambar 4.15 Trajektori *End-Effector* Robot pada Step ke-5.000

Trajektori pada gambar tersebut menunjukkan posisi akhir *end-effector* dan posisi akhir objek terlihat berbeda. Kondisi ini mengindikasikan bahwa objek telah terlepas dari genggaman *gripper* sebelum tugas selesai. Dengan kata lain, *end-effector* masih melanjutkan pergerakan, tetapi objek tidak lagi bergerak mengikuti lintasan *end-effector*. Hal ini menunjukkan bahwa pada fase awal pelatihan, *insert policy* belum mampu menghasilkan gerakan yang cukup stabil untuk mempertahankan objek tetap dalam genggaman selama proses menuju target.

Selain itu, trajektori yang dihasilkan juga masih menunjukkan pola gerakan yang belum stabil dan cenderung berfluktuasi.

Oleh karena itu, trajektori pada *timestep* ke-5.000 menunjukkan bahwa *insert policy* masih berada pada tahap eksplorasi. *End-effector* belum memiliki pola gerakan yang stabil, sementara objek belum dapat dibawa menuju target secara konsisten. Hasil ini sejalan dengan grafik *reward* sebelumnya, di mana *reward* posisi, *reward* orientasi, dan total *reward* pada fase awal pelatihan masih menunjukkan performa yang relatif rendah.

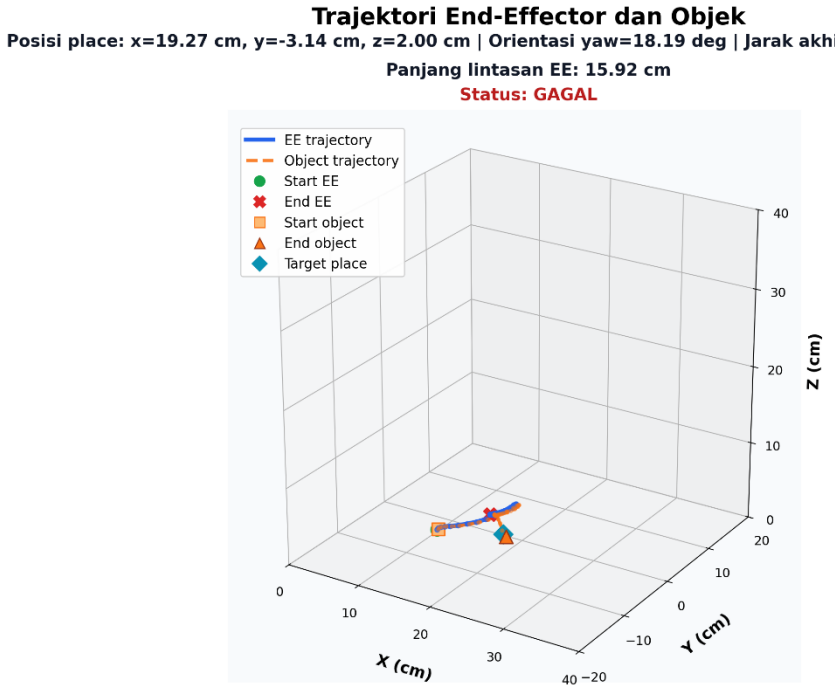
Gambar 4.16 menunjukkan contoh trajektori *insert policy* pada *timestep* ke-500.000. Pada fase ini, trajektori *end-effector* dan objek sudah menunjukkan perbaikan dibandingkan pada *timestep* ke-5.000. Lintasan yang dihasilkan terlihat lebih terarah menuju area target, dan objek mulai dapat mengikuti pergerakan *end-effector* dengan lebih baik. Hal ini menunjukkan bahwa *policy* mulai mempelajari hubungan antara gerakan *end-effector*, posisi objek, dan posisi target.

Meskipun demikian, contoh trajektori pada *timestep* ke-500.000 masih berstatus gagal. Jarak akhir objek terhadap target masih berada pada kisaran 0.51 cm, sehingga objek belum memenuhi batas toleransi keberhasilan. Namun, episode tersebut tetap berstatus gagal. Kondisi ini menunjukkan bahwa keberhasilan *insert policy* tidak hanya ditentukan oleh kedekatan posisi objek terhadap target, tetapi juga dapat dipengaruhi oleh faktor lain seperti kesesuaian orientasi objek terhadap lubang target atau terpenuhinya kriteria keberhasilan secara keseluruhan.

Dibandingkan dengan fase awal pelatihan, trajektori pada *timestep* ke-500.000 menunjukkan bahwa objek tidak lagi terlepas secara jelas dari pergerakan *end-effector*. Objek cenderung mengikuti lintasan *end-effector* hingga mendekati area target. Namun, *policy* belum sepenuhnya stabil dalam menghentikan objek pada posisi yang tepat dan menjaga kondisi akhir agar sesuai dengan kriteria keberhasilan. Dengan demikian, *timestep* ke-500.000 dapat dipahami sebagai fase menengah pelatihan, di mana *insert policy* sudah mulai menghasilkan perilaku yang relevan, tetapi belum cukup konsisten untuk menyelesaikan tugas secara andal.

Secara keseluruhan, analisis trajektori pada *timestep* ke-500.000 memperlihatkan adanya peningkatan kualitas gerakan dibandingkan *timestep* ke-

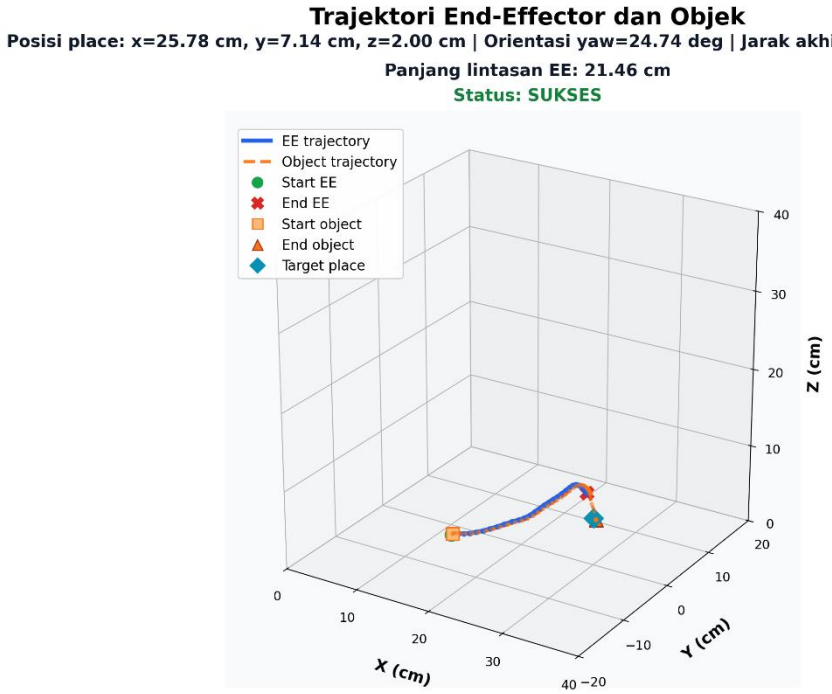
5.000. *Policy* mulai mampu membawa objek mendekati target, tetapi masih membutuhkan proses pelatihan lanjutan untuk meningkatkan ketepatan posisi akhir dan kesesuaian orientasi.



Gambar 4.16 Trajektori *End-Effector* Robot pada *Step* ke-500.000

Gambar 4.17 menunjukkan contoh trajektori pada fase akhir pelatihan. Pada fase ini, trajektori *end-effector* dan objek sudah menunjukkan pola gerakan yang jauh lebih stabil dibandingkan pada *timestep* ke-5.000 maupun *timestep* ke-500.000. Lintasan yang dihasilkan terlihat lebih stabil, lebih terarah, dan bergerak menuju area target tanpa penyimpangan yang besar. Kondisi ini menunjukkan bahwa *policy* telah mampu mempelajari strategi pergerakan yang lebih efektif untuk membawa objek menuju target.

Pada contoh trajektori tersebut, objek juga terlihat tetap mengikuti pergerakan *end-effector* sepanjang lintasan utama. Hal ini menunjukkan bahwa objek dapat dipertahankan dalam genggaman *gripper* selama proses pergerakan, berbeda dengan fase awal pelatihan ketika objek masih dapat terlepas dari genggaman. Kesesuaian lintasan antara *end-effector* dan objek mengindikasikan bahwa kestabilan trajektori yang dihasilkan sudah mengalami peningkatan yang signifikan.



Gambar 4.17 Trajektori *End-Effector* Robot pada *Step* ke-1.500.000

Keberhasilan *policy* juga terlihat dari posisi akhir objek yang berada sangat dekat dengan target. Pada contoh pertama, jarak akhir objek terhadap target tercatat sebesar 0,37 cm. Nilai tersebut menunjukkan bahwa objek telah berhasil dibawa menuju posisi target dengan tingkat ketelitian yang baik. Selain itu, status episode yang berhasil menunjukkan bahwa *policy* tidak hanya mampu mendekatkan objek ke target, tetapi juga memenuhi kriteria keberhasilan yang telah ditetapkan pada lingkungan pelatihan.

Secara keseluruhan, trajektori pada fase akhir pelatihan menunjukkan bahwa *insert policy* telah mampu menghasilkan gerakan yang stabil, terarah, dan konsisten dalam membawa objek menuju target. Hasil ini memperlihatkan perkembangan dibandingkan fase awal dan fase menengah pelatihan, di mana sebelumnya objek masih dapat terlepas dari *gripper* atau belum berhenti pada posisi target secara tepat. Dengan demikian, analisis trajektori pada fase akhir memperkuat hasil grafik *reward* sebelumnya bahwa *insert policy* telah berhasil mempelajari subtugas membawa objek menuju target dengan baik.

Setelah analisis trajektori dilakukan, *insert policy* dievaluasi melalui 100 episode pengujian pada lingkungan simulasi. Evaluasi ini dilakukan untuk

mengukur kemampuan *policy* dalam membawa objek yang telah tergenggam menuju posisi target secara konsisten. Pada setiap episode evaluasi, posisi dan orientasi awal objek serta posisi target divariasikan dalam batas *workspace* robot.

Kriteria keberhasilan pada evaluasi *insert policy* ditentukan berdasarkan kemampuan untuk menyisipkan objek ke dalam target. Selain itu, objek juga harus tetap berada dalam kondisi terkendali selama proses pergerakan hingga mencapai target. Insert *policy* dinyatakan memadai apabila memperoleh tingkat keberhasilan minimal 90% dari total 100 episode pengujian. Berdasarkan hasil evaluasi, *Insert Policy* berhasil menyelesaikan 95 dari 100 episode pengujian, sehingga memperoleh *success rate* sebesar 95%. Rata-rata jumlah *step* yang dibutuhkan untuk mencapai kondisi berhasil adalah 110 *step*. Nilai tersebut menunjukkan bahwa *policy* telah memenuhi kriteria minimal yang ditetapkan, yaitu *success rate* sebesar 90%. Hasil ini menunjukkan bahwa *insert policy* mampu membawa objek menuju posisi target secara konsisten pada berbagai variasi kondisi awal.

Meskipun demikian, beberapa episode masih mengalami kegagalan. Kegagalan tersebut dapat disebabkan oleh objek yang terlepas dari genggaman gripper, posisi target yang lebih sulit dijangkau, atau episode yang mencapai batas maksimum *step* sebelum kondisi keberhasilan terpenuhi. Namun, karena tingkat keberhasilan sudah melampaui batas minimal yang ditentukan, *insert policy* dinyatakan memadai untuk digunakan sebagai *primitive policy* pada tahap pelatihan *target policy*.

4.2.3 Hasil Pelatihan *Target Policy*

Setelah *grasp policy* dan *insert policy* selesai dilatih serta memenuhi kriteria evaluasi yang telah ditentukan, kedua *primitive policy* tersebut digunakan pada tahap pelatihan *target policy*. *Target policy* merupakan kebijakan utama yang dilatih untuk menyelesaikan tugas secara *end-to-end*, mulai dari mengarahkan *end-effector* menuju objek, membawa objek, hingga mencapai posisi target.

Berbeda dengan *primitive policy* yang dilatih menggunakan *reward* informatif, *target policy* dilatih menggunakan *sparse reward*. Pada *reward* ini, agen hanya memperoleh nilai 0 ketika objek berhasil mencapai target, sedangkan kondisi lainnya diberikan *reward* -1. Penggunaan *sparse reward* membuat proses

pembelajaran menjadi lebih sulit karena agen tidak memperoleh informasi bertahap mengenai seberapa dekat objek terhadap target. Oleh karena itu, pendekatan QMP-HER digunakan untuk membantu proses pembelajaran *target policy*.

Pada pelatihan fase ini, *grasp* dan *insert policy* tidak lagi diperbarui parameternya, melainkan digunakan sebagai sumber kandidat aksi. Pada setiap langkah interaksi, kandidat aksi dihasilkan oleh *grasp policy*, *insert policy*, dan *target policy*. Seluruh kandidat aksi tersebut kemudian dievaluasi menggunakan *critic* dari *target policy*, dan aksi dengan nilai Q tertinggi dipilih untuk dieksekusi di lingkungan. Selain itu, *Hindsight Experience Replay* digunakan untuk memanfaatkan kembali pengalaman gagal melalui proses *goal relabeling*, sehingga pengalaman yang tidak mencapai target utama tetap dapat memberikan informasi pembelajaran. Pelatihan dilakukan selama 2.000.000 *timestep*. Konfigurasi hyperparameter utama yang digunakan pada pelatihan *target policy* ditunjukkan pada Tabel 4.3.

Tabel 4.3 Konfigurasi *Hyperparameter Target Policy*

Hyperparameter	Nilai
Total *timesteps*	2.000.000
Learning rate	3×10^{-5}
Learning starts	5.000
Buffer size	1.000.000
Batch size	256
Discount factor (γ)	0,99
Soft update coefficient (τ)	0,005
Entropy coefficient	0,01
Actor / policy network	[512, 512, 256]
Critic / Q-function	[512, 512, 256]
Activation function	ReLU

Hindsight Experience Replay digunakan dengan jumlah *sampled goal* sebanyak 4 dan strategi pemilihan *goal* berupa *future*. Nilai *sampled goal* berarti setiap transisi asli dapat dipasangkan dengan empat *goal* tambahan hasil *relabeling*. Dengan demikian, pengalaman yang semula dianggap gagal terhadap target utama

tetap dapat digunakan sebagai data pembelajaran dengan mengganti *goal* berdasarkan keadaan lain yang berhasil dicapai selama episode.

Strategi *future* berarti *goal* tambahan dipilih dari *state* yang terjadi setelah transisi tertentu dalam episode yang sama. Dengan kata lain, ketika agen berada pada suatu *timestep*, HER dapat mengambil keadaan yang dicapai pada *timestep* berikutnya sebagai *goal* alternatif. Strategi ini sesuai untuk tugas manipulasi objek karena meskipun agen gagal mencapai target utama, agen biasanya tetap mencapai posisi tertentu selama episode. Posisi yang berhasil dicapai tersebut dapat dijadikan *goal* alternatif sehingga transisi yang awalnya tidak berhasil tetap dapat memberikan sinyal pembelajaran. Pendekatan ini membantu *policy* belajar pada kondisi *sparse reward*.

Pada pelatihan tahap ini, mekanisme *Q-Switch* digunakan untuk memilih kandidat aksi terbaik dari *grasp policy*, *insert policy*, dan *target policy*. Pemilihan aksi dilakukan berdasarkan estimasi nilai Q dari *critic target policy*. Konfigurasi *Q-Switch* yang digunakan ditunjukkan pada Tabel 4.4.

Tabel 4.4 Konfigurasi *Q-Switch* pada Pelatihan *Target Policy*

Parameter *Q-Switch*	**Nilai**
Epsilon *initial*	0,2
Epsilon *final*	0,02
Epsilon *decay steps*	100.000
Target candidate starts	100.000
Target include probability ramp steps	300.000
Q aggregation	Minimum
Target Q *margin*	0,3

Pada konfigurasi ini, nilai *epsilon initial* ditetapkan sebesar 0,2 dan *epsilon final* sebesar 0,02. Nilai tersebut menunjukkan bahwa pada awal pelatihan masih terdapat peluang eksplorasi sebesar 20% dalam pemilihan aksi, kemudian peluang eksplorasi tersebut diturunkan secara bertahap hingga menjadi 2%. Penurunan epsilon dilakukan selama 100.000 *timestep* melalui parameter *epsilon decay steps*. Dengan demikian, pada fase awal pelatihan sistem masih memberikan ruang

eksplorasi yang cukup besar, sedangkan pada fase berikutnya pemilihan aksi menjadi lebih bergantung pada estimasi nilai Q.

Parameter *target candidate starts* ditetapkan sebesar 100.000 *timestep*. Artinya, pada 100.000 *timestep* pertama, *target policy* belum langsung dijadikan kandidat dalam mekanisme *Q-Switch*. Pada fase ini, pemilihan aksi lebih banyak bergantung pada *primitive policy* yang telah dilatih sebelumnya. Hal ini dilakukan karena pada awal pelatihan *target policy* belum memiliki kemampuan yang stabil, sehingga aksi dari *grasp policy* dan *insert policy* digunakan sebagai panduan eksplorasi awal.

Setelah melewati 100.000 *timestep*, *target policy* mulai dimasukkan sebagai kandidat aksi secara bertahap. Proses peningkatan keterlibatan *target policy* diatur melalui parameter *target include probability ramp steps* sebesar 300.000 *timestep*. Dengan konfigurasi ini, *target policy* tidak langsung digunakan secara penuh, tetapi keterlibatannya dinaikkan secara bertahap hingga semakin sering ikut dibandingkan pada fase awal. Tujuannya adalah agar *target policy* memiliki waktu untuk belajar dari *replay buffer* sebelum bersaing secara penuh dengan kandidat aksi dari *primitive policy*.

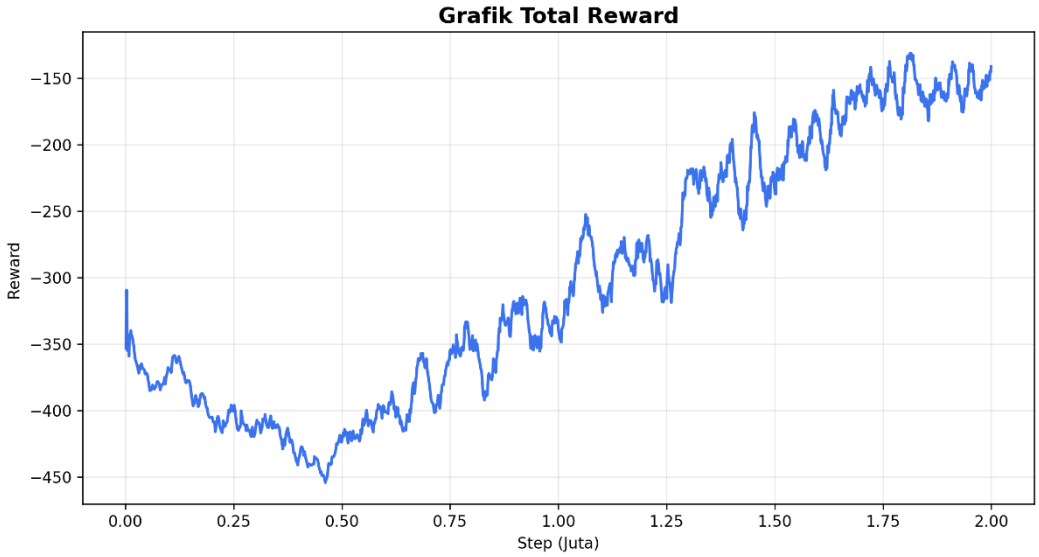
Parameter Q *aggregation* menggunakan nilai minimum. Hal ini berarti ketika terdapat lebih dari satu estimasi nilai Q, sistem menggunakan nilai Q yang lebih kecil sebagai acuan pemilihan aksi. Pendekatan ini digunakan untuk mengurangi risiko *overestimation* pada *critic*. Dengan menggunakan nilai minimum, aksi yang dipilih cenderung lebih konservatif karena harus memiliki estimasi nilai yang baik pada evaluasi *critic* yang lebih ketat.

Parameter target Q *margin* ditetapkan sebesar 0,3. *Margin* ini digunakan agar *target policy* tidak langsung dipilih hanya karena memiliki nilai Q yang sedikit lebih tinggi. *Target policy* perlu memiliki keunggulan nilai Q yang cukup terhadap kandidat lain agar dapat dipilih. Dengan demikian, mekanisme ini membantu mencegah *target policy* yang masih belum stabil mengambil alih pemilihan aksi terlalu cepat pada fase awal pelatihan.

Konfigurasi *Q-Switch* ini dirancang agar proses pelatihan *target policy* berlangsung secara bertahap. Pada fase awal, sistem lebih banyak memanfaatkan

primitive policy sebagai sumber aksi yang sudah memiliki kemampuan dasar. Setelah *target policy* mulai belajar, keterlibatannya dalam pemilihan aksi meningkat secara perlahan. Penggunaan epsilon decay, target candidate starts, target include probability ramp, serta *margin* nilai Q membantu menjaga keseimbangan antara eksplorasi, pemanfaatan *primitive policy*, dan pembelajaran *target policy* sebagai *policy* utama.

Gambar 4.18 menunjukkan grafik total *reward* selama proses pelatihan *target policy* menggunakan pendekatan QMP-HER. Pada skema *reward* ini, agen memperoleh *reward* 0 ketika objek berhasil mencapai target, sedangkan kondisi lainnya memperoleh *reward* -1. Oleh karena itu, nilai total *reward* pada *target policy* cenderung berada pada rentang negatif. Semakin besar nilai *reward* atau semakin mendekati nol, maka semakin baik performa *policy* karena menunjukkan bahwa agen semakin sering mencapai target atau membutuhkan jumlah langkah yang lebih sedikit untuk mencapai kondisi berhasil.



Gambar 4.18 Grafik Total *Reward Target Policy*

Berdasarkan grafik tersebut, total *reward* pada awal pelatihan berada pada kisaran sekitar -350. Nilai ini menunjukkan bahwa sejak fase awal pelatihan, *target policy* tidak sepenuhnya memulai proses pembelajaran dari eksplorasi acak. Hal ini berkaitan dengan penggunaan *primitive policy* melalui mekanisme *Q-Switch*. Pada fase awal, kandidat aksi dari *grasp policy* dan *insert policy* yang telah dilatih

sebelumnya dapat membantu sistem menghasilkan aksi yang lebih bermakna dibandingkan eksplorasi acak sepenuhnya. Dengan demikian, *reward* awal *target policy* tidak berada pada nilai yang terlalu rendah.

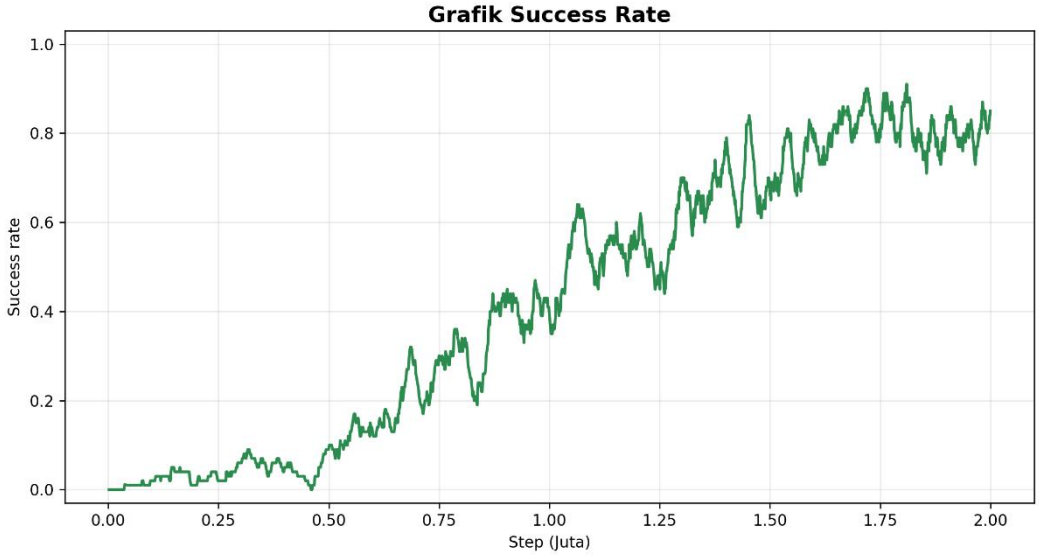
Meskipun demikian, pada fase awal hingga sekitar 400.000 *timestep*, total *reward* sempat mengalami penurunan hingga mencapai kisaran sekitar -450. Penurunan ini menunjukkan bahwa *target policy* masih berada pada tahap adaptasi awal. Pada fase ini, *critic* dari *target policy* masih menyesuaikan estimasi nilai Q terhadap kandidat aksi yang berasal dari *primitive policy* dan *target policy.* Selain itu, karena *reward* yang digunakan bersifat *sparse*, perubahan kecil pada tingkat keberhasilan episode dapat menyebabkan fluktuasi *reward* yang cukup besar.

Setelah melewati fase awal tersebut, total *reward* mulai menunjukkan peningkatan secara bertahap. Pada kisaran 600.000 hingga 1.200.000 *timestep*, total *reward* meningkat dari kisaran sekitar -400 menuju kisaran sekitar -300. Selanjutnya, setelah melewati sekitar 1.300.000 *timestep*, peningkatan *reward* menjadi lebih jelas hingga mencapai kisaran sekitar -150 pada fase akhir pelatihan. Peningkatan ini menunjukkan bahwa *target policy* semakin mampu menghasilkan aksi yang efektif untuk menyelesaikan tugas utama, yaitu membawa objek menuju posisi target. Dengan demikian, peningkatan total *reward* yang semakin mendekati nol menunjukkan bahwa *target policy* semakin sering mencapai target atau mampu menyelesaikan episode dengan jumlah langkah yang lebih sedikit.

Gambar 4.19 menunjukkan grafik *success rate target policy* selama proses pelatihan. *Success rate* digunakan sebagai metrik utama untuk menilai performa *target policy* karena *reward* yang digunakan bersifat *sparse*. Pada fase awal pelatihan, *success rate* masih berada sangat rendah, yaitu mendekati 0. Hal ini menunjukkan bahwa *target policy* belum mampu menyelesaikan tugas utama secara konsisten pada tahap awal pelatihan. Kondisi tersebut wajar karena *target policy* masih berada pada tahap awal pembelajaran dan belum sepenuhnya mampu menghasilkan aksi yang stabil.

Setelah melewati sekitar 500.000 *timestep*, *success rate* mulai menunjukkan peningkatan secara bertahap. Pada kisaran 800.000 hingga 1.200.000 *timestep*, *success rate* meningkat cukup signifikan dan mulai berada pada kisaran sekitar 0,4

hingga 0,6. Peningkatan ini menunjukkan bahwa *target policy* mulai mampu memanfaatkan pengalaman dari *replay buffer* dan hasil *relabeling* HER untuk menghasilkan aksi yang lebih efektif.



Gambar 4.19 Grafik *Success Rate*

Pada fase akhir pelatihan, *success rate* mencapai kisaran sekitar 0,8 hingga 0,85. Nilai ini menunjukkan bahwa *target policy* sudah mampu menyelesaikan tugas utama dengan tingkat keberhasilan yang cukup tinggi. Meskipun masih terdapat fluktuasi, tren keseluruhan menunjukkan peningkatan yang jelas dari awal hingga akhir pelatihan. Peningkatan *success rate* ini memperkuat hasil grafik total *reward* bahwa pendekatan QMP-HER membantu *target policy* belajar pada kondisi *sparse reward*.

Grafik total *reward* dan *success rate* menunjukkan bahwa *target policy* mengalami peningkatan performa selama proses pelatihan. Total *reward* yang semakin mendekati nol menunjukkan bahwa episode berhasil semakin sering terjadi atau jumlah langkah menuju keberhasilan semakin berkurang. Sementara itu, peningkatan *success rate* menunjukkan bahwa *policy* semakin mampu menyelesaikan tugas utama secara konsisten. Dengan demikian, hasil ini menunjukkan bahwa kombinasi *primitive policy*, mekanisme *Q-Switch*, dan HER memberikan kontribusi positif terhadap pelatihan *target policy*.

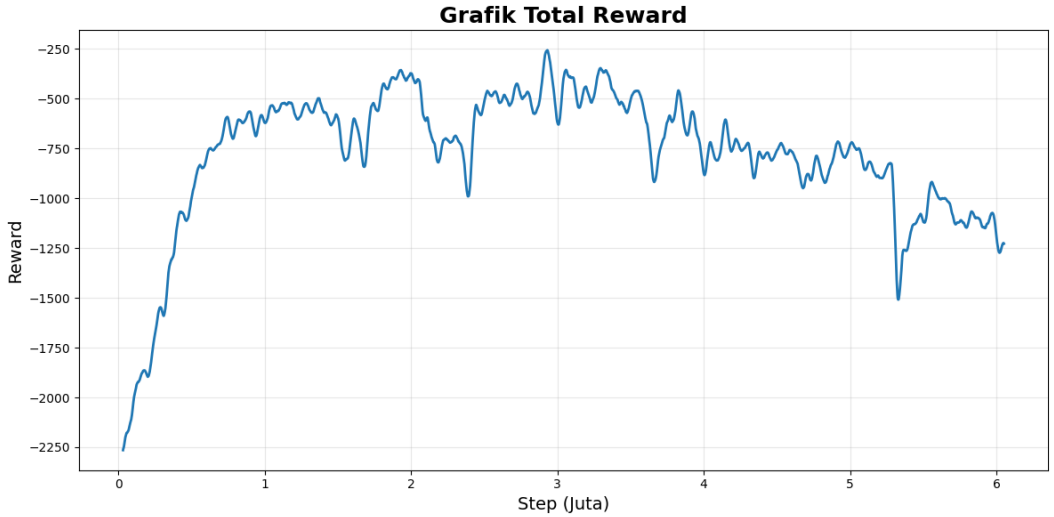
Selain dianalisis berdasarkan *reward* dan *success rate target policy* dengan pendekatan QMP-HER, pelatihan *target policy* juga dibandingkan dengan pelatihan SAC tanpa mekanisme QMP-HER. Perbandingan ini dilakukan untuk melihat pengaruh penggunaan *primitive policy* dan QMP-HER terhadap efisiensi sampel selama proses pelatihan. Efisiensi sampel dalam konteks ini dilihat dari kemampuan metode untuk menghasilkan peningkatan performa dengan jumlah *timestep* yang lebih sedikit.

Gambar 4.20 menunjukkan grafik total *reward* pada pelatihan SAC standar. Berbeda dengan *target policy* yang dilatih menggunakan QMP-HER, pelatihan SAC standar masih menunjukkan performa yang sangat rendah meskipun telah dilatih hingga 6.000.000 *timestep*. *Reward* yang diperoleh tidak menunjukkan peningkatan yang signifikan menuju kondisi berhasil. Hal ini menunjukkan bahwa SAC standar masih kesulitan memperoleh kebijakan terbaik pada tugas yang kompleks.

Kesulitan tersebut terjadi karena SAC standar harus mempelajari seluruh rangkaian tugas secara langsung melalui eksplorasi, mulai dari mendekati objek, menghasilkan kondisi *grasping*, membawa objek menuju target, hingga mencapai kondisi akhir. Pada tugas yang bersifat *sparse reward*, agen hanya memperoleh sinyal keberhasilan ketika tujuan akhir tercapai. Akibatnya, sebagian besar pengalaman yang diperoleh pada fase awal pelatihan tidak memberikan informasi yang cukup untuk memperbaiki *policy*. Berdasarkan hasil pengamatan, SAC standar belum mampu menghasilkan perilaku *grasping* yang stabil meskipun telah dilatih hingga 6.000.000 *timestep*.

Selain dibandingkan dengan SAC standar, *target policy* juga dibandingkan dengan *baseline* SAC+HER. *Baseline* ini digunakan untuk mengetahui apakah mekanisme *Hindsight Experience Replay* saja sudah cukup untuk membantu agen mempelajari tugas penyisipan objek pada lingkungan yang sama. Pada SAC+HER, agen menggunakan ruang keadaan, ruang aksi, fungsi *reward sparse*, dan konfigurasi lingkungan yang sama dengan *target policy* SAC berbasis QMP-HER. Perbedaannya adalah SAC+HER tidak menggunakan *primitive policy* dan

mekanisme *Q-Switch*, sehingga agen hanya mengandalkan *policy* utama dan proses *goal relabeling* dari HER.



Gambar 4.20 Grafik Total *Reward* SAC Standar

Gambar 4.21 menunjukkan grafik total *reward* SAC+HER selama proses pelatihan. Grafik ditampilkan hingga 6.000.000 *timestep* agar konsisten dengan baseline SAC standar. Berdasarkan grafik tersebut, total *reward* SAC+HER cenderung konstan pada kisaran -500 dan tidak menunjukkan peningkatan yang signifikan. Karena *reward target policy* dirancang dalam bentuk *sparse reward*, yaitu bernilai 0 ketika objek berhasil mencapai target dan -1 pada kondisi lainnya, nilai total *reward* sekitar -500 menunjukkan bahwa sebagian besar episode berakhir pada batas maksimum 500 *step* tanpa mencapai kondisi sukses.

Hasil ini menunjukkan bahwa mekanisme HER saja belum cukup untuk mengatasi kesulitan eksplorasi pada tugas penyisipan objek. Meskipun HER dapat memanfaatkan kembali pengalaman gagal melalui proses *goal relabeling*, agen tetap harus menemukan sendiri rangkaian aksi awal yang bermakna. Pada lingkungan penelitian ini, SAC+HER belum mampu menghasilkan perilaku penyisipan objek yang stabil hingga 6.000.000 *timestep*. Oleh karena itu, grafik *success rate* SAC+HER tidak ditampilkan secara terpisah karena nilainya tidak menunjukkan peningkatan yang bermakna dan tetap berada pada 0%.



Gambar 4.21 Grafik Total *Reward* SAC+HER

Berbeda dengan SAC standar dan SAC+HER, *target policy* dengan pendekatan SAC berbasis QMP-HER menunjukkan peningkatan performa yang lebih jelas. Pada metode ini, *grasp policy* dan *insert policy* yang telah dilatih sebelumnya digunakan sebagai sumber kandidat aksi melalui mekanisme *Q-Switch*. Dengan demikian, *target policy* tidak sepenuhnya bergantung pada eksplorasi acak atau *goal relabeling* dari HER, tetapi memperoleh bantuan dari *primitive policy* yang telah memiliki perilaku dasar. Perbedaan ini membuat proses eksplorasi menjadi lebih terarah, sehingga *target policy* mampu menunjukkan peningkatan *reward* dan *success rate* dalam jumlah *timestep* yang lebih sedikit. Ringkasan perbandingan hasil pelatihan target *policy* pada ketiga metode ditunjukkan pada Tabel 4.5.

Tabel 4.5 Ringkasan Perbandingan Pelatihan *Target Policy*

Metode	**Komponen Utama**	**Timestep**	**Hasil Pelatihan**
SAC standar	SAC tanpa HER dan tanpa *primitive policy*	6.000.000	*Reward* tidak meningkat signifikan, belum menghasilkan *grasping* stabil
SAC+HER	SAC dengan *goal relabeling* HER	6.000.000	*Reward* tetap sekitar -500; *success rate* tetap 0%
SAC+QMP-HER	SAC+HER dengan mekanisme QMP	5.500.000 (total dengan pelatihan *primitive*)	*Reward* meningkat *dan success rate* mencapai sekitar 0,8–0,85

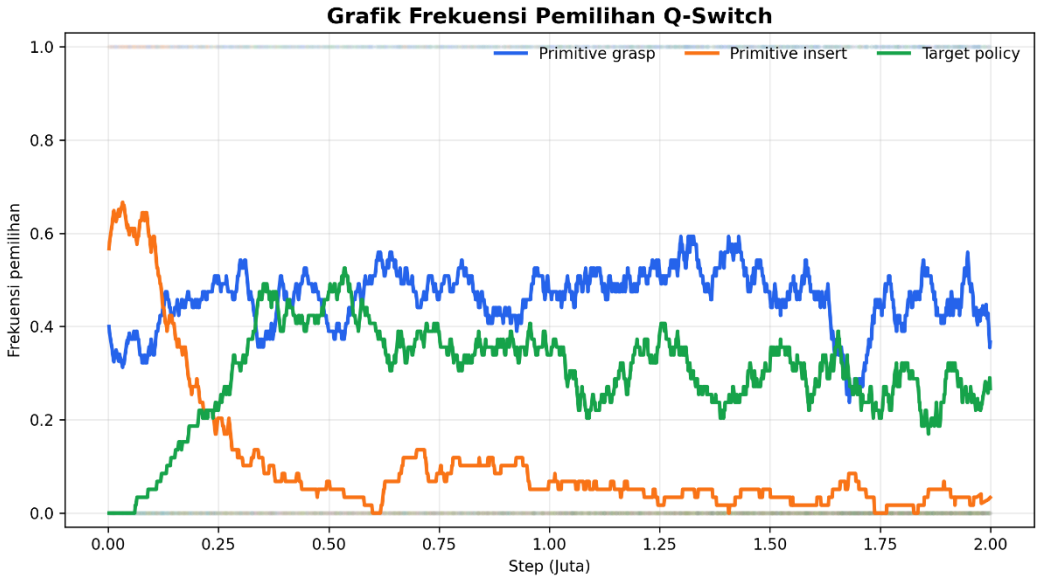
Dengan demikian, dari sisi efisiensi sampel, SAC berbasis QMP-HER menunjukkan performa yang lebih baik dibandingkan SAC standar dan SAC+HER. *Target policy* tidak hanya mengalami peningkatan *reward* dan *success rate* dalam jumlah *timestep* yang lebih sedikit, tetapi juga mampu mempelajari urutan tugas yang lebih kompleks. Hasil ini memperkuat bahwa QMP-HER berperan penting dalam membantu proses pembelajaran pada tugas manipulasi objek dengan tugas yang kompleks.

Gambar 4.22 menunjukkan proporsi pemilihan aksi oleh mekanisme *Q-Switch* selama proses pelatihan *target policy*. Pada grafik tersebut, terdapat tiga sumber kandidat aksi, yaitu *grasp policy*, *insert policy*, dan *target policy*. Grafik ini dapat digunakan untuk melihat bagaimana mekanisme *Q-Switch* memanfaatkan *primitive policy* dan *target policy* selama proses pelatihan. Pada fase awal pelatihan, *insert policy* memiliki proporsi pemilihan yang cukup tinggi, yaitu berada pada kisaran sekitar 0,55 hingga 0,65. Sementara itu, *target policy* hampir tidak dipilih pada awal pelatihan. Kondisi ini menunjukkan bahwa pada tahap awal, *target policy* belum mampu menghasilkan kandidat aksi yang kompetitif. Oleh karena itu, mekanisme *Q-Switch* lebih sering memilih aksi dari *primitive policy*, terutama *insert policy*, karena *primitive policy* tersebut telah memiliki kemampuan awal untuk membawa objek menuju target.

Seiring bertambahnya *timestep* pelatihan, proporsi pemilihan *insert policy* mengalami penurunan secara bertahap. Penurunan ini menunjukkan bahwa aksi dari *insert policy* tidak lagi selalu menjadi kandidat dengan estimasi nilai Q tertinggi setelah *target policy* mulai belajar dari pengalaman. Pada saat yang sama, proporsi pemilihan *target policy* mulai meningkat, terutama setelah melewati fase awal pelatihan. Peningkatan proporsi *target policy* menunjukkan bahwa *policy* utama mulai mampu menghasilkan aksi yang lebih kompetitif dibandingkan kandidat aksi dari *primitive policy* pada sebagian kondisi.

Grasp policy juga masih dipilih dalam proporsi yang cukup besar selama proses pelatihan, yaitu berada pada kisaran sekitar 0,4 hingga 0,55 pada beberapa fase. Hal ini menunjukkan bahwa kemampuan dasar dari *grasp policy* masih relevan dalam pelatihan *target policy*. Walaupun tugas akhir *target policy* adalah

membawa objek menuju posisi target, proses tersebut tetap memerlukan perilaku dasar untuk mendekati objek, menjaga hubungan *end-effector* terhadap objek, dan mempertahankan kondisi manipulasi yang stabil. Oleh karena itu, pemilihan *grasp policy* oleh *Q-Switch* masih dapat terjadi ketika kandidat aksinya dinilai lebih menguntungkan oleh *critic target policy*.



Gambar 4.22 Grafik Frekuensi Pemilihan *Q-Switch*

Pada fase akhir pelatihan, *target policy* sudah dipilih dalam proporsi yang lebih besar dibandingkan fase awal. Hal ini menunjukkan bahwa *target policy* telah mengalami peningkatan kemampuan dan mulai mampu menghasilkan aksi yang bernilai tinggi berdasarkan estimasi *critic*. Namun, *target policy* tidak sepenuhnya mendominasi pemilihan aksi. *Primitive policy* masih tetap dapat dipilih ketika nilai Q dari kandidat aksinya lebih tinggi. Kondisi ini menunjukkan bahwa mekanisme *Q-Switch* tetap memilih kandidat aksi yang dinilai paling baik pada *state* tertentu.

Perlu diperhatikan bahwa pemilihan *primitive policy* pada fase akhir tidak berarti *target policy* gagal belajar. Sebaliknya, hal tersebut menunjukkan bahwa *Q-Switch* tetap menjalankan fungsinya sebagai mekanisme seleksi aksi berbasis nilai Q. Jika pada suatu keadaan aksi dari *grasp policy* atau *insert policy* memiliki estimasi nilai yang lebih baik, maka aksi tersebut tetap dapat dipilih. Dengan demikian, *primitive policy* berperan sebagai pendukung eksplorasi dan sumber

perilaku dasar selama pelatihan, sedangkan *target policy* secara bertahap belajar menghasilkan aksi yang semakin kompetitif.

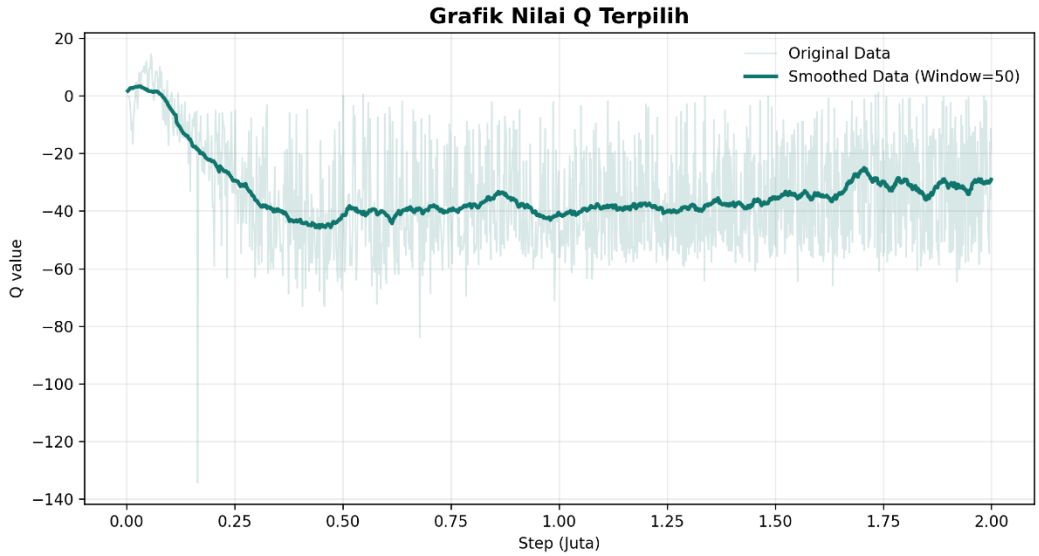
Grafik pemilihan *Q-Switch* menunjukkan bahwa *primitive policy* memberikan kontribusi penting pada fase awal pelatihan *target policy*. *Insert policy* lebih dominan pada fase awal karena telah memiliki kemampuan membawa objek menuju target, sedangkan *grasp policy* tetap relevan karena berhubungan dengan kestabilan manipulasi objek. Seiring proses pelatihan berlangsung, proporsi pemilihan *target policy* meningkat, yang menunjukkan bahwa *target policy* mulai mengambil peran lebih besar dalam menghasilkan aksi. Hasil ini mendukung tujuan penggunaan QMP-HER, yaitu memanfaatkan *primitive policy* untuk memberikan eksplorasi yang lebih bermakna dan membantu *target policy* belajar pada tugas yang memiliki *sparse reward*.

Selain *reward*, *success rate*, dan proporsi pemilihan aksi oleh *Q-Switch*, proses pelatihan *target policy* juga dianalisis melalui nilai Q terpilih, *actor loss*, dan *critic loss.* Ketiga metrik ini digunakan untuk melihat kestabilan proses pembelajaran. Nilai Q digunakan untuk mengevaluasi kualitas suatu aksi pada *state* tertentu, *actor loss* digunakan untuk melihat proses pembaruan jaringan *actor*, sedangkan *critic loss* digunakan untuk melihat kestabilan jaringan *critic* dalam mempelajari estimasi nilai yang dihasilkan *actor*.

Gambar 4.23 menunjukkan grafik nilai Q terpilih selama proses pelatihan *target policy*. Nilai Q terpilih merepresentasikan estimasi nilai dari aksi yang dipilih oleh mekanisme *Q-Switch*. Nilai Q yang semakin besar atau semakin mendekati nol menunjukkan bahwa aksi yang dipilih diperkirakan memiliki peluang lebih baik untuk membawa agen menuju kondisi berhasil.

Pada fase awal pelatihan, nilai Q terpilih sempat berada pada nilai positif, kemudian mengalami penurunan cukup tajam hingga berada pada kisaran sekitar -40 sampai -45. Penurunan ini dapat dipahami sebagai proses penyesuaian *critic* terhadap struktur *reward sparse* yang digunakan. Pada awal pelatihan, *critic* belum memiliki estimasi nilai yang stabil terhadap kandidat aksi yang berasal dari *grasp policy*, *insert policy*, maupun *target policy*. Setelah data interaksi terkumpul di

replay buffer, *critic* mulai memperbaiki estimasi nilai Q berdasarkan *reward* dan transisi yang diperoleh.



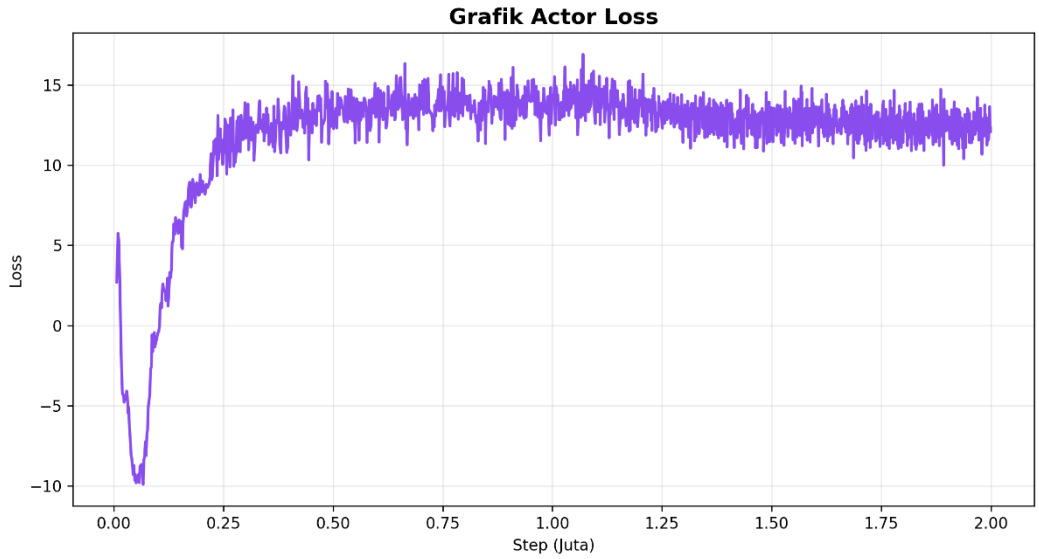
Gambar 4.23 Grafik Nilai Q Terpilih

Setelah fase awal, nilai Q terpilih mulai menunjukkan pola yang lebih stabil. Pada kisaran sekitar 500.000 hingga 1.300.000 *timestep*, nilai Q banyak berada pada kisaran sekitar -40. Selanjutnya, pada fase akhir pelatihan, nilai Q mengalami peningkatan bertahap hingga berada pada kisaran sekitar -30. Peningkatan ini menunjukkan bahwa *critic* mulai memberikan estimasi nilai yang lebih baik terhadap aksi yang dipilih. Hal ini sejalan dengan meningkatnya *success rate*, karena aksi yang dipilih oleh *Q-Switch* semakin sering mengarah pada keberhasilan mencapai target.

Actor loss digunakan untuk mengamati proses pembaruan jaringan *actor* selama pelatihan. Dalam SAC, *actor* bertugas menghasilkan aksi berdasarkan *state* yang diamati. Analisis *actor loss* mengacu pada fungsi objektif *actor* SAC yang telah dijelaskan pada persamaan (2.19). Berdasarkan persamaan tersebut, pembaruan actor tidak hanya diarahkan untuk memilih aksi dengan estimasi nilai Q yang tinggi, tetapi juga mempertimbangkan komponen entropi agar policy tetap memiliki kemampuan eksplorasi. Oleh karena itu, nilai *actor loss* tidak dapat dimaknai secara langsung bahwa semakin kecil nilainya maka semakin baik

performa *policy*. Nilai *actor loss* digunakan untuk melihat kestabilan proses pembaruan *policy*, yaitu apakah pembaruan parameter berlangsung terkendali atau mengalami perubahan yang terlalu ekstrem selama pelatihan.

Gambar 4.24 menunjukkan grafik *actor loss* selama pelatihan *target policy*. Pada fase awal pelatihan, *actor loss* mengalami perubahan yang cukup tajam. Nilai *actor loss* sempat turun hingga sekitar -10, kemudian meningkat secara bertahap hingga berada pada kisaran sekitar 12 sampai 14.



Gambar 4.24 Grafik *Actor Loss*

Perubahan tersebut menunjukkan bahwa jaringan *actor* masih berada pada fase adaptasi awal, yaitu menyesuaikan parameter *policy* terhadap estimasi nilai Q dari *critic* serta distribusi data awal yang terkumpul di *replay buffer*. Nilai *actor loss* yang meningkat pada fase ini tidak secara langsung menunjukkan penurunan performa, karena skala *actor loss* dipengaruhi oleh estimasi Q dan komponen entropi dalam objektif SAC.

Setelah melewati sekitar 300.000 *timestep*, *actor loss* cenderung berada pada kisaran yang lebih stabil. Kestabilan ini menunjukkan bahwa proses pembaruan *policy* berlangsung lebih terkendali dan tidak mengalami perubahan parameter yang ekstrem. Hal ini penting karena *target policy* harus belajar menghasilkan aksi yang semakin kompetitif dibandingkan kandidat aksi dari *primitive policy*. Apabila *actor loss* terus berfluktuasi tajam atau berubah secara

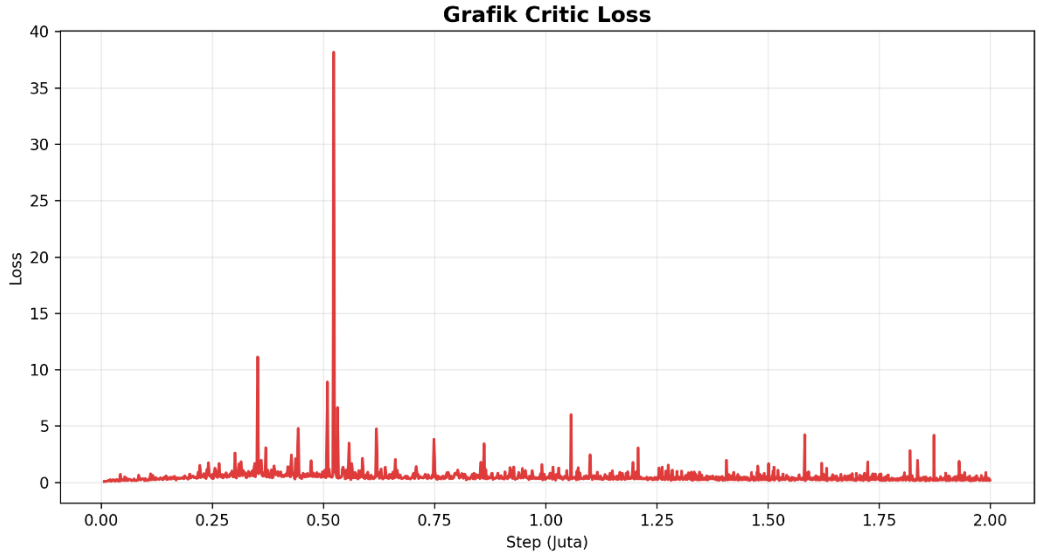
ekstrem sepanjang pelatihan, aksi yang dihasilkan *target policy* berpotensi menjadi tidak konsisten dan dapat mengganggu proses pemilihan aksi oleh mekanisme *Q-switch*. Pada fase akhir pelatihan, *actor loss* relatif stabil pada kisaran sekitar 12 hingga 13. Jika dikaitkan dengan peningkatan *success rate* yang telah dibahas sebelumnya, kondisi ini menunjukkan bahwa pembaruan *target policy* berlangsung konsisten selama performa kebijakan meningkat.

Critic loss digunakan untuk mengamati kemampuan jaringan *critic* dalam mempelajari estimasi nilai Q. Analisis *critic loss* mengacu pada fungsi kerugian *critic* SAC yang telah dijelaskan pada persamaan (2.12), dengan target nilai Q yang mengacu pada Persamaan (2.13). Pada persamaan tersebut, *critic loss* merepresentasikan selisih antara nilai Q yang diprediksi oleh *critic* dan target nilai Q yang digunakan dalam pembelajaran. Berbeda dengan *actor loss*, nilai *critic loss* yang lebih kecil secara umum menunjukkan bahwa estimasi nilai Q dari *critic* semakin mendekati target pembelajaran. Namun, nilai *critic loss* tetap perlu dianalisis bersama dengan *reward* dan *success rate*, karena *critic loss* yang kecil saja belum cukup untuk menyimpulkan bahwa *policy* sudah optimal.

.Gambar 4.25 menunjukkan grafik *critic loss* selama proses pelatihan target *policy*. Secara umum, *critic loss* berada pada nilai yang relatif rendah sepanjang pelatihan. Namun, terdapat beberapa lonjakan pada titik tertentu, terutama sekitar 500.000 *timestep*. Kondisi ini dapat terjadi akibat perubahan distribusi pengalaman dalam *replay buffer*, eksplorasi *policy*, atau proses *goal relabeling* pada HER.

Meskipun terdapat beberapa lonjakan, *critic loss* kembali berada pada kisaran rendah pada sebagian besar proses pelatihan. Hal ini menunjukkan bahwa *critic* masih mampu menyesuaikan estimasi nilai Q terhadap perubahan data pelatihan. Dengan kata lain, lonjakan sesaat pada *critic loss* tidak menunjukkan kegagalan pelatihan, selama tidak berkembang menjadi tren kenaikan yang berkelanjutan. Kestabilan *critic loss* pada fase akhir pelatihan menunjukkan bahwa jaringan *critic* mampu memberikan estimasi nilai aksi secara lebih konsisten. Hal ini penting karena mekanisme *Q-Switch* pada penelitian ini bergantung pada estimasi nilai Q dari *critic* untuk memilih kandidat aksi terbaik dari *grasp policy*, *insert policy*, dan *target policy*. Dengan *critic* yang relatif stabil, proses pemilihan

aksi oleh *Q-switch* dapat berjalan lebih konsisten dan mendukung peningkatan performa *target policy*.

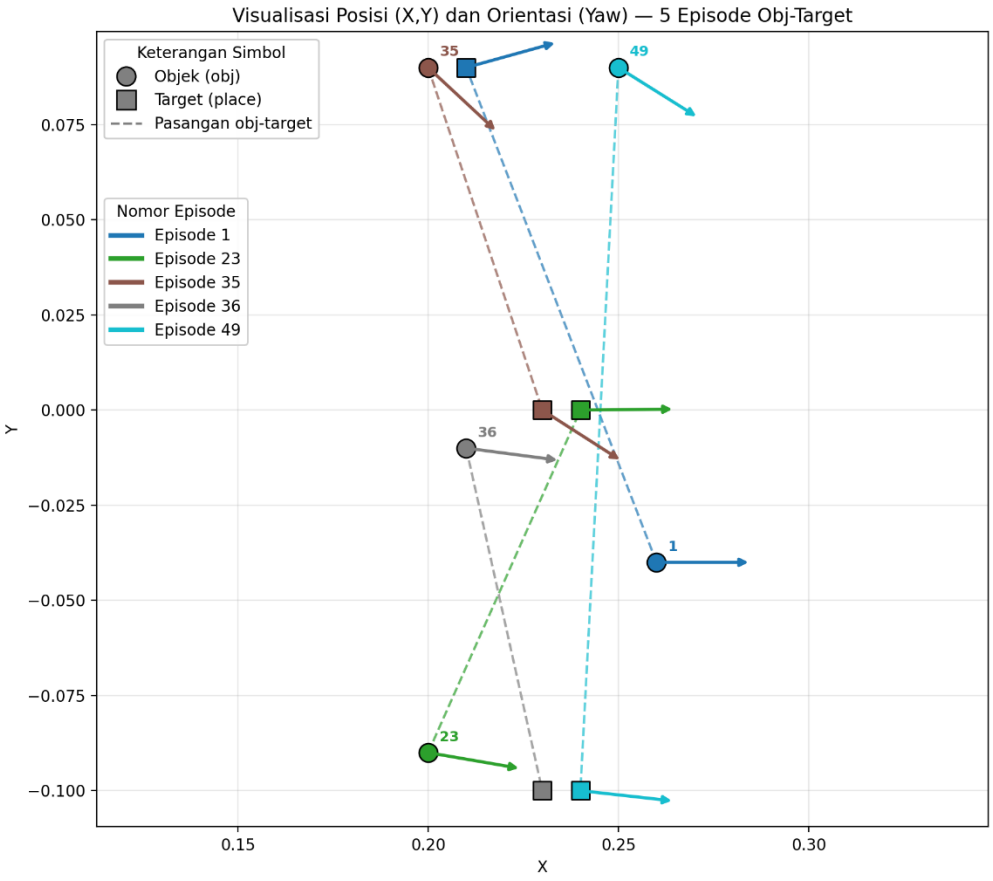


Gambar 4.25 Grafik *Critic Loss*

Secara keseluruhan, hasil pelatihan *target policy* menunjukkan bahwa SAC berbasis QMP-HER mampu membantu proses pembelajaran pada tugas dengan *sparse reward*. Hal ini terlihat dari total *reward* yang semakin mendekati nol, *success rate* yang meningkat hingga kisaran 0,8–0,85, serta performa yang lebih efisien dibandingkan SAC standar dan SAC+HER yang belum mampu menghasilkan *reward* optimal meskipun telah dilatih hingga 6.000.000 *timestep*. Mekanisme *Q-Switch* juga menunjukkan peran penting dalam memanfaatkan *grasp policy* dan *insert policy* sebagai sumber kandidat aksi pada fase awal pelatihan, kemudian secara bertahap memberi ruang lebih besar bagi *target policy* ketika kemampuannya meningkat. Selain itu, nilai Q terpilih, *actor loss*, dan *critic loss* menunjukkan bahwa proses optimasi berjalan relatif stabil meskipun terdapat fluktuasi akibat *sparse reward*, eksplorasi, dan *goal relabeling* pada HER. Dengan demikian, *target policy* yang dihasilkan telah memiliki dasar performa yang cukup baik untuk diuji lebih lanjut pada lingkungan simulasi dan lingkungan nyata.

4.3 Hasil Pengujian di Lingkungan Simulasi

Pengujian dilakukan pada lingkungan simulasi untuk mengevaluasi performa *target policy* dalam menyelesaikan tugas penyisipan objek pada tiga jenis objek, yaitu kubus, prisma segitiga, dan silinder. Setiap objek diuji sebanyak 50 episode. Penting untuk dicatat bahwa 50 episode ini disusun dengan kondisi awal yang sama antarobjek, sehingga posisi awal, orientasi awal objek, serta posisi dan orientasi lubang target pada episode ke-1 sampai ke-50 bersifat konsisten dan telah ditentukan sebelumnya. Gambar 4.26 menunjukkan 5 contoh randomisasi pose awal objek dan lubang target, detail koordinat posisi dan orientasi dari tiap episode berada di Lampiran 1.1. Dengan demikian, perbedaan hasil yang muncul pada setiap objek dapat diinterpretasikan sebagai akibat dari karakteristik objek dan respons kebijakan kontrol, bukan karena perbedaan skenario pengujian. Sementara itu, rincian hasil pengujian masing-masing objek disajikan pada Lampiran 1.2 untuk kubus, Lampiran 1.3 untuk silinder, dan Lampiran 1.4 untuk prisma segitiga.



Gambar 4.26 Contoh Randomisasi Pose Awal Objek dan Lubang Target

Berbeda dengan tahap pelatihan *target policy* yang menggunakan mekanisme QMP-HER untuk membantu proses pembelajaran, tahap pengujian dilakukan dengan menjalankan *target policy* sebagai *policy* utama. Dengan demikian, hasil pengujian ini digunakan untuk menilai sejauh mana *target policy* mampu menyelesaikan tugas secara mandiri setelah memperoleh bantuan dari *primitive policy* selama proses pelatihan.

Pada subbab ini, evaluasi dilakukan menggunakan beberapa indikator utama, yaitu *success rate*, jarak akhir objek terhadap lubang target, jumlah episode yang mencapai fase *release*, serta jumlah *step* sebagai representasi waktu penyelesaian pada lingkungan simulasi. Kriteria keberhasilan episode ditentukan berdasarkan jarak akhir objek terhadap lubang target. Episode dinyatakan berhasil apabila jarak akhir objek terhadap lubang target kurang dari atau sama dengan 5 mm. *Success rate* kemudian dihitung sebagai persentase episode berhasil terhadap total episode pengujian.

Pada pengujian ini, istilah *released* digunakan untuk menunjukkan episode ketika sistem telah menjalankan perintah membuka *gripper*. Fase *released* terjadi ketika objek dianggap telah mencapai kondisi pelepasan yang ditentukan oleh sistem, yaitu ketika objek berhasil dibawa menuju area target sesuai batas toleransi yang digunakan pada mekanisme kontrol. Dengan demikian, episode *released* menunjukkan jumlah episode ketika robot berhasil membawa objek hingga tahap pelepasan, tetapi belum tentu episode tersebut memenuhi kriteria keberhasilan akhir. Keberhasilan akhir tetap ditentukan berdasarkan jarak akhir objek terhadap lubang target setelah objek dilepaskan.

Sementara itu, metrik *success* setelah *released* menunjukkan rasio episode berhasil terhadap jumlah episode yang telah mencapai fase *released*. Metrik ini digunakan untuk membedakan kegagalan yang terjadi sebelum objek dilepaskan dan kegagalan yang terjadi setelah objek dilepaskan. Dengan demikian, analisis tidak hanya melihat *success rate* total, tetapi juga dapat menunjukkan apakah kegagalan lebih banyak terjadi pada proses membawa objek menuju target atau pada presisi posisi akhir setelah objek dilepaskan.

Tabel 4.6 Ringkasan Hasil Pengujian Simulasi

Objek	**Episode Sukses**	***Success Rate***	**Episode *released***	**Success setelah *released***	**Rata-rata jarak objek ke lubang target pada episode sukses**
Kubus	42	84%	48	87,50%	4,45 mm
Silinder	37	74%	41	90,24%	3,67 mm
Prisma Segitiga	37	74%	46	80,43%	4,90 mm

Berdasarkan Tabel 4.6, objek kubus menunjukkan performa terbaik dengan *success rate* sebesar 84%, sedangkan silinder dan prisma segitiga sama-sama memperoleh *success rate* sebesar 74%. Hasil ini menunjukkan bahwa *target policy* secara umum telah mampu menyelesaikan tugas penempatan untuk seluruh jenis objek, tetapi tingkat keberhasilannya berbeda-beda bergantung pada karakteristik objek yang ditangani.

Dari sisi jumlah episode yang mencapai fase *released*, kubus mencapai 48 episode, prisma segitiga 46 episode, dan silinder 41 episode. Hal ini menunjukkan bahwa kubus dan prisma segitiga relatif lebih sering berhasil mencapai tahap pelepasan objek, sedangkan silinder lebih sering mengalami kegagalan sebelum objek benar-benar dilepas. Namun demikian, jika hanya dilihat pada episode yang sudah berhasil *released*, silinder justru memiliki *success rate* setelah *release* paling tinggi, yaitu 90,24%, diikuti kubus 87,50%, dan prisma segitiga 80,43%. Temuan ini menunjukkan bahwa pada silinder, tantangan utama terletak pada proses transisi menuju *release*, sedangkan pada prisma segitiga, tantangan lebih banyak muncul setelah *release*, yaitu pada presisi posisi akhir objek terhadap lubang target. Secara keseluruhan, kubus merupakan objek dengan performa paling stabil. Silinder dan prisma segitiga memiliki tingkat keberhasilan total yang sama, tetapi menunjukkan pola kegagalan yang berbeda. Karena seluruh skenario episode dibuat sama antarobjek, maka perbedaan ini mengindikasikan bahwa bentuk dan karakteristik geometri objek memengaruhi kemudahan *policy* dalam menempatkan objek secara akurat.

Objek kubus memperoleh hasil terbaik dengan 42 episode sukses dari 50 episode, atau *success rate* sebesar 84%. Sebanyak 48 episode berakhir pada fase

released, yang menunjukkan bahwa *target policy* hampir selalu berhasil membawa objek sampai tahap *release*. Dari 48 episode yang berhasil dilepas, sebanyak 42 episode memenuhi kriteria keberhasilan penempatan. Dengan demikian, kubus dapat dikatakan sebagai objek yang paling mudah ditangani oleh *policy* dalam skenario pengujian ini. Jarak akhir objek ke posisi target pada episode sukses juga cukup baik, dengan rata-rata 4,45 mm, yang berarti berada di bawah ambang batas keberhasilan 5 mm. Hasil ini menunjukkan bahwa saat *target policy* berhasil pada objek kubus, penempatan yang dihasilkan relatif presisi dan stabil.

Meskipun demikian, masih terdapat 8 episode gagal. Kegagalan didominasi oleh kondisi ketika objek sudah dilepas tetapi posisi akhirnya sedikit melewati threshold keberhasilan. Episode seperti 7, 20, 23, dan 27 yang dapat dilihat di Lampiran 1.2, termasuk dalam kategori ini, dengan jarak akhir ke lubang target berada di kisaran 1,08–1,19 cm. Selain itu, terdapat juga beberapa kegagalan besar seperti episode 21 dan episode 25 yang juga dapat dilihat pada Lampiran 1.2, di mana objek berakhir cukup jauh dari posisi target. Oleh karena itu, walaupun kubus menunjukkan performa terbaik, masih terdapat kelemahan pada konsistensi posisi akhir pada beberapa episode. Secara umum, hasil pada objek kubus menunjukkan bahwa bentuk objek yang lebih stabil dan permukaan yang relatif sederhana kemungkinan mempermudah *policy* dalam mempertahankan kontrol saat proses *grasping* hingga *insert*.

Objek silinder memperoleh 37 episode sukses dari 50 episode, sehingga *success rate*-nya adalah 74%. Dibandingkan kubus, silinder menunjukkan tingkat keberhasilan yang lebih rendah. Dari seluruh episode, hanya 41 episode yang berhasil mencapai fase *released*, sehingga silinder merupakan objek yang paling sering gagal sebelum mencapai tahap *release*.

Namun, ketika silinder sudah berhasil dilepas, performanya sebenarnya cukup baik. Dari 41 episode *released*, sebanyak 37 episode berhasil, sehingga *success rate* setelah *release* mencapai 90,24%, yang merupakan nilai tertinggi di antara ketiga objek. Artinya, permasalahan utama pada silinder bukan terletak pada penempatan akhir setelah *release*, melainkan pada proses menuju fase *release* itu sendiri.

Kegagalan silinder dapat dibagi menjadi dua kelompok utama. Kelompok pertama adalah episode yang gagal mencapai *release*. Contohnya adalah episode 12, 13, 14, 16, 18, 20, 33, dan 40 yang ada pada Lampiran 1.3. Kelompok kedua adalah episode yang sudah *released*, tetapi tetap gagal karena objek belum cukup dekat dengan lubang target. Episode seperti 10, 21, 23, dan 39 yang juga dapat dilihat pada Lampiran 1.3 termasuk dalam kategori ini. Temuan ini menunjukkan bahwa pada silinder, tantangan utama terletak pada kemampuan *policy* membawa objek hingga mencapai kondisi *release* secara stabil.

Objek segitiga juga memperoleh 37 episode sukses dari 50 episode, sehingga *success rate*-nya sama dengan silinder, yaitu 74%. Berbeda dari silinder, segitiga relatif lebih sering berhasil mencapai fase *released*, yaitu sebanyak 46 episode. Akan tetapi, dari 46 episode tersebut, hanya 37 episode yang berhasil memenuhi batas toleransi penempatan. Dengan demikian, *success rate* setelah *release* pada segitiga adalah 80,43%, lebih rendah dibandingkan kubus dan silinder.

Hal ini menunjukkan bahwa pada segitiga, permasalahan utama bukan terletak pada pencapaian fase *release*, melainkan pada presisi posisi akhir setelah objek dilepas. Banyak episode segitiga yang gagal meskipun objek telah berhasil dibawa dan dilepaskan di sekitar target. Kegagalan seperti ini dapat dilihat pada episode 3, 5, 7, 12, 19, 28, 29, 40, dan 46 yang ada pada Lampiran 1.4. Selain itu, terdapat beberapa episode gagal pada fase *grasping*, seperti episode 10, 39, 47, dan 50 yang juga dapat dilihat pada Lampiran 1.4. Episode 47 merupakan *outlier* paling ekstrem, karena objek berakhir sangat jauh dari target. Dengan demikian, segitiga menunjukkan bahwa *target policy* sudah cukup mampu membawa objek hingga tahap *release*, tetapi masih memerlukan peningkatan pada akurasi penempatan akhir, terutama pada kestabilan orientasi dan posisi akhir objek setelah dilepas.

Karena seluruh episode pengujian antarobjek menggunakan kondisi awal yang sama, maka hasil pengujian ini memungkinkan dilakukan perbandingan yang adil antara ketiga objek. Dari sisi *success rate total*, kubus menempati posisi tertinggi dengan 84%, sedangkan silinder dan segitiga sama-sama berada pada 74%. Hasil ini menunjukkan bahwa kubus merupakan objek yang paling konsisten untuk ditangani oleh *target policy*.

Dari sisi pencapaian fase *release*, segitiga dan kubus menunjukkan hasil yang lebih baik dibanding silinder. Artinya, *target policy* relatif lebih mudah membawa kubus dan segitiga sampai ke tahap pelepasan. Sebaliknya, silinder lebih sering mengalami kegagalan sebelum *release*. Namun setelah berhasil dilepas, silinder justru menunjukkan keberhasilan yang tinggi. Hal ini mengindikasikan bahwa pada silinder, proses mendekati target dan melakukan *release* merupakan tahap yang paling krusial.

Sementara itu, segitiga menunjukkan pola yang berbeda. Segitiga sering berhasil mencapai *release*, tetapi keberhasilan total tetap rendah karena banyak episode yang gagal tipis akibat posisi akhir objek sedikit berada di luar batas toleransi. Dengan kata lain, segitiga lebih menuntut presisi akhir yang tinggi agar dapat dikategorikan berhasil. Berdasarkan hasil tersebut, dapat dikatakan bahwa perbedaan performa ketiga objek bukan hanya disebabkan oleh keberhasilan atau kegagalan secara umum, tetapi juga oleh jenis kegagalan yang berbeda. Oleh karena itu, analisis performa tidak cukup hanya melihat *success rate*, tetapi juga perlu mempertimbangkan fase akhir, serta jarak objek ke lubang target.

Selain berdasarkan *success rate*, fase *release*, dan jarak akhir objek terhadap lubang target, performa *target policy* pada lingkungan simulasi juga dianalisis berdasarkan jumlah step yang dibutuhkan hingga episode berakhir. Jumlah step digunakan sebagai indikator waktu penyelesaian pada simulasi, karena merepresentasikan banyaknya langkah kontrol yang dilakukan *target policy* selama satu episode. Semakin sedikit jumlah step pada episode sukses, maka semakin efisien *policy* dalam menyelesaikan tugas penempatan objek. Ringkasan jumlah step pada pengujian simulasi ditunjukkan pada Tabel 4.7.

Tabel 4.7 Ringkasan Jumlah *Step* pada Pengujian di Lingkungan Simulasi

Objek	**Rata-rata Step Semua Episode**	**Rata-rata Step Episode Sukses**	**Rata-rata Step Episode Gagal**
Kubus	210,96	209,74	217,38
Silinder	229,72	225,97	240,38
Prisma segitiga	226,88	224,68	233,15

Berdasarkan Tabel 4.7, objek kubus memiliki rata-rata *step* paling rendah, baik pada seluruh episode maupun pada episode sukses. Rata-rata *step* seluruh episode pada kubus adalah 210,96 *step*, sedangkan rata-rata *step* pada episode sukses adalah 209,74 step. Hal ini menunjukkan bahwa *target policy* tidak hanya lebih sering berhasil pada kubus, tetapi juga cenderung menyelesaikan tugas dengan jumlah langkah yang lebih sedikit. Dengan kata lain, kubus merupakan objek yang paling efisien untuk dimanipulasi oleh *policy* pada pengujian simulasi.

Objek silinder memiliki rata-rata *step* paling tinggi. Rata-rata *step* seluruh episode pada silinder mencapai 229,72 *step*, sedangkan rata-rata step pada episode gagal mencapai 240,38 *step*. Nilai tersebut mendekati batas maksimum episode, yaitu 250 *step*. Hal ini menunjukkan bahwa ketika *target policy* gagal pada objek silinder, episode cenderung berlangsung lama hingga mendekati batas maksimum *step*. Kondisi ini sejalan dengan hasil sebelumnya bahwa silinder lebih sering mengalami kegagalan sebelum mencapai fase release.

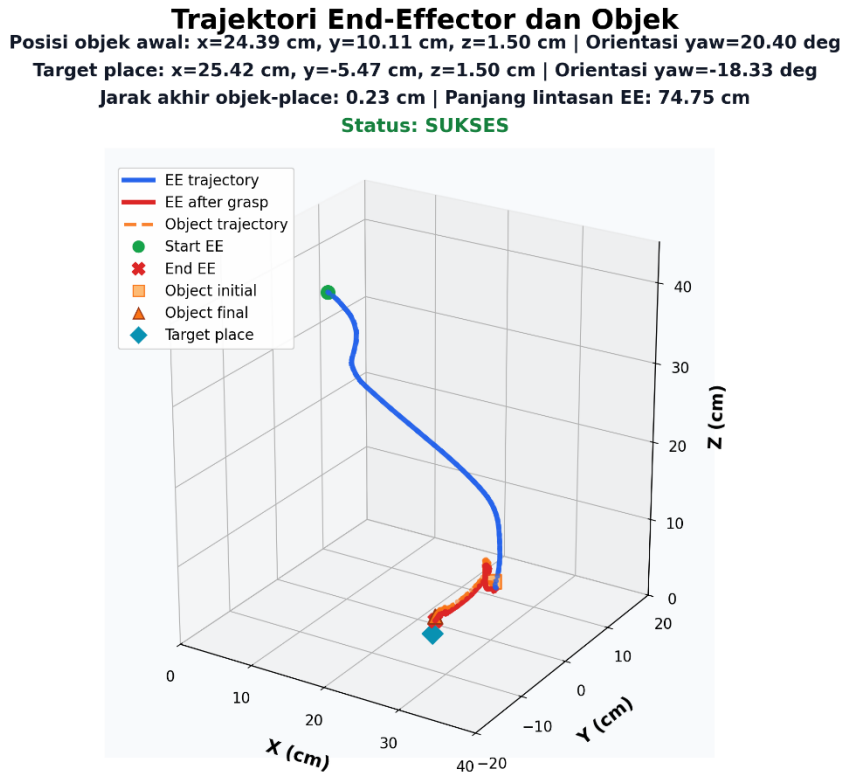
Objek prisma segitiga memiliki rata-rata *step* sebesar 226,88 step untuk seluruh episode dan 224,68 *step* pada episode sukses. Nilai ini lebih tinggi dibandingkan kubus, tetapi sedikit lebih rendah dibandingkan silinder. Hal ini menunjukkan bahwa meskipun prisma segitiga cukup sering mencapai fase *release*, *policy* masih membutuhkan jumlah step yang relatif besar untuk menyelesaikan proses penempatan. Kondisi ini sejalan dengan temuan sebelumnya bahwa kegagalan pada prisma segitiga banyak terjadi akibat posisi akhir objek yang sedikit berada di luar batas toleransi keberhasilan.

Dengan demikian, dari sisi waktu penyelesaian, kubus merupakan objek yang paling efisien, sedangkan silinder merupakan objek yang paling sulit diselesaikan secara cepat. Perbedaan ini menunjukkan bahwa karakteristik geometri objek tidak hanya memengaruhi tingkat keberhasilan, tetapi juga memengaruhi jumlah langkah yang dibutuhkan *policy* dalam menyelesaikan tugas.

Analisis trajektori juga dilakukan dengan memilih beberapa episode representatif yang mewakili tiga kondisi utama, yaitu episode berhasil, episode mendekati berhasil, dan episode gagal. Episode berhasil digunakan untuk menunjukkan pola gerakan ideal ketika *target policy* mampu menyelesaikan tugas.

Episode mendekati berhasil digunakan untuk menunjukkan kondisi ketika *policy* sebenarnya sudah mampu membawa objek ke sekitar lubang target, tetapi posisi akhirnya masih sedikit berada di luar batas keberhasilan. Sementara itu, episode gagal digunakan untuk menunjukkan kondisi ketika *policy* tidak berhasil mempertahankan kontrol terhadap objek atau tidak berhasil mencapai fase *release* dengan benar.

Gambar 4.27 menunjukkan contoh trajektori berhasil pada objek kubus. Pada episode ini, posisi awal objek berada pada koordinat x = 24,39 cm, y = 10,11 cm, z = 1,82 cm, sedangkan lubang target berada pada koordinat x = 24,00 cm, y = -5,00 cm, z = 0,00 cm. Berdasarkan hasil akhir episode, jarak akhir objek terhadap lubang target adalah 0,23 cm, sehingga episode ini memenuhi kriteria keberhasilan.

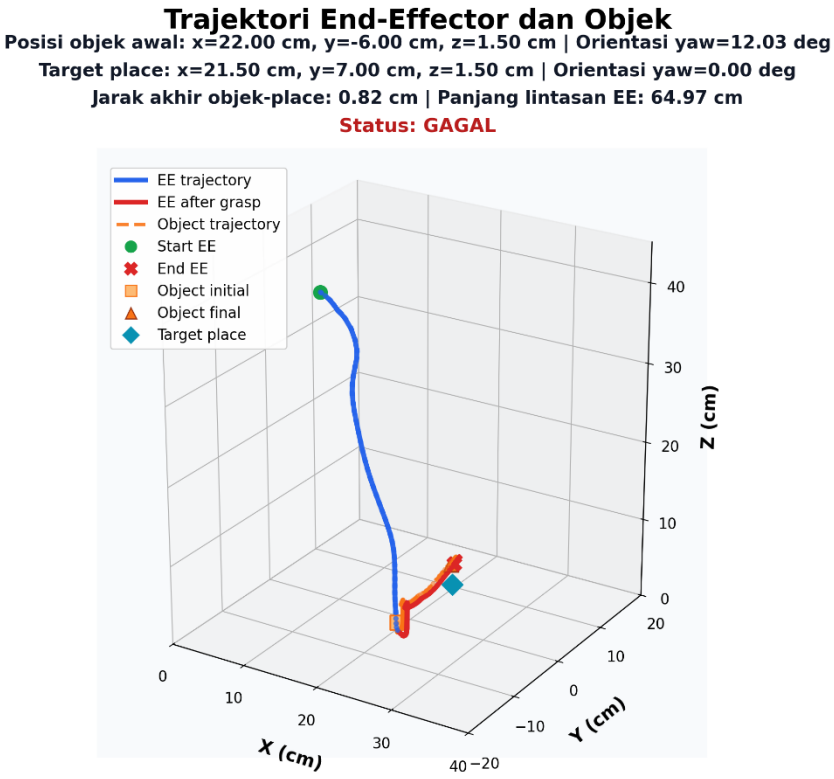


Gambar 4.27 Trajektori Episode Berhasil pada Objek Kubus

Dari trajektori yang ditampilkan, *end-effector* bergerak dari posisi awal menuju area objek, kemudian membawa objek menuju lubang target. Lintasan *end-effector* terlihat relatif terarah dan tidak menunjukkan penyimpangan besar. Setelah objek dipindahkan mendekati target, objek berhasil dilepaskan pada posisi yang sangat dekat dengan lubang target.

Lintasan objek juga menunjukkan pola yang stabil. Objek bergerak mengikuti arah perpindahan *end-effector* dan berakhir di sekitar posisi target. Jarak akhir sebesar 0,23 cm menunjukkan bahwa *target policy* mampu menghasilkan penempatan yang cukup presisi pada objek kubus. Hal ini sejalan dengan hasil sebelumnya, yaitu objek kubus memperoleh *success rate* tertinggi dibandingkan objek lainnya.

Gambar 4.28 menunjukkan contoh trajektori near success pada objek prisma segitiga. Pada episode ini, posisi awal objek berada pada koordinat x = 22,00 cm, y = -6,00 cm, z = 2,68 cm, sedangkan lubang target berada pada koordinat x = 20,00 cm, y = 7,00 cm, z = 0,00 cm. Jarak akhir objek terhadap lubang target adalah 0,82 cm, sehingga episode ini tercatat sebagai gagal pada hasil evaluasi.



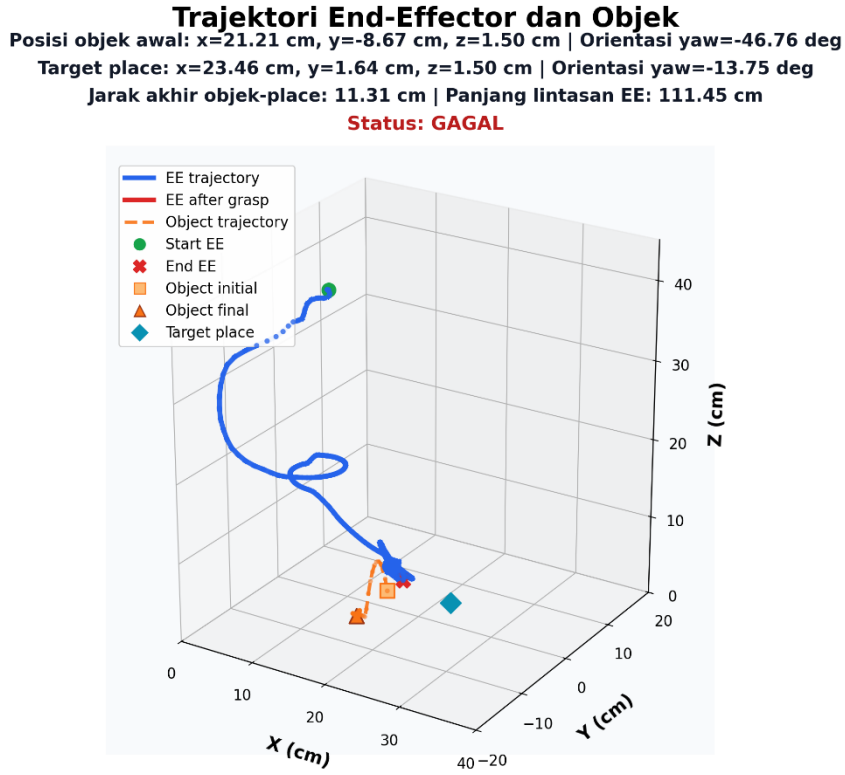
Gambar 4.28 Trajektori Hampir Berhasil pada Objek Prisma Segitiga.

Meskipun episode ini berstatus gagal, trajektori yang ditampilkan menunjukkan bahwa *policy* sebenarnya sudah mampu membawa objek ke area sekitar lubang target. *End-effector* bergerak menuju objek dan lintasan objek juga mengarah ke posisi target. Tidak terlihat penyimpangan besar seperti pada episode

gagal total. Oleh karena itu, episode ini lebih tepat dikategorikan sebagai hampir berhasil.

Kegagalan pada episode ini terjadi karena jarak akhir objek terhadap lubang target sebesar 0,82 cm atau 8,2 mm, sehingga masih berada 3,2 mm di atas ambang keberhasilan 5 mm. Episode ini penting untuk dianalisis karena menunjukkan bahwa tidak semua episode gagal merupakan kegagalan total. Pada kasus ini, *policy* sudah menghasilkan lintasan yang benar secara umum, tetapi masih membutuhkan peningkatan presisi pada tahap akhir, terutama agar posisi objek setelah dilepas benar-benar berada di dalam area lubang target.

Gambar 4.29 menunjukkan contoh trajektori gagal pada objek silinder. Pada episode ini, posisi awal objek berada pada koordinat x = 21,21 cm, y = -8,67 cm, z = 7,38 cm, sedangkan lubang target berada pada koordinat x = 22,00 cm, y = 2,00 cm, z = 0,00 cm. Jarak akhir objek terhadap lubang target adalah 11,31 cm, sehingga episode ini gagal memenuhi kriteria keberhasilan.



Gambar 4.29 Trajektori Episode Gagal pada Objek Silinder.

Berbeda dengan episode berhasil dan hampir berhasil, trajektori pada episode ini menunjukkan pergerakan yang kurang stabil. Lintasan *end-effector*

terlihat memiliki lengkungan dan perubahan arah yang cukup besar sebelum mendekati area objek. Selain itu, lintasan objek tidak berakhir di sekitar lubang target, melainkan berhenti pada posisi yang masih cukup jauh dari target.

Jarak akhir sebesar 11,31 cm menunjukkan bahwa kegagalan pada episode ini bukan hanya disebabkan oleh kesalahan kecil pada tahap akhir, tetapi merupakan kegagalan yang lebih signifikan. *Policy* tidak berhasil mempertahankan kontrol objek hingga posisi akhir yang sesuai. Hal ini sejalan dengan hasil kuantitatif pada objek silinder, di mana beberapa episode gagal terjadi karena *policy* tidak berhasil menyelesaikan transisi menuju fase *release* secara stabil. Dengan demikian, episode ini merepresentasikan kegagalan yang berbeda dari kategori hampir berhasil. Pada kategori hampir berhasil, lintasan sudah mendekati target tetapi belum cukup presisi, sedangkan pada episode gagal ini objek masih berakhir jauh dari lubang target.

Berdasarkan ketiga contoh trajektori tersebut, terlihat bahwa perbedaan antara episode berhasil, hampir berhasil, dan gagal tidak hanya terletak pada nilai jarak akhir objek terhadap lubang target, tetapi juga pada pola gerakan selama episode berlangsung. Episode berhasil menunjukkan lintasan yang relatif stabil dan berakhir sangat dekat dengan lubang target. Episode hampir berhasil menunjukkan lintasan yang sudah mengarah ke target, tetapi posisi akhir objek masih belum memenuhi ambang keberhasilan. Sementara itu, episode gagal menunjukkan penyimpangan lintasan yang lebih besar dan jarak akhir objek yang jauh dari target. Oleh karena itu, analisis trajektori memberikan informasi tambahan yang penting untuk memahami penyebab keberhasilan maupun kegagalan *target policy*.

Secara keseluruhan, hasil pengujian di lingkungan simulasi menunjukkan bahwa *target policy* mampu menyelesaikan tugas penempatan objek pada tiga bentuk objek uji dengan tingkat keberhasilan yang berbeda-beda. Kubus memperoleh performa paling stabil, sedangkan silinder dan prisma segitiga menunjukkan pola kegagalan yang berbeda. Pada silinder, kegagalan lebih banyak terjadi sebelum fase *release*, sedangkan pada prisma segitiga kegagalan lebih banyak terjadi setelah objek dilepaskan karena posisi akhir belum memenuhi batas toleransi. Analisis trajektori menunjukkan bahwa episode berhasil, hampir berhasil,

dan gagal memiliki pola gerakan yang berbeda, sehingga trajektori dapat digunakan untuk memahami penyebab keberhasilan maupun kegagalan *policy*.

4.4 Hasil Pengujian di Lingkungan Nyata

4.4.1 Hasil Kalibrasi Kamera

Sebelum pengujian pada lingkungan nyata dilakukan, kamera terlebih dahulu dikalibrasi untuk memperoleh parameter intrinsik kamera dan koefisien distorsi lensa. Parameter tersebut digunakan pada proses deteksi AprilTag agar estimasi pose objek dan lubang target dapat dihitung berdasarkan parameter kamera yang sesuai. Dengan demikian, hasil kalibrasi kamera menjadi bagian penting dari sistem persepsi pada pengujian lingkungan nyata.

Setelah citra diperoleh, titik sudut dalam pada pola *chessboard* dideteksi menggunakan OpenCV. Titik sudut yang berhasil terdeteksi kemudian diperhalus dan digunakan sebagai pasangan titik dua dimensi pada citra, sedangkan susunan titik pada pola *chessboard* digunakan sebagai titik referensi tiga dimensi pada bidang datar. Contoh hasil deteksi titik sudut pada citra kalibrasi ditunjukkan pada Gambar 4.30. Berdasarkan hasil kalibrasi, sebanyak 24 dari 30 citra berhasil mendeteksi pola *chessboard* dan digunakan dalam proses perhitungan parameter kamera. Hasil kalibrasi menghasilkan matriks intrinsik kamera yang dapat dilihat pada matriks (4.1) berikut.

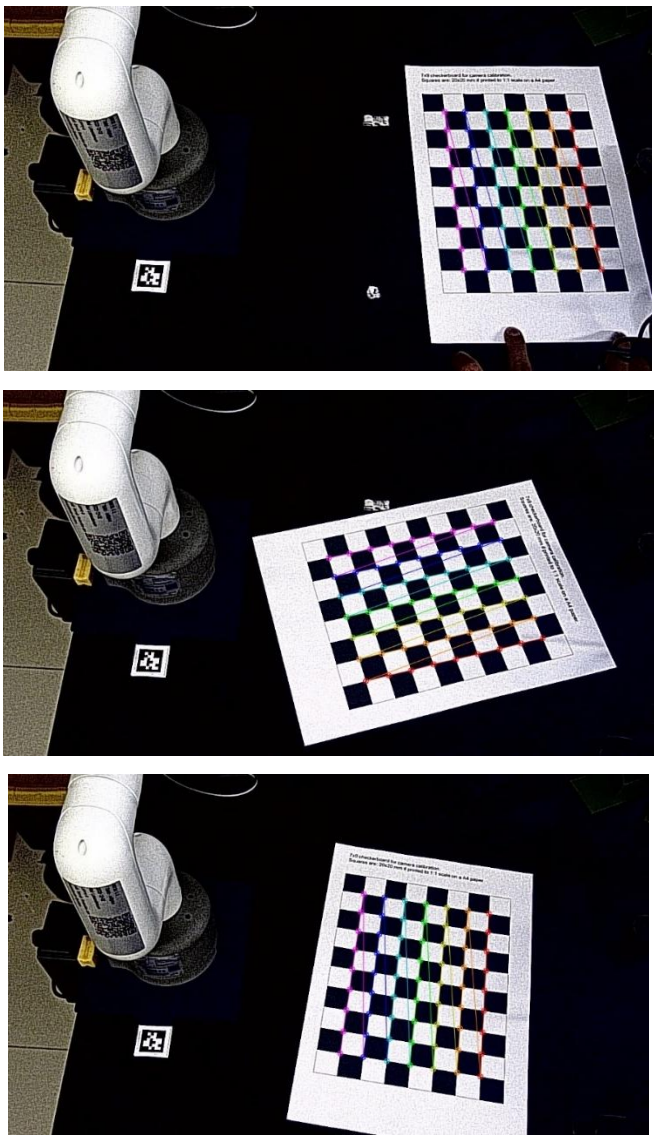
$$\boldsymbol{K} = \begin{bmatrix} 1844{,}17 & 0 & 703{,}70 \\ 0 & 2057{,}06 & 398{,}87 \\ 0 & 0 & 1 \end{bmatrix} \tag{4.1}$$

Berdasarkan matriks tersebut, nilai f_x dan f_y masing-masing sebesar 1844,17 piksel dan 2057,06 piksel. Nilai tersebut menunjukkan panjang fokus kamera pada sumbu horizontal dan vertikal dalam satuan piksel. Sementara itu, nilai $c_x$dan c_y masing-masing sebesar 703,70 piksel dan 398,87 piksel, yang menunjukkan posisi titik pusat optik pada bidang citra.

Sedangkan koefisien distorsi kamera yang diperoleh dari hasil kalibrasi dapat dilihat pada matriks (4.2) berikut.

$$\boldsymbol{D} = [0{,}273 \quad -1{,}115 \quad 0{,}002 \quad 0{,}001 \quad -10{,}374] \tag{4.2}$$

Koefisien distorsi tersebut terdiri atas koefisien distorsi radial dan tangensial. Parameter ini digunakan untuk mengoreksi penyimpangan citra akibat karakteristik lensa kamera sebelum citra digunakan pada proses deteksi AprilTag.



Gambar 4.30 Contoh hasil deteksi titik sudut *chessboard*

Kualitas hasil kalibrasi dievaluasi menggunakan nilai *reprojection error* seperti yang telah dijelaskan pada Subbab 2.7. Berdasarkan hasil perhitungan, diperoleh nilai *root mean square reprojection error* sebesar 0,3328 piksel. Nilai ini menunjukkan bahwa rata-rata selisih antara titik sudut *chessboard* yang terdeteksi pada citra dan titik hasil proyeksi ulang berdasarkan parameter kamera berada di bawah satu piksel. Dengan demikian, hasil kalibrasi kamera dapat digunakan

sebagai parameter pendukung pada proses deteksi AprilTag dalam pengujian lingkungan nyata.

Meskipun demikian, nilai *reprojection error* tidak secara langsung merepresentasikan akurasi akhir posisi objek pada robot nyata. Akurasi posisi objek dan lubang target tetap dipengaruhi oleh proses deteksi AprilTag, kondisi pencahayaan, posisi kamera, transformasi koordinat, serta eksekusi gerakan robot. Oleh karena itu, hasil kalibrasi kamera pada bagian ini digunakan sebagai dasar sistem persepsi, sedangkan performa akhir tetap dianalisis melalui hasil pengujian robot pada lingkungan nyata.

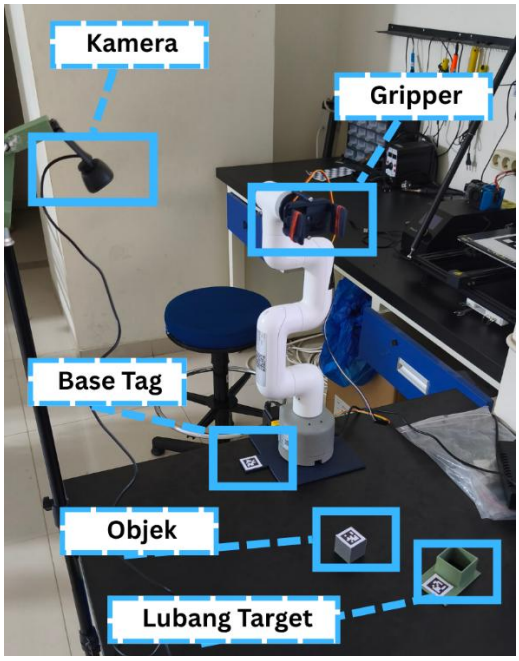
4.4.2 Hasil Pengujian Robot

Setelah parameter kamera diperoleh, parameter tersebut digunakan pada proses deteksi AprilTag untuk memperkirakan pose awal objek dan lubang target pada lingkungan nyata. Pose tersebut kemudian ditransformasikan ke sistem koordinat acuan robot dan digunakan sebagai kondisi awal pada simulasi MuJoCo sebelum *trajectory replay* dijalankan pada robot fisik. Dengan demikian, hasil kalibrasi kamera pada Subbab 4.4.1 menjadi dasar bagi proses pengujian robot yang dibahas pada Subbab 4.4.2.

Pengujian robot di lingkungan nyata dilakukan untuk mengevaluasi sejauh mana *target policy* yang telah dilatih di lingkungan simulasi dapat diterapkan pada robot fisik. Pengujian ini penting karena keberhasilan pada simulasi belum tentu menghasilkan performa yang sama pada kondisi nyata. Pada lingkungan fisik, terdapat berbagai faktor yang tidak sepenuhnya terwakili di simulasi, seperti *error* persepsi, perbedaan koordinat, ketidakpastian kontak antara gripper dan objek, keterbatasan presisi aktuator, serta kemungkinan objek bergeser selama proses manipulasi. Oleh karena itu, pengujian lingkungan nyata digunakan untuk melihat kemampuan *transfer policy* dari simulasi ke robot fisik serta mengidentifikasi faktor-faktor yang menyebabkan perubahan performa.

Gambar 4.31 menunjukkan konfigurasi lingkungan nyata yang digunakan pada proses pengujian. Pada konfigurasi ini, robot myCobot digunakan sebagai manipulator utama, *gripper* digunakan untuk menggenggam objek, sedangkan kamera digunakan untuk mendeteksi AprilTag pada objek, lubang target, dan *base*

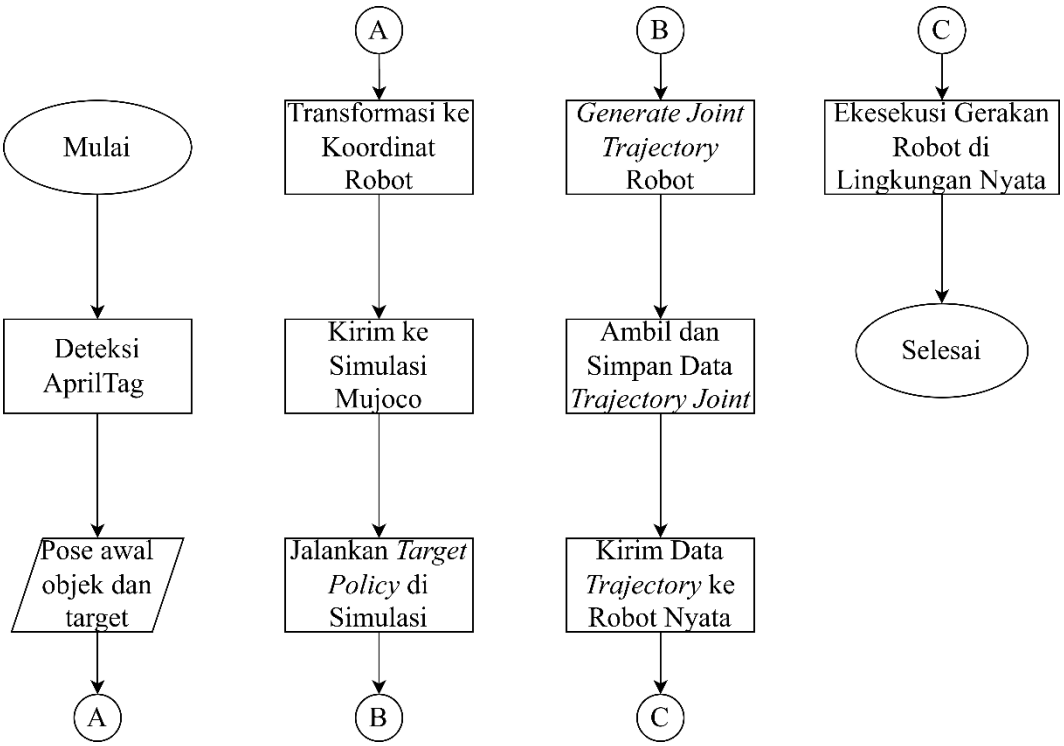
tag. Objek uji diletakkan pada area kerja robot, sementara lubang target ditempatkan pada posisi tertentu sesuai skenario pengujian. Konfigurasi ini digunakan untuk memperoleh pose awal objek dan lubang target sebelum data tersebut dikirim ke lingkungan simulasi MuJoCo untuk menghasilkan *trajectory joint*.



Gambar 4.31 Konfigurasi Lingkungan Pengujian Nyata

Pada pengujian lingkungan nyata, posisi awal objek dan lubang target diperoleh menggunakan kamera dan AprilTag. Pose hasil deteksi kemudian ditransformasikan ke sistem koordinat robot dengan mempertimbangkan *offset* antara *base tag* dan *base* robot, serta *offset* antara *tag* objek atau target terhadap pusat objek dan lubang target. Setelah pose objek dan target diperoleh, kondisi tersebut dikirim ke lingkungan simulasi MuJoCo. *Target policy* kemudian dijalankan di simulasi untuk menghasilkan *trajectory joint*. *Trajectory joint* yang dihasilkan disimpan dan dikirim ke robot myCobot untuk dieksekusi pada lingkungan nyata. Dengan demikian, pengujian nyata pada penelitian ini dilakukan menggunakan pendekatan *trajectory replay* dari simulasi ke robot fisik. Diagram pada Gambar 4.32 berikut menjelaskan alur pengujian *target policy* di lingkungan nyata.

Pendekatan *trajectory replay* digunakan karena sistem persepsi pada lingkungan nyata belum dapat memantau pose objek secara *real-time* selama seluruh proses eksekusi. Ketika objek sudah berada dalam genggaman *gripper*, AprilTag yang ditempel pada objek dapat tertutup oleh *gripper* atau tidak lagi terlihat oleh kamera. Kondisi tersebut menyebabkan sistem tidak memperoleh informasi posisi dan orientasi objek secara langsung setelah fase *grasp*. Oleh karena itu, *target policy* dijalankan terlebih dahulu di simulasi untuk menghasilkan *trajectory joint*, kemudian *trajectory* tersebut dieksekusi pada robot nyata.



Gambar 4.32 Desain Alur Pengujian *Target Policy* di Lingkungan Nyata

Pada proses eksekusi *trajectory replay* di robot nyata, *trajectory joint* yang dihasilkan dari simulasi tidak dieksekusi pada seluruh *timestep* secara langsung, melainkan dilakukan *subsampling* menggunakan *replay stride*. Pada penelitian ini, *replay stride* yang digunakan adalah 7, yang berarti robot mengeksekusi satu titik *trajectory* untuk setiap tujuh titik *trajectory* yang dihasilkan dari simulasi. Penggunaan *replay stride* dilakukan karena proses pengiriman dan eksekusi satu langkah perintah pada robot nyata membutuhkan waktu yang relatif besar, bergantung pada kecepatan komunikasi antara komputer dan robot. Berdasarkan

pengamatan selama pengujian, satu langkah eksekusi dapat memerlukan waktu sekitar 2 hingga 3 detik, sehingga apabila seluruh *timestep* dari simulasi dijalankan secara langsung, durasi eksekusi menjadi terlalu lama dan kurang efisien.

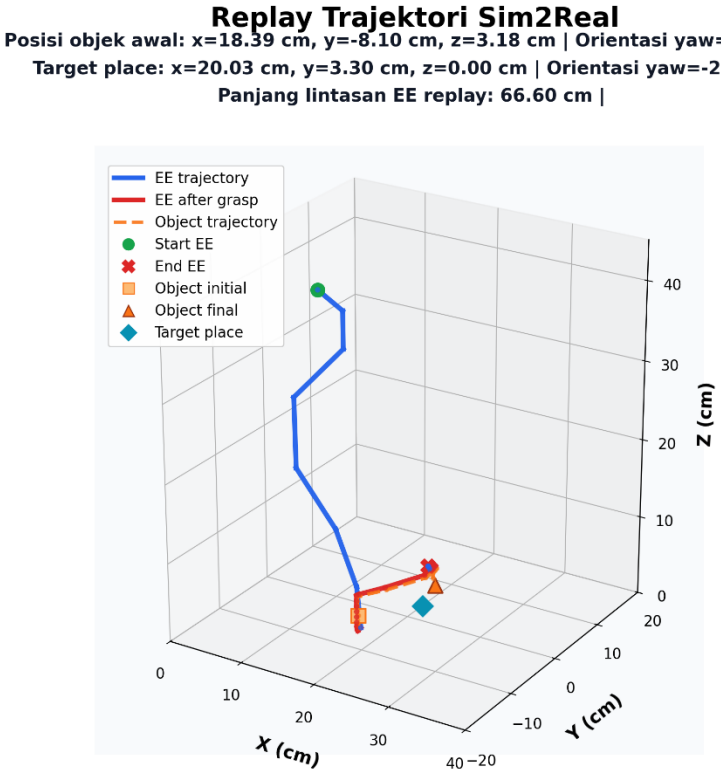
Replay stride digunakan untuk mengurangi jumlah titik *trajectory* yang harus dikirimkan ke robot tanpa mengubah urutan gerakan utama yang telah dihasilkan oleh *target policy*. Dengan cara ini, robot tetap mengikuti pola lintasan dari simulasi, tetapi jumlah perintah yang dieksekusi pada robot fisik menjadi lebih sedikit. Meskipun demikian, penggunaan *replay stride* juga memiliki konsekuensi terhadap kehalusan gerakan. Semakin besar nilai *stride*, semakin sedikit titik *trajectory* yang dieksekusi, sehingga gerakan robot dapat menjadi lebih kasar dan berpotensi menimbulkan penyimpangan terhadap lintasan asli. Oleh karena itu, *replay stride* 7 dipilih agar waktu eksekusi tetap efisien tanpa menghilangkan pola utama lintasan.

Gambar 4.33 menunjukkan contoh *trajectory replay* yang digunakan pada pengujian *sim-to-real*. Berdasarkan contoh tersebut, *trajectory replay* mampu menggambarkan bagaimana lintasan dari simulasi digunakan sebagai dasar eksekusi pada robot nyata. Namun, karena eksekusi dilakukan secara *open-loop*, robot tidak melakukan koreksi apabila terjadi perbedaan antara kondisi simulasi dan kondisi fisik. Apabila objek bergeser selama proses atau terdapat *error* pada estimasi posisi awal, *trajectory* tetap dijalankan sesuai hasil simulasi

Pada sistem nyata, *base tag* digunakan sebagai acuan koordinat *world*. *Base tag* tidak diletakkan tepat pada pusat *base* robot, melainkan memiliki offset sebesar 10 cm terhadap *base* myCobot. Selain itu, AprilTag pada lubang target juga memiliki offset sebesar 3 cm terhadap pusat lubang target. Oleh karena itu, pose yang diperoleh dari AprilTag tidak langsung digunakan sebagai pose objek dan target, tetapi dikoreksi terlebih dahulu berdasarkan transformasi offset yang telah ditentukan. Gambar 4.34 menunjukkan deteksi *base* robot, objek, serta target menggunakan AprilTag.

Sistem deteksi AprilTag juga dievaluasi untuk melihat tingkat kesalahan posisi yang muncul pada lingkungan nyata. Evaluasi ini penting karena pose hasil deteksi AprilTag digunakan sebagai kondisi awal pada simulasi. Apabila terdapat

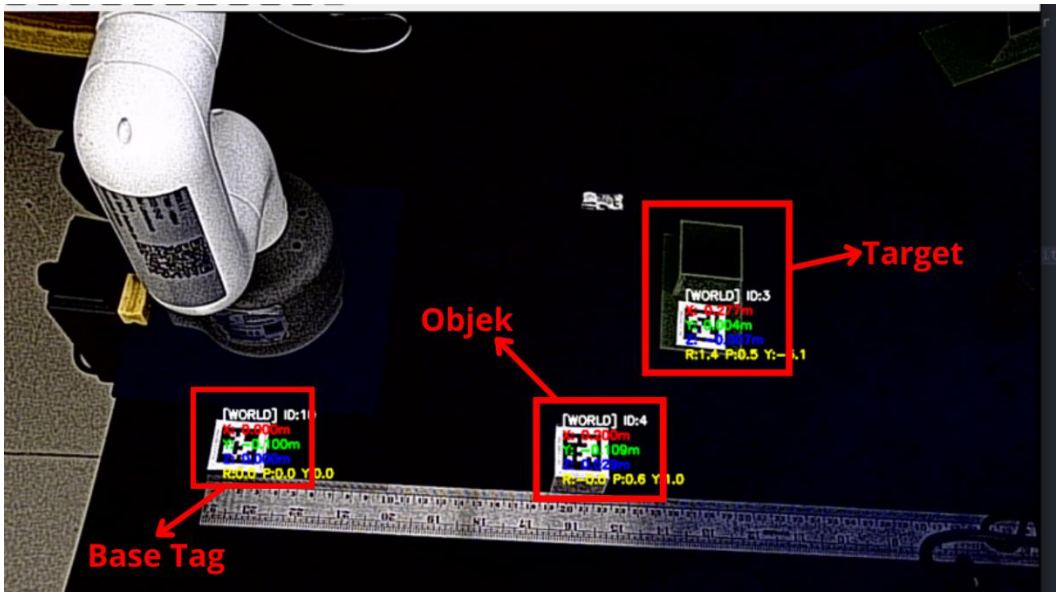
error pada posisi objek atau target, maka kondisi awal yang dikirim ke simulasi tidak sepenuhnya sama dengan kondisi nyata. Akibatnya, *trajectory joint* yang dihasilkan di simulasi dapat mengalami ketidaksesuaian ketika dieksekusi pada robot fisik.



Gambar 4.33 Contoh *Trajectory Replay*

Error deteksi posisi dihitung berdasarkan selisih antara posisi objek yang diperoleh dari hasil deteksi AprilTag dan posisi objek yang diukur secara manual pada lingkungan nyata. Pengukuran manual dilakukan sebagai acuan posisi aktual objek. Secara umum, *error* posisi dapat dinyatakan sebagai selisih jarak antara posisi hasil deteksi dan posisi aktual objek.

Berdasarkan hasil pengamatan, *error* deteksi posisi objek berada pada kisaran 0,5 cm hingga 1,5 cm. Nilai *error* ini cukup berpengaruh terhadap hasil pengujian karena toleransi keberhasilan penyisipan objek hanya 5 mm. Dengan demikian, kesalahan deteksi posisi sebesar beberapa milimeter hingga lebih dari satu sentimeter dapat menyebabkan trajektori yang dihasilkan simulasi tidak sepenuhnya sesuai dengan posisi objek pada lingkungan nyata.



Gambar 4.34 Deteksi Menggunakan AprilTag

Pengujian lingkungan nyata dilakukan pada tiga jenis objek, yaitu kubus, silinder, dan prisma segitiga. Setiap objek diuji sebanyak 10 episode. Pada setiap episode, posisi awal objek dan lubang target dideteksi menggunakan AprilTag, kemudian dikirim ke simulasi untuk menghasilkan *trajectory joint*. Pengujian dilakukan untuk melihat apakah trajektori yang berhasil di simulasi dapat dieksekusi dengan baik pada robot nyata. Tabel 4.8 menunjukkan hasil pengujian *policy* pada lingkungan nyata.

Tabel 4.8 Hasil Pengujian *Target Policy* di Lingkungan Nyata

Objek	**Jumlah Episode**	**Berhasil Grasp**	**Episode Sukses**	***Success Rate***
Kubus	10	7	0	0%
Silinder	10	5	1	10%
Prisma Segitiga	10	3	0	0%

Berdasarkan Tabel 4.8, kubus menunjukkan kemampuan *grasp* paling baik dengan 7 dari 10 episode berhasil digenggam. Namun, tidak ada episode yang berhasil mencapai kondisi akhir karena objek selalu meleset setelah proses *release* atau tidak berhasil dibawa secara presisi ke target. Hasil ini menunjukkan bahwa meskipun bentuk kubus relatif lebih mudah digenggam dibandingkan objek

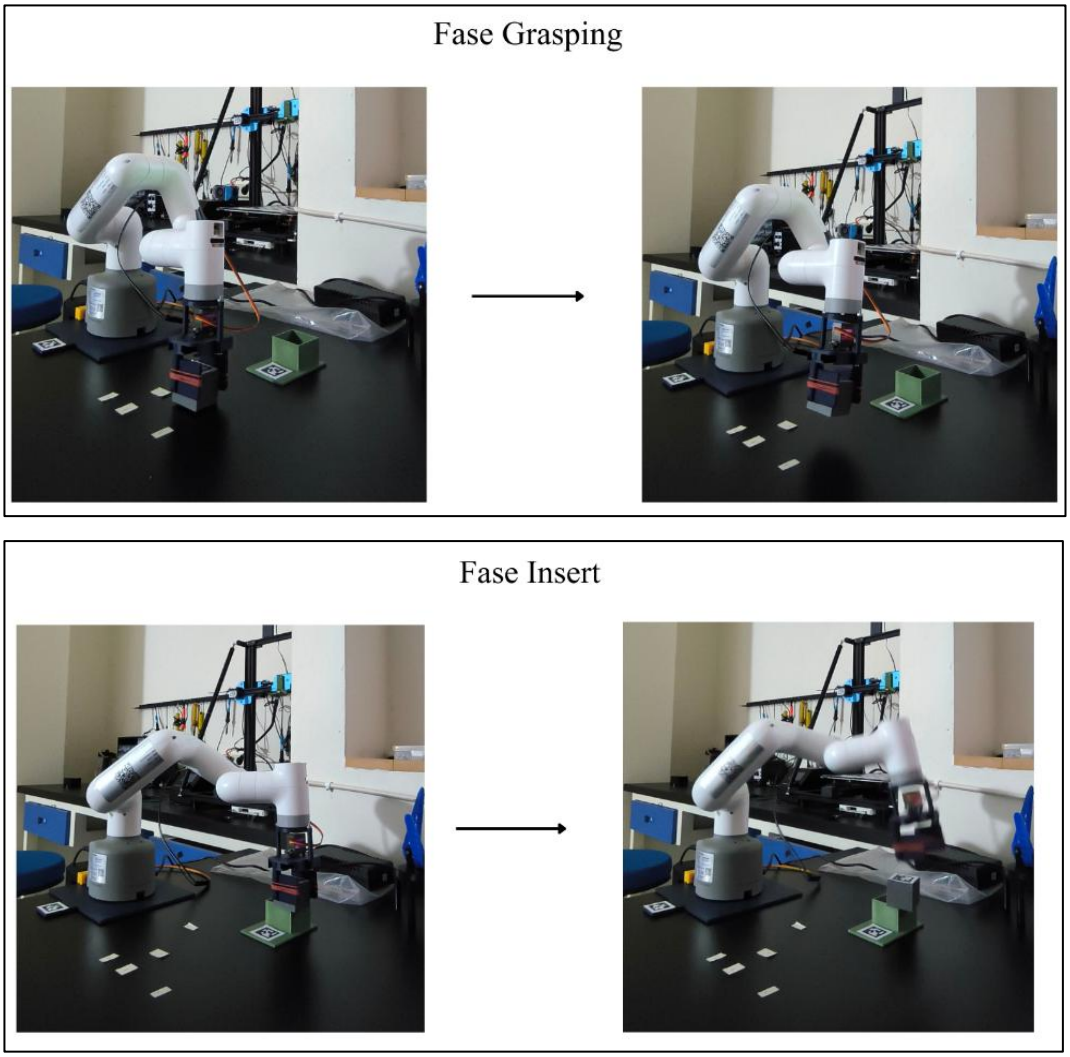
lainnya, keberhasilan *grasp* belum cukup untuk menjamin keberhasilan penempatan akhir pada lingkungan nyata.

Gambar 4.35 menunjukkan salah satu contoh kegagalan pada pengujian objek kubus di lingkungan nyata. Pada kondisi tersebut, objek kubus telah berhasil digenggam oleh *gripper* dan dibawa menuju area lubang target. Namun, objek tidak berhasil masuk ke dalam lubang. Hal ini menunjukkan bahwa kegagalan pada pengujian nyata tidak selalu terjadi pada fase *grasping*, tetapi juga dapat terjadi pada fase penempatan akhir.

Silinder menunjukkan tingkat keberhasilan *grasp* yang lebih rendah dibandingkan kubus, yaitu 5 dari 10 episode. Sebagian kegagalan terjadi pada fase *grasp*, yang menunjukkan bahwa bentuk silinder lebih sulit distabilkan oleh *gripper*. Namun, silinder menjadi satu-satunya objek yang berhasil mencapai target pada satu episode. Hal ini menunjukkan bahwa ketika *grasp* berhasil dilakukan dengan cukup stabil, trajektori masih memiliki kemungkinan untuk membawa objek hingga mencapai target.

Prisma segitiga menunjukkan performa terendah, dengan 3 dari 10 episode berhasil digenggam dan tidak ada episode yang berhasil mencapai target. Objek ini sulit digenggam dan juga sulit diposisikan menuju target. Kegagalan terjadi baik pada fase *grasp* maupun setelah objek berhasil dibawa. Kondisi ini menunjukkan bahwa bentuk prisma segitiga memberikan tantangan paling besar pada pengujian lingkungan nyata, terutama karena orientasi dan kestabilan objek lebih sulit dipertahankan selama proses pergerakan.

Berdasarkan Tabel 4.8, hasil pengujian lingkungan nyata perlu diinterpretasikan sebagai evaluasi awal *sim-to-real*. Jumlah pengujian sebanyak 10 episode per objek belum cukup untuk menghasilkan estimasi statistik yang kuat terhadap performa sistem pada robot fisik. Oleh karena itu, *success rate* sebesar 10% pada objek silinder tidak dapat langsung dianggap sebagai performa representatif sistem secara umum. Nilai tersebut hanya menunjukkan bahwa dari skenario pengujian terbatas yang dilakukan, sistem masih memiliki kemungkinan keberhasilan, tetapi tingkat keberhasilannya masih sangat rendah.



Gambar 4.35 Contoh Kegagalan Penyisipan Objek Kubus pada Pengujian

Secara keseluruhan, hanya 1 dari 30 episode pengujian nyata yang berhasil mencapai kondisi sukses. Dengan demikian, *success rate* total pada lingkungan nyata adalah 3,33%. Hasil ini menunjukkan bahwa *target policy* yang berhasil belajar pada lingkungan simulasi belum berhasil ditransfer secara andal ke robot fisik. Oleh karena itu, pengujian nyata pada penelitian ini tidak digunakan untuk menyatakan bahwa sistem telah layak diterapkan secara praktis, melainkan untuk mengidentifikasi keterbatasan utama pada proses transfer dari simulasi ke lingkungan nyata.

4.5 Analisis Performa Keseluruhan

Analisis performa keseluruhan dilakukan dengan mengacu pada metrik evaluasi yang telah dijelaskan pada Bab 2, yaitu tingkat keberhasilan, waktu

penyelesaian, efisiensi sampel, dan analisis trajektori *end-effector*. Keempat metrik tersebut digunakan untuk menilai performa *target policy* dari beberapa aspek, yaitu efektivitas dalam menyelesaikan tugas, efisiensi jumlah langkah yang dibutuhkan, efisiensi proses pembelajaran, serta kestabilan pola gerakan yang dihasilkan. Dengan demikian, analisis ini tidak hanya melihat apakah *policy* berhasil atau gagal, tetapi juga menjelaskan bagaimana proses keberhasilan dan kegagalan tersebut terjadi.

Berdasarkan tingkat keberhasilan, *target policy* mampu menyelesaikan sebagian besar episode pengujian pada lingkungan simulasi. Objek kubus memperoleh *success rate* tertinggi, yaitu 84%, sedangkan silinder dan prisma segitiga masing-masing memperoleh *success rate* sebesar 74%. Hasil ini menunjukkan bahwa *target policy* telah mampu menyelesaikan tugas penempatan objek pada lingkungan simulasi. Perbedaan *success rate* antar objek menunjukkan bahwa karakteristik geometri objek memengaruhi performa *policy*. Kubus menjadi objek dengan performa paling stabil karena memiliki bentuk dan permukaan yang relatif mudah dikendalikan selama proses *grasping* dan pemindahan menuju target. Silinder dan prisma segitiga memiliki *success rate* yang lebih rendah, tetapi menunjukkan pola kegagalan yang berbeda. Pada silinder, kegagalan lebih banyak terjadi sebelum fase *release*, sedangkan pada prisma segitiga, kegagalan lebih banyak terjadi setelah objek dilepaskan karena posisi akhir belum memenuhi batas toleransi keberhasilan.

Pada lingkungan nyata, performa *target policy* mengalami penurunan yang signifikan dibandingkan simulasi. Dari 10 episode pengujian pada setiap objek, kubus berhasil digenggam sebanyak 7 kali tetapi tidak menghasilkan episode sukses, silinder berhasil digenggam sebanyak 5 kali dan menghasilkan 1 episode sukses, sedangkan prisma segitiga berhasil digenggam sebanyak 3 kali tanpa episode sukses. Hasil ini menunjukkan bahwa keberhasilan *policy* di simulasi belum berhasil ditransfer ke robot fisik. Penurunan performa tersebut dipengaruhi oleh *error* deteksi AprilTag, *error* transformasi koordinat, perbedaan kondisi fisik antara simulasi dan robot nyata, serta pendekatan *trajectory replay* yang belum melakukan koreksi secara *real-time* selama eksekusi.

Dari sisi waktu penyelesaian, jumlah *step* digunakan sebagai indikator waktu penyelesaian pada lingkungan simulasi karena merepresentasikan banyaknya langkah kontrol atau evaluasi yang dilakukan *target policy* dalam satu episode. Berdasarkan hasil pengujian simulasi, kubus memiliki rata-rata *step* paling rendah, yaitu 210 step untuk seluruh episode dan 209 step pada episode sukses. Hal ini menunjukkan bahwa *target policy* tidak hanya lebih sering berhasil pada objek kubus, tetapi juga cenderung menyelesaikan tugas dengan jumlah langkah yang lebih sedikit. Silinder memiliki rata-rata *step* paling tinggi, yaitu 229 step untuk seluruh episode dan 240 step pada episode gagal. Nilai tersebut mendekati batas maksimum episode, yaitu 250 *step*, sehingga menunjukkan bahwa ketika *target policy* gagal pada objek silinder, episode cenderung berlangsung lama hingga mendekati batas maksimum *step*. Prisma segitiga memiliki rata-rata *step* sebesar 226 *step* untuk seluruh episode dan 224 *step* pada episode sukses. Dengan demikian, karakteristik geometri objek tidak hanya memengaruhi tingkat keberhasilan, tetapi juga memengaruhi jumlah langkah yang dibutuhkan *policy* dalam menyelesaikan tugas.

Efisiensi sampel dianalisis berdasarkan kemampuan metode pembelajaran dalam meningkatkan performa dengan jumlah *timestep* pelatihan yang lebih sedikit. Pada penelitian ini, efisiensi sampel diamati melalui perbandingan antara SAC berbasis QMP-HER dan SAC standar serta SAC+HER. SAC standar merujuk pada pelatihan SAC langsung pada tugas utama tanpa mekanisme QMP-HER, sedangkan SAC+HER merujuk pada pelatihan SAC dengan bantuan HER tanpa ada mekanisme *Q-Switch*. Berdasarkan hasil pelatihan, SAC standar dan SAC+HER masih belum mampu menghasilkan *reward* optimal meskipun telah dilatih hingga 6.000.000 *timestep*.

Sebaliknya, SAC berbasis QMP-HER menunjukkan peningkatan performa yang lebih cepat. *Target policy* mulai menunjukkan peningkatan *success rate* setelah sekitar 500.000 *timestep*. Pada kisaran 800.000 hingga 1.200.000 *timestep*, *success rate* meningkat ke rentang sekitar 0,4 hingga 0,6. Pada fase akhir pelatihan, yaitu sekitar 2.000.000 *timestep*, *success rate* mencapai kisaran 0,8 hingga 0,85. Hasil ini menunjukkan bahwa penggunaan *primitive policy* melalui mekanisme *Q-*

Switch dan pemanfaatan pengalaman gagal melalui HER dapat membantu *target policy* memperoleh pengalaman pembelajaran yang lebih informatif. Dengan demikian, SAC berbasis QMP-HER memiliki efisiensi sampel yang lebih baik dibandingkan SAC standar dan SAC+HER, karena mampu menunjukkan peningkatan performa dalam jumlah *timestep* yang lebih sedikit.

Analisis trajektori digunakan untuk memahami pola gerakan yang dihasilkan *policy* selama proses pelatihan dan pengujian. Pada pelatihan *grasp policy*, trajektori *end-effector* menunjukkan perkembangan dari gerakan yang belum terarah pada fase awal menjadi lebih stabil dan mengarah ke objek pada fase akhir pelatihan. Hal ini memperkuat hasil *reward* posisi dan orientasi yang menunjukkan peningkatan kemampuan *grasp policy* dalam mendekati objek dan menyesuaikan orientasi *end-effector*. Pada pelatihan *insert policy*, trajektori juga menunjukkan peningkatan kualitas gerakan. Pada fase awal, objek masih dapat terlepas dari *gripper* dan lintasan *end-effector* belum stabil. Pada fase menengah, objek mulai mengikuti gerakan *end-effector*, tetapi belum selalu mencapai target secara presisi. Pada fase akhir, trajektori menjadi lebih stabil dan objek dapat dibawa mendekati target dengan lebih konsisten.

Pada pengujian simulasi, analisis trajektori menunjukkan perbedaan yang jelas antara episode berhasil, hampir berhasil, dan gagal. Episode berhasil menunjukkan lintasan yang relatif stabil dan berakhir sangat dekat dengan lubang target. Episode hampir berhasil menunjukkan lintasan yang sudah mengarah ke target, tetapi posisi akhir objek masih sedikit berada di luar batas toleransi. Sementara itu, episode gagal menunjukkan penyimpangan lintasan yang lebih besar atau kegagalan dalam mempertahankan kontrol objek hingga fase *release*. Dengan demikian, analisis trajektori memberikan informasi tambahan yang tidak dapat diperoleh hanya dari *success rate*, karena dapat menunjukkan pola gerakan dan penyebab kegagalan pada setiap episode.

Pada lingkungan nyata, kegagalan sistem lebih banyak dipengaruhi oleh faktor *sim-to-real*. *Error* deteksi posisi objek dan target menggunakan AprilTag berada pada kisaran 0,5 cm hingga 1,5 cm, sehingga kondisi awal yang dikirim ke simulasi dapat berbeda dari kondisi fisik sebenarnya. Selain itu, *trajectory replay*

dari simulasi ke robot fisik belum memiliki koreksi *real-time* apabila terjadi pergeseran objek, *error grasping*, atau ketidaksesuaian eksekusi *joint*. Oleh karena itu, kegagalan pada lingkungan nyata tidak hanya disebabkan oleh *target policy*, tetapi juga oleh akurasi persepsi, transformasi koordinat, kestabilan *grasp*, dan keterbatasan kontrol *open-loop*.

Secara keseluruhan, hasil analisis performa menunjukkan bahwa SAC berbasis QMP-HER mampu menghasilkan *policy* yang belajar secara progresif dan cukup efektif pada lingkungan simulasi. *Target policy* mampu menyelesaikan sebagian besar episode pengujian simulasi, memiliki waktu penyelesaian yang lebih efisien pada objek kubus, serta menunjukkan efisiensi sampel yang lebih baik dibandingkan SAC standar dan SAC+HER. Namun, performa pada lingkungan nyata masih mengalami penurunan karena adanya *sim-to-real gap*, *error* persepsi, dan keterbatasan *trajectory replay*. Dengan demikian, metode yang dikembangkan telah menunjukkan potensi pada simulasi, tetapi masih memerlukan peningkatan pada proses transfer ke lingkungan nyata yang lebih baik.

4.6 Pembahasan

Berdasarkan seluruh hasil pelatihan dan pengujian yang telah dijelaskan pada subbab sebelumnya, algoritma *Soft Actor-Critic* berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* mampu menunjukkan performa yang cukup baik pada lingkungan simulasi, tetapi belum mampu mempertahankan performa tersebut ketika diterapkan pada lingkungan nyata. Pada pengujian simulasi, *target policy* memperoleh tingkat keberhasilan sebesar 84% pada objek kubus, 74% pada objek silinder, dan 74% pada objek prisma segitiga. Hasil tersebut menunjukkan bahwa *target policy* telah mampu mempelajari urutan perilaku dasar dalam tugas penyisipan objek, mulai dari mendekati objek, membentuk kondisi *grasping*, membawa objek menuju area target, hingga melakukan *release*. Namun, hasil tersebut juga menunjukkan bahwa performa *policy* belum sepenuhnya stabil untuk semua bentuk objek, terutama ketika bentuk geometri objek menyebabkan proses *grasping*, perpindahan, dan release menjadi lebih sulit.

Peningkatan performa pada *target policy* menunjukkan bahwa penggunaan QMP-HER memberikan pengaruh positif terhadap proses pembelajaran. Hal ini

terlihat dari kemampuan *target policy* dalam memanfaatkan pengalaman yang lebih bermakna melalui bantuan *grasp policy* dan *insert policy*. Kedua *primitive policy* tersebut berperan sebagai sumber kandidat aksi selama proses pelatihan, sehingga eksplorasi *target policy* tidak sepenuhnya bergantung pada aksi acak. Mekanisme *Q-switch* kemudian memilih kandidat aksi berdasarkan nilai Q yang diprediksi oleh *critic* dari *target policy*. Dengan demikian, *target policy* dapat memperoleh arahan eksplorasi dari *primitive policy* tanpa menjadikan *primitive policy* sebagai pengendali akhir. Setelah proses pelatihan selesai, *target policy* tetap menjadi *policy* tunggal yang digunakan pada tahap pengujian.

Hasil tersebut sejalan dengan penelitian Sanchez dkk. (2023) yang memperkenalkan *Q-Switch Mixture of Primitives Hindsight Experience Replay* untuk meningkatkan efisiensi pembelajaran pada HER. Pada penelitian tersebut, *primitive policy* digunakan untuk memberikan kandidat aksi selama eksplorasi, sedangkan *critic* dari *target policy* digunakan untuk menentukan aksi yang paling bernilai pada setiap *timestep*. Sanchez dkk. menunjukkan bahwa pendekatan tersebut mampu mempercepat pembelajaran dibandingkan HER biasa, terutama pada tugas yang lebih kompleks. Pada tugas FetchPush, QMP-HER mampu mencapai *success rate* 90% dengan jumlah *policy update* yang lebih sedikit dibandingkan HER. Pada tugas FetchPickObstacle, HER biasa tidak mampu menyelesaikan tugas, sedangkan QMP-HER mampu mencapai *success rate* sekitar 95%. Dalam penelitian ini, prinsip yang sama juga terlihat, karena *target policy* lebih terbantu ketika memperoleh kandidat aksi dari *grasp policy* dan *insert policy* dibandingkan ketika hanya mengandalkan eksplorasi SAC standar atau SAC+HER. Dengan kata lain, hasil penelitian ini mendukung gagasan bahwa *primitive policy* dapat membantu memperkaya eksplorasi pada tugas manipulasi yang memiliki *sparse reward*.

Meskipun demikian, tingkat keberhasilan simulasi pada penelitian ini masih lebih rendah dibandingkan hasil yang dilaporkan oleh Sanchez dkk. (2023). Perbedaan tersebut dapat disebabkan oleh beberapa faktor. Pertama, penelitian ini tidak hanya menuntut agen untuk menggerakkan *end-effector* menuju target, tetapi juga menuntut robot untuk memanipulasi objek secara fisik dari awal hingga akhir.

Robot harus mendekati objek, menggenggam objek, mempertahankan kestabilan objek selama perpindahan, menyelaraskan objek terhadap target, dan melepaskan objek pada posisi yang sesuai. Kedua, penelitian ini menggunakan tiga bentuk objek yang berbeda, yaitu kubus, silinder, dan prisma segitiga. Variasi bentuk tersebut membuat *policy* harus menghadapi karakteristik geometri yang berbeda pada setiap objek.

Perbedaan bentuk objek menjadi salah satu faktor penting yang memengaruhi tingkat keberhasilan pada pengujian simulasi. Objek kubus memperoleh tingkat keberhasilan tertinggi karena memiliki bidang permukaan yang lebih stabil untuk digenggam dan lebih mudah diprediksi ketika dipindahkan maupun dilepaskan. Sebaliknya, objek silinder memiliki permukaan lengkung yang dapat menyebabkan objek lebih mudah bergeser atau berotasi selama proses *grasping*. Objek prisma segitiga juga memberikan tantangan tersendiri karena orientasi sisi objek terhadap *gripper* dan target lebih sulit dipertahankan secara konsisten. Dengan demikian, perbedaan *success rate* antarobjek menunjukkan bahwa *policy* masih sensitif terhadap karakteristik geometri objek, terutama pada tahap *grasping* dan *release*.

Selain bentuk objek, toleransi lubang juga menjadi faktor yang sangat memengaruhi tingkat keberhasilan. Pada penelitian ini, lubang target dirancang dengan toleransi yang relatif kecil, yaitu hanya 5 mm terhadap dimensi objek. Dengan toleransi tersebut, objek tidak cukup hanya dibawa mendekati area target, tetapi juga harus memiliki posisi dan orientasi yang presisi agar dapat masuk ke dalam lubang. Kondisi ini membuat tugas penyisipan menjadi lebih sulit, terutama ketika objek mengalami sedikit pergeseran saat proses *grasping* atau *insert*. Kesalahan kecil pada posisi akhir objek dapat menyebabkan episode gagal meskipun *trajectory end-effector* sudah terlihat mendekati target. Oleh karena itu, *success rate* pada penelitian ini perlu dipahami dalam konteks tugas yang memiliki batas keberhasilan cukup ketat.

Toleransi lubang yang kecil juga menjelaskan mengapa tugas pada penelitian ini lebih sensitif terhadap *error* dibandingkan tugas manipulasi yang hanya menilai pencapaian pose *end-effector* pada penelitain yang dilakukan oleh

Xia dkk (2023). Pada tugas *trajectory planning*, keberhasilan umumnya ditentukan oleh kemampuan end-effector mencapai posisi dan orientasi target. Sementara itu, pada penelitian ini, keberhasilan ditentukan oleh kondisi akhir objek yang harus sesuai dengan lubang target. Artinya, selain harus mengendalikan *end-effector*, *policy* juga harus menghasilkan interaksi yang stabil antara *gripper* dan objek. Apabila objek sedikit bergeser, berotasi, atau tidak terlepas tepat di atas lubang, maka episode dapat dinyatakan gagal. Dengan demikian, toleransi 5 mm memberikan batas keberhasilan yang lebih ketat dan membuat tugas penyisipan objek menjadi lebih sensitif terhadap *error* posisi, *error* orientasi, dan kestabilan *grasp*.

Hasil penelitian ini juga dapat dibandingkan dengan penelitian Xu dkk. (2025) yang mengusulkan *Separate Primitive Policy* untuk tugas *peg-in-hole* berbasis *visual reinforcement learning*. Pada penelitian tersebut, proses *peg-in-hole* dipisahkan menjadi *location action* dan *insertion action*, sehingga agen dapat mempelajari dua *primitive* utama secara lebih terarah.

Pendekatan tersebut terbukti mampu meningkatkan *sample efficiency* dan *success rate* pada berbagai bentuk *peg-in-hole*. Secara konsep, penelitian ini memiliki kemiripan karena sama-sama memanfaatkan pemisahan perilaku dasar untuk membantu penyelesaian tugas utama. Namun, terdapat perbedaan mendasar pada cara *primitive* digunakan. Pada penelitian Xu dkk., *primitive location* dan *insertion* dijalankan secara berurutan sebagai bagian dari struktur *policy*. Pada penelitian ini, *grasp policy* dan *insert policy* digunakan sebagai sumber kandidat aksi selama pelatihan *target policy*, sedangkan *policy* akhir tetap berupa satu *target policy*. Selain itu, pada penelitian Xu dkk., *peg* sudah berada dalam kondisi tergenggam oleh *gripper* sejak awal, sedangkan penelitian ini juga mencakup proses mendekati dan menggenggam objek. Hal ini membuat ruang kegagalan pada penelitian ini menjadi lebih besar.

Jika dibandingkan dengan penelitian X. Liu dkk. (2025), perbedaan paling menonjol terdapat pada strategi generalisasi dan validasi *sim-to-real*. X. Liu dkk. melatih *policy peg-in-hole* menggunakan PPO pada ribuan lingkungan paralel dengan variasi pose lubang dalam enam derajat kebebasan. Hasilnya menunjukkan

keberhasilan hampir 100% pada simulasi dan validasi *sim-to-real* melalui eksekusi *trajectory* pada robot nyata.

Pada penelitian ini, *trajectory replay* digunakan dalam pengujian lingkungan nyata. Akan tetapi, hasil pengujian nyata menunjukkan penurunan performa yang cukup besar, yaitu *success rate* sebesar 0% pada kubus, 10% pada silinder, dan 0% pada prisma segitiga. Penurunan ini menunjukkan bahwa *trajectory* yang berhasil di simulasi belum tentu dapat langsung dieksekusi dengan keberhasilan yang sama pada robot fisik. Faktor utama yang memengaruhi kondisi tersebut adalah *error* deteksi AprilTag, *error* transformasi koordinat dari kamera ke base robot, perbedaan kondisi fisik antara simulasi dan robot nyata, serta tidak adanya koreksi selama eksekusi. Ketika objek bergeser setelah proses *grasping* atau pose awal objek tidak sesuai dengan hasil estimasi, robot tetap mengikuti *trajectory* yang telah dihasilkan sebelumnya tanpa memperbarui aksi berdasarkan kondisi aktual.

Pengaruh toleransi lubang yang kecil menjadi semakin besar pada pengujian lingkungan nyata. *Error* deteksi AprilTag, *backlash* atau keterbatasan akurasi aktuator, serta perbedaan kondisi kontak antara simulasi dan robot nyata dapat menghasilkan penyimpangan dalam skala beberapa milimeter. Pada tugas dengan toleransi lubang hanya 5 mm, penyimpangan tersebut sudah cukup untuk menyebabkan objek tidak masuk ke target. Dengan demikian, rendahnya performa pada lingkungan nyata tidak hanya disebabkan oleh kegagalan *policy*, tetapi juga oleh ketatnya toleransi penyisipan yang membuat sistem sangat bergantung pada akurasi persepsi, kalibrasi koordinat, dan kestabilan eksekusi *trajectory*.

Hasil pengujian nyata pada penelitian ini lebih tepat dipahami sebagai evaluasi awal *sim-to-real*, bukan sebagai bukti bahwa *policy* sudah siap diterapkan secara langsung pada sistem fisik. Pada simulasi, posisi objek, orientasi objek, kondisi kontak, dan respons *gripper* dapat dimodelkan dengan lebih stabil. Sementara itu, pada lingkungan nyata terdapat banyak sumber ketidakpastian, seperti *noise* kamera, pencahayaan, ketidaktepatan estimasi pose, fleksibilitas mekanik *gripper*, keterbatasan akurasi aktuator, dan kemungkinan objek bergeser saat digenggam. Karena sistem belum menggunakan kontrol *closed-loop*, kesalahan

kecil pada tahap awal dapat menyebabkan kegagalan pada tahap akhir. Kondisi ini menjadi semakin kritis karena lubang target memiliki toleransi yang kecil terhadap ukuran objek.

Berdasarkan pembahasan tersebut, terdapat beberapa aspek yang perlu dikembangkan pada penelitian selanjutnya. Pertama, *domain randomization* perlu diperluas, tidak hanya pada posisi dan orientasi objek, tetapi juga pada parameter visual, posisi kamera, massa objek, friksi, *noise* sensor, dan dinamika aktuator. Kedua, sistem persepsi perlu ditingkatkan melalui kalibrasi kamera dan transformasi koordinat yang lebih akurat. Ketiga, pengujian nyata sebaiknya dikembangkan dari *trajectory replay* menjadi kontrol *closed-loop*, sehingga robot dapat memperbarui aksi berdasarkan kondisi aktual objek dan target selama eksekusi. Keempat, rancangan *gripper* dan strategi *grasping* perlu diperbaiki agar objek tidak mudah bergeser setelah digenggam. Dengan pengembangan tersebut, metode SAC berbasis QMP-HER diharapkan dapat menghasilkan performa yang lebih stabil tidak hanya pada lingkungan simulasi, tetapi juga pada implementasi robot nyata.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan penelitian yang telah dilakukan, kesimpulan yang diperoleh adalah sebagai berikut.

1. Algoritma *Soft Actor-Critic* berbasis *Q-Switch Mixture of Primitives Hindsight Experience Replay* berhasil diimplementasikan untuk sistem kendali robot manipulator pada tugas penyisipan objek dengan berbagai bentuk. Implementasi dilakukan melalui dua tahap utama, yaitu pelatihan *primitive policy* dan pelatihan *target policy*. *Primitive policy* yang digunakan terdiri atas *grasp policy* dan *insert policy*. *Grasp policy* berperan untuk mengarahkan *end-effector* menuju objek dan membentuk kondisi *grasping*, sedangkan *insert policy* berperan untuk membawa objek menuju posisi target dan menyesuaikan orientasi objek terhadap lubang target. Kedua *primitive policy* tersebut kemudian digunakan sebagai sumber kandidat aksi pada pelatihan *target policy* melalui mekanisme *Q-Switch*. *Target policy* dilatih menggunakan *sparse reward* dan *Hindsight Experience Replay*, sehingga mampu memanfaatkan pengalaman gagal melalui *goal relabeling*. Pada lingkungan nyata, implementasi dilakukan dengan pendekatan *trajectory replay*, yaitu pose objek dan target dari AprilTag dikirim ke simulasi MuJoCo untuk menghasilkan *trajectory joint*, kemudian *trajectory* tersebut dieksekusi pada robot myCobot.
2. Performa model SAC berbasis QMP-HER menunjukkan hasil yang cukup baik pada lingkungan simulasi, tetapi mengalami penurunan signifikan pada lingkungan nyata. Pada proses pelatihan, *target policy* mampu mencapai *success rate* sekitar 0,8–0,85 pada fase pelatihan *target policy* selama 2.000.000 *timestep*, dengan total proses pelatihan hingga diperoleh *target policy final* sebesar 5.500.000 *timestep*. Perbandingan dengan SAC standar dan SAC+HER menunjukkan bahwa QMP-HER memiliki efisiensi sampel yang lebih baik, karena SAC standar dan SAC+HER belum menunjukkan peningkatan performa yang signifikan hingga 6.000.000 *timestep*. Pada

pengujian simulasi, *target policy* memperoleh *success rate* sebesar 84% pada objek kubus, 74% pada objek silinder, dan 74% pada objek prisma segitiga. Namun, pada pengujian lingkungan nyata, *success rate* menurun menjadi 0% pada objek kubus, 10% pada objek silinder, dan 0% pada objek prisma segitiga. Dari 10 episode pengujian pada setiap objek, kubus berhasil digenggam sebanyak 7 kali tetapi tidak menghasilkan episode sukses, silinder berhasil digenggam sebanyak 5 kali dan menghasilkan 1 episode sukses, sedangkan prisma segitiga berhasil digenggam sebanyak 3 kali tanpa episode sukses. Penurunan performa ini menunjukkan adanya *sim-to-real gap* yang dipengaruhi oleh *error* deteksi AprilTag, ketidakstabilan *grasp*, perbedaan kondisi fisik robot nyata, serta pendekatan *trajectory replay* yang belum melakukan koreksi secara *real-time*.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan penelitian selanjutnya.

1. Hasil pengujian menunjukkan bahwa terdapat perbedaan performa antara lingkungan simulasi dan lingkungan nyata. Salah satu faktor yang dapat memengaruhi perbedaan tersebut adalah interaksi kontak antara *gripper*, objek, dan lubang target yang belum sepenuhnya merepresentasikan kondisi fisik sebenarnya. Oleh karena itu, penelitian selanjutnya dapat mengembangkan pemodelan kontak manipulasi secara lebih rinci, seperti parameter friksi, gaya tekan *gripper*, karakteristik permukaan objek, massa objek, serta bentuk geometri objek agar interaksi antara robot dan objek pada simulasi lebih mendekati kondisi nyata.
2. Sistem persepsi dan kalibrasi koordinat pada lingkungan nyata perlu ditingkatkan. Hal ini penting karena *error* deteksi posisi menggunakan AprilTag masih berada pada kisaran 0,5 cm hingga 1,5 cm, sedangkan tugas penyisipan objek membutuhkan presisi pada orde milimeter. Penelitian selanjutnya dapat menggunakan kamera dengan resolusi lebih baik, pencahayaan yang lebih stabil, model *computer vision* yang lebih akurat, atau lebih dari satu kamera untuk meningkatkan akurasi estimasi pose.

3. Pendekatan *trajectory replay* yang digunakan pada penelitian ini masih bersifat *open-loop*, sehingga robot belum dapat melakukan koreksi apabila terjadi pergeseran objek, ketidakstabilan *grasp*, atau perbedaan eksekusi antara simulasi dan robot nyata. Kondisi ini dapat menyebabkan *trajectory* yang berhasil pada simulasi tidak selalu berhasil saat dijalankan pada robot fisik. Oleh karena itu, penelitian selanjutnya disarankan mengembangkan sistem kendali *closed-loop* agar robot dapat memperbarui aksi berdasarkan kondisi aktual objek dan target selama proses eksekusi.
4. Penelitian ini telah menerapkan *domain randomization* pada posisi dan orientasi awal pada objek dan lubang target, juga parameter gesek pada *gripper*, tetapi variasi yang digunakan masih terbatas. Padahal, penurunan performa pada lingkungan nyata juga dapat dipengaruhi oleh perbedaan parameter fisik dan visual yang belum dimodelkan selama pelatihan. Oleh karena itu, penelitian selanjutnya dapat memperluas *domain randomization* pada massa objek, parameter aktuator, *noise* sensor, posisi kamera, pencahayaan, *delay* komunikasi, serta karakteristik kontak lainnya. Selain itu, pendekatan *domain adaptation* atau *fine-tuning policy* pada robot nyata juga dapat dipertimbangkan agar *policy* lebih mampu beradaptasi terhadap ketidakpastian pada lingkungan fisik.
5. Hasil pengujian nyata menunjukkan bahwa keberhasilan penyisipan objek sangat dipengaruhi oleh kestabilan *grasp*. Beberapa episode gagal bukan hanya karena *target policy* tidak mampu menghasilkan *trajectory*, tetapi juga karena objek bergeser atau tidak tergenggam dengan stabil selama eksekusi. Oleh karena itu, penelitian selanjutnya dapat meningkatkan mekanisme *grasping* melalui perbaikan desain *gripper*, penggunaan material permukaan yang lebih stabil, penambahan sensor gaya atau tekanan, serta pengembangan strategi *grasping* yang lebih adaptif terhadap bentuk objek.

DAFTAR PUSTAKA

Almahamid, F., & Grolinger, K. (2021). Reinforcement Learning Algorithms: An Overview and Classification. *Canadian Conference on Electrical and Computer Engineering*, *2021-September*. https://doi.org/10.1109/CCECE53047.2021.9569056

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., & Zaremba, W. (2018). *Hindsight Experience Replay*. http://arxiv.org/abs/1707.01495

Automatic Addison. (2021). *Robot State Publisher vs. Joint State Publisher*. Diakses pada 20 Oktober 2025, dari https://automaticaddison.com/robot-state-publisher-vs-joint-state-publisher/

Avhad, P., Kumar, G. P., Sunil, A. T., Rallapalli, M., Kumar, B. H., & Kumar, V. (2024). Adaptive Deep Reinforcement Learning for Robotic Manipulation in Dynamic Environments. *2nd IEEE International Conference on Data Science and Network Security, ICDSNS 2024*. https://doi.org/10.1109/ICDSNS62112.2024.10690981

Cai, Z., Ding, Z., & Zhu, M. (2025). Optimal Maintenance Scheduling of Transmission Systems with a Reinforcement Learning Approach. *2025 IEEE International Conference on Electrical Energy Conversion Systems and Control, IEECSC 2025*, 337–343. https://doi.org/10.1109/IEECSC64206.2025.11099913

Calderon-Cordova, C., Sarango, R., Castillo, D., & Lakshminarayanan, V. (2024). A Deep Reinforcement Learning Framework for Control of Robotic Manipulators in Simulated Environments. *IEEE Access*, *12*(May), 103133–103161. https://doi.org/10.1109/ACCESS.2024.3432741

Chen, X., Hu, J., Jin, C., Li, L., & Wang, L. (2021). Understanding Domain Randomization for Sim-to-real Transfer. *ICLR 2022 - 10th International Conference on Learning Representations*. https://arxiv.org/pdf/2110.03239

Crane, C. D., & Duffy, Joseph. (2008). *Kinematic analysis of robot manipulators*. Cambridge University Press. https://doi.org/https://doi.org/10.1017/CBO9780511530159

Elephant Robotics. (2023). *myCobot 280 pi(2023)*. Diakses pada 20 Oktober 2025, dari https://docs.elephantrobotics.com/docs/mycobot_280_pi_en/

Ghasemi, M., Moosavi, A. H., & Ebrahimi, D. (2025). *A Comprehensive Survey of Reinforcement Learning: From Algorithms to Practical Challenges*. http://arxiv.org/abs/2411.18892

Genovese, K. (2024). Single-image camera calibration with model-free distortion correction. *Optics and Lasers in Engineering*. doi:10.1016/j.optlaseng.2024.108348

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. http://arxiv.org/abs/1801.01290

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., & Levine, S. (2019). *Soft Actor-Critic Algorithms and Applications*. http://arxiv.org/abs/1812.05905

Hagemann, A., Knorr, M., & Stiller, C. (2023). Deep geometry-aware camera self-calibration from video. *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE. https://doi.org/10.1109/ICCV51070.2023.00318

Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023). A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation. Dalam *Sensors* (Vol. 23, Nomor 7). MDPI. https://doi.org/10.3390/s23073762

Harib, M. (2023). *Deep Reinforcement Learning for Robust Control of 6-DOF Robotic Manipulators* [University of Western Ontario]. https://doi.org/10.22215/etd/2023-15476

He, X., & Lv, C. (2024). Robotic Control in Adversarial and Sparse Reward Environments: A Robust Goal-Conditioned Reinforcement Learning Approach. *IEEE Transactions on Artificial Intelligence*, *5*(1), 244–253. https://doi.org/10.1109/TAI.2023.3237665

Husakovic, A., Banjanović-Mehmedovic, L., Gurdić-Ribic, A., Prljaca, N., & Karabegovic, I. (2025). Reinforcement learning for robot manipulation tasks in human-robot collaboration using the CQL/SAC algorithms. *Advances in Production Engineering & Management*, *20*(1), 5–17. https://doi.org/10.14743/apem2025.1.523

Ibrahim, S., Mostafa, M., Jnadi, A., Salloum, H., & Osinenko, P. (2024). Comprehensive Overview of Reward Engineering and Shaping in Advancing

Reinforcement Learning Applications. *IEEE Access*, *12*, 175473–175500. https://doi.org/10.1109/ACCESS.2024.3504735

Kasaura, K., Miura, S., Kozuno, T., Yonetani, R., Hoshino, K., & Hosoe, Y. (2023). *Benchmarking Actor-Critic Deep Reinforcement Learning Algorithms for Robotics Control with Action Constraints*. https://doi.org/10.1109/LRA.2023.3284378

Kim, B., Kwon, G., Park, C., & Kwon, N. K. (2023). The Task Decomposition and Dedicated Reward-System-Based Reinforcement Learning Algorithm for Pick-and-Place. *Biomimetics*, *8*(2). https://doi.org/10.3390/biomimetics8020240

Li, Z., Huang, C., Deng, S., Qiu, W., & Gao, X. (2023). A soft actor-critic reinforcement learning algorithm for network intrusion detection. *Computers and Security*, *135*. https://doi.org/10.1016/j.cose.2023.103502

Linder, T., Yilmaz, K., Adrian, D. B., & Leibe, B. (2025). *Acquisition of high-quality images for camera calibration in robotics applications via speech prompts*. arXiv preprint arXiv:2504.11031. doi:10.48550/arXiv.2504.11031

Liu, R., Nageotte, F., Zanne, P., de Mathelin, M., & Dresp-Langley, B. (2021). Deep reinforcement learning for the control of robotic manipulation: A focussed mini-review. *Robotics*, *10*(1), 1–13. https://doi.org/10.3390/robotics10010022

Liu, S. (2024). An evaluation of DDPG, TD3, SAC, and PPO: Deep reinforcement learning algorithms for controlling continuous system. Dalam B. H. Ahmad (Ed.), *Proceedings of the 2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023)* (Vol. 180, hlm. 15–24). Atlantis Press. https://doi.org/10.2991/978-94-6463-370-2_3

Liu, W., Niu, H., Skilton, R., & Carrasco, J. (2023). Sim-to-real deep reinforcement learning with manipulators for pick-and-place. Dalam F. Iida, P. Maiolino, A. Abdulali, & M. Wang (Ed.), *Towards Autonomous Robotic Systems: 24th Annual Conference, TAROS 2023, Cambridge, UK, September 13–15, 2023, Proceedings* (Vol. 14136, hlm. 240–252). Springer, Cham. https://doi.org/10.1007/978-3-031-43360-3_20

Liu, X., Kramberger, A., & Bodenhagen, L. (2025). *A General Peg-in-Hole Assembly Policy Based on Domain Randomized Reinforcement Learning*. http://arxiv.org/abs/2504.04148

Marougkas, I., Ramesh, D. M., Doerr, J. H., Granados, E., Sivaramakrishnan, A., Boularias, A., & Bekris, K. E. (2025). *Integrating Model-based Control and RL for Sim2Real Transfer of Tight Insertion Policies*. http://arxiv.org/abs/2505.11858

Murray, R. M., Li, Z., & Sastry, S. S. (1994). *A mathematical introduction to robotic manipulation*. CRC Press. https://doi.org/10.1201/9781315136370

Naeem, M., Rizvi, S. T. H., & Coronato, A. (2020). A Gentle Introduction to Reinforcement Learning and its Application in Different Fields. *IEEE Access*, *8*, 209320–209344. https://doi.org/10.1109/ACCESS.2020.3038605

NVIDIA Corporation. (2025). *Isaac ROS AprilTag — Isaac ROS*. Diakses pada 1 November 2025, dari https://nvidia-isaac-ros.github.io/repositories_and_packages/isaac_ros_apriltag/index.html

Olson, E. (2011). AprilTag: A robust and flexible visual fiducial system. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA 2011)*, 3400–3407. https://doi.org/10.1109/ICRA.2011.5979561

OpenCV. (2016). *Camera calibration*. OpenCV-Python Tutorials. Diakses pada 7 Juni 2026, dari https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_calib3d/py_calibration/py_calibration.html

Prakash, R., Gupta, K. V., & Behera, L. (2020). Adaptive Backstepping Neural Tracking Control of an Uncertain Robot Manipulator with Dynamic Disturbances. *2020 IEEE Symposium Series on Computational Intelligence, SSCI 2020*, 1936–1943. https://doi.org/10.1109/SSCI47803.2020.9308160

Rawat, D., Gupta, M. K., & Sharma, A. (2023). Intelligent control of robotic manipulators: a comprehensive review. Dalam *Spatial Information Research* (Vol. 31, Nomor 3, hlm. 345–357). Springer Science and Business Media B.V. https://doi.org/10.1007/s41324-022-00500-2

Roldan Sanchez, F. (2025). *Sample and time efficient strategies for off-policy reinforcement learning in robotic manipulation tasks* [Dublin City University]. https://doras.dcu.ie/30641/

Sanchez, F. R., Wang, Q., Bulens, D. C., McGuinness, K., Redmond, S., & O'Connor, N. (2023). *Learning and reusing primitive behaviours to improve*

Hindsight Experience Replay sample efficiency. http://arxiv.org/abs/2310.01827

Sewak, M. (2019). *Introduction to Reinforcement Learning. In: Deep Reinforcement Learning*. Springer Nature Singapore. https://doi.org/10.1007/978-981-13-8285-7

Shakerimov, A., Alizadeh, T., & Varol, H. A. (2023). Efficient Sim-to-Real Transfer in Reinforcement Learning Through Domain Randomization and Domain Adaptation. *IEEE Access*, *11*, 136809–136824. https://doi.org/10.1109/ACCESS.2023.3339568

Shao, Y., Zhou, H., Zhao, S., Fan, X., & Jiang, J. (2023). A Control Method of Robotic Arm Based on Improved Deep Deterministic Policy Gradient. *2023 IEEE International Conference on Mechatronics and Automation, ICMA 2023*, 473–478. https://doi.org/10.1109/ICMA57826.2023.10215662

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html

Song, C., Shin, J., Jeon, M.-H., Lim, J., & Kim, A. (2024). Unbiased estimator for distorted conics in camera calibration. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE. https://doi.org/10.1109/CVPR52733.2024.00043

Tang, J., Yuan, X., & Li, S. (2025). Visual–Tactile Fusion and SAC-Based Learning for Robot Peg-in-Hole Assembly in Uncertain Environments. *Machines*, *13*(7). https://doi.org/10.3390/machines13070605

Ulrich, M., Steger, C., Butsch, F., & Liebe, M. (2024). Vision-guided robot calibration using photogrammetric methods. *ISPRS Journal of Photogrammetry and Remote Sensing, 218*, 645–662. doi:10.1016/j.isprsjprs.2024.09.037

DeepMind Technologies Limited. (2023). *Model gallery*. MuJoCo Documentation. Diakses pada 19 Mei 2026, dari https://mujoco.readthedocs.io/en/stable/models.html

Wang, D., & Snooks, R. (2021). Artificial Intuitions of Generative Design: An Approach Based on Reinforcement Learning. *Proceedings of the 2020 DigitalFUTURES*, 189–198. https://doi.org/10.1007/978-981-33-4400-6_18

Wang, J., & Olson, E. (2016, Oktober 9). *AprilTag 2: Efficient and robust fiducial detection*. https://doi.org/10.1109/IROS.2016.7759617

Weiming, Q., Tianlin, L., & Dingsheng, L. (2023). *High-Precise Robot Arm Manipulation based on Online Iterative Learning and Forward Simulation with Positioning Error Below End-Effector Physical Minimum Displacement*. https://doi.org/10.48550/arXiv.2302.13338

Wong, C. C., Chien, S. Y., Feng, H. M., & Aoyama, H. (2021). Motion planning for dual-arm robot based on soft actor-critic. *IEEE Access*, *9*, 26871–26885. https://doi.org/10.1109/ACCESS.2021.3056903

Xia, W., Lu, Y., & Xu, X. (2023). Application and Evaluation of Soft-Actor Critic Reinforcement Learning in Constrained Trajectory Planning for 6DOF Robotic Manipulators. *2023 29th International Conference on Mechatronics and Machine Vision in Practice, M2VIP 2023*. https://doi.org/10.1109/M2VIP58386.2023.10413400

Xu, Z., Wang, Z., Li, Y., Zhuang, L., Zhao, Z., Yang, G., & Zhao, J. (2025). *Accelerating Visual Reinforcement Learning with Separate Primitive Policy for Peg-in-Hole Tasks*. https://arxiv.org/abs/2504.14820

Yang, R., Lyu, J., Yang, Y., Yan, J., Luo, F., Luo, D., Li, L., & Li, X. (2021). Bias-reduced Multi-step Hindsight Experience Replay for Efficient Multi-goal Reinforcement Learning. *Proceedings of Machine Learning Research*. https://arxiv.org/pdf/2102.12962

Yudha, H. M., Dewi, T., Risma, P., & Oktarina, Y. (2018). Arm robot manipulator design and control for trajectory tracking; A review. *International Conference on Electrical Engineering, Computer Science and Informatics (EECSI)*, *2018-Octob*, 304–309. https://doi.org/10.1109/EECSI.2018.8752950

Zhang, G., Jain, A., Hwang, I., Sun, S.-H., & Lim, J. J. (2023). *Efficient Multi-Task Reinforcement Learning via Selective Behavior Sharing*. http://arxiv.org/abs/2302.00671

Zhang, H., Kebria, P. M., Mohamed, S., Yu, S., & Nahavandi, S. (2023). *A Review on Robot Manipulation Methods in Human-Robot Interactions*. http://arxiv.org/abs/2309.04687

Zhao, W., Queralta, J. P., & Westerlund, T. (2020). Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: A Survey. *2020 IEEE Symposium Series*

on Computational Intelligence, SSCI 2020, 737–744. https://doi.org/10.1109/SSCI47803.2020.9308468

Zhou, P., Yin, H., Xu, G., Li, L., Yao, J., Li, J., Liu, C., & Shi, Z. (2024). Meta-Calib: A generic, robust and accurate camera calibration framework with ArUco-encoded meta-board. *ISPRS Journal of Photogrammetry and Remote Sensing, 212*, 357–380. doi:10.1016/j.isprsjprs.2024.05.005

LAMPIRAN

Lampiran 1. Hasil Pengujian *Target Policy* di Lingkungan Simulasi

Lampiran 1.1 Data Koordinat Posisi Awal Objek dan Lubang Target

episode	obj_x	obj_y	place_x	place_y	obj_yaw	place_yaw
1.00	0.26	-0.04	0.21	0.09	0.00	0.27
2.00	0.24	0.05	0.21	-0.01	-0.10	-0.48
3.00	0.22	-0.06	0.20	0.07	0.21	0.00
4.00	0.25	-0.05	0.23	0.06	-0.63	-0.58
5.00	0.23	0.05	0.23	-0.08	-0.09	-0.34
6.00	0.24	0.06	0.22	-0.05	-0.02	0.25
7.00	0.20	-0.09	0.24	0.08	-0.65	-0.45
8.00	0.22	-0.02	0.23	0.09	-0.68	-0.12
9.00	0.26	0.05	0.22	-0.07	-0.20	0.23
10.00	0.24	-0.03	0.20	0.08	0.10	-0.18
11.00	0.26	-0.04	0.21	0.07	-0.23	0.01
12.00	0.23	-0.07	0.25	0.07	-0.40	0.02
13.00	0.22	-0.09	0.22	0.02	-0.68	-0.24
14.00	0.26	-0.09	0.22	0.03	-0.36	0.54
15.00	0.24	0.05	0.25	-0.02	-0.56	-0.27
16.00	0.24	-0.09	0.24	0.05	-0.66	-0.23
17.00	0.20	0.00	0.24	-0.09	-0.73	-0.06
18.00	0.26	-0.08	0.23	0.08	-0.33	0.50
19.00	0.20	-0.09	0.20	0.08	0.64	0.02
20.00	0.22	-0.09	0.24	0.08	0.06	-0.04
21.00	0.20	0.05	0.21	-0.08	0.40	0.66
22.00	0.26	-0.04	0.22	0.09	0.30	-0.30
23.00	0.20	-0.09	0.24	0.00	-0.17	0.01
24.00	0.21	0.07	0.24	-0.05	-0.28	0.46
25.00	0.21	0.09	0.21	-0.01	0.37	0.21
26.00	0.21	0.03	0.20	-0.09	0.12	-0.71
27.00	0.21	0.07	0.23	-0.03	-0.55	-0.05
28.00	0.24	0.06	0.22	-0.08	-0.61	-0.20
29.00	0.20	0.03	0.25	-0.05	-0.32	-0.49
30.00	0.25	0.08	0.21	-0.09	0.21	-0.14
31.00	0.26	0.09	0.21	0.00	0.40	0.36
32.00	0.24	0.08	0.24	-0.05	-0.33	-0.32
33.00	0.24	-0.09	0.23	0.04	0.30	-0.25
34.00	0.21	0.04	0.22	-0.06	-0.61	-0.46
35.00	0.20	0.09	0.23	0.00	-0.75	-0.57
36.00	0.21	-0.01	0.23	-0.10	-0.13	-0.52

episode	obj_x	obj_y	place_x	place_y	obj_yaw	place_yaw
37.00	0.23	-0.05	0.23	0.08	-0.02	0.14
38.00	0.25	-0.09	0.23	0.07	-0.05	-0.36
39.00	0.22	0.00	0.24	-0.03	-0.18	-0.27
40.00	0.23	-0.09	0.23	0.05	-0.60	-0.30
41.00	0.21	-0.02	0.23	0.08	0.39	-0.11
42.00	0.26	0.04	0.24	-0.06	-0.11	-0.38
43.00	0.25	0.06	0.23	-0.05	-0.28	-0.45
44.00	0.20	-0.07	0.24	0.09	0.15	-0.02
45.00	0.22	-0.07	0.22	0.07	-0.30	0.70
46.00	0.21	0.03	0.20	-0.07	-0.48	-0.01
47.00	0.25	0.03	0.24	-0.07	0.19	-0.63
48.00	0.25	0.08	0.25	-0.08	-0.27	-0.09
49.00	0.25	0.09	0.24	-0.10	-0.56	-0.11
50.00	0.25	0.05	0.21	-0.09	0.65	-0.45

Lampiran 1.2 Hasil Pengujian pada Objek Kubus

episode	jarak akhir objek *place* (mm)	*steps*	*success*
1	7.47	249	1
2	3.43	207	1
3	5.42	208	1
4	6.27	195	1
5	34.35	250	0
6	5.19	210	1
7	11.23	203	0
8	6.07	199	1
9	3.21	210	1
10	5.90	206	1
11	3.15	202	1
12	6.57	202	1
13	5.48	199	1
14	4.48	207	1
15	2.94	200	1
16	5.49	204	1
17	6.71	202	1
18	6.84	206	1
19	2.22	223	1
20	11.71	216	0
21	312.43	205	0
22	2.47	213	1
23	10.88	195	0

episode	jarak akhir objek *place* (mm)	*steps*	*success*
24	5.88	207	1
25	196.93	250	0
26	2.19	209	1
27	11.89	207	0
28	4.45	207	1
29	5.81	201	1
30	4.23	216	1
31	4.74	208	1
32	2.31	229	1
33	4.15	217	1
34	3.07	202	1
35	3.60	202	1
36	2.48	202	1
37	3.03	205	1
38	5.41	218	1
39	3.23	197	1
40	4.22	205	1
41	5.92	199	1
42	3.93	216	1
43	3.70	217	1
44	27.19	213	0
45	6.70	210	1
46	3.24	206	1
47	5.04	213	1
48	5.50	218	1
49	2.33	217	1
50	2.37	246	1

Lampiran 1.3 Hasil Pengujian pada Objek Silinder

episode	jarak akhir objek *place* (mm)	*steps*	*success*
1	4.40	250	1
2	2.36	222	1
3	4.20	229	1
4	4.72	214	1
5	4.77	226	1
6	3.58	227	1
7	2.21	217	1
8	4.55	216	1

episode	**jarak akhir objek *place* (mm)**	***steps***	***success***
9	3.56	232	1
10	15.71	231	0
11	4.48	217	1
12	80.91	250	0
13	113.13	250	0
14	521.37	250	0
15	4.85	215	1
16	146.40	250	0
17	3.36	220	1
18	168.08	250	0
19	4.64	250	1
20	180.46	250	0
21	24.58	223	0
22	4.41	239	1
23	25.78	209	0
24	4.31	223	1
25	3.55	224	1
26	2.12	226	1
27	4.50	220	1
28	35.39	250	0
29	2.59	217	1
30	3.50	225	1
31	3.55	224	1
32	4.73	234	1
33	138.05	250	0
34	4.43	218	1
35	3.64	217	1
36	4.16	217	1
37	3.24	229	1
38	2.35	250	1
39	26.54	212	0
40	2372.23	250	0
41	2.94	217	1
42	3.75	224	1
43	2.85	227	1
44	2.45	237	1
45	4.23	229	1
46	3.29	223	1
47	2.77	229	1
48	2.46	219	1

episode	jarak akhir objek *place* (mm)	*steps*	*success*
49	5.21	230	1
50	3.25	228	1

Lampiran 1.4 Hasil Pengujian pada Objek Prisma Segitiga

episode	jarak akhir objek *place* (mm)	*steps*	*success*
1	6.12	250	1
2	3.57	220	1
3	8.22	228	0
4	4.75	211	1
5	11.85	227	0
6	2.79	228	1
7	11.20	219	0
8	7.59	212	1
9	3.77	233	1
10	163.23	250	0
11	7.11	219	1
12	8.21	218	0
13	2.79	217	1
14	6.86	219	1
15	5.09	214	1
16	5.90	218	1
17	3.52	217	1
18	4.93	229	1
19	12.80	250	0
20	5.25	241	1
21	6.76	216	1
22	6.49	233	1
23	6.46	211	1
24	7.99	227	1
25	3.55	220	1
26	2.36	226	1
27	2.88	219	1
28	10.69	238	0
29	8.83	215	0
30	6.78	223	1
31	4.34	222	1
32	4.19	238	1

episode	jarak akhir objek *place* (mm)	*steps*	*success*
33	2.39	239	1
34	2.39	218	1
35	6.82	216	1
36	4.98	217	1
37	3.47	229	1
38	3.06	233	1
39	86.32	250	0
40	12.85	217	0
41	6.20	223	1
42	3.54	223	1
43	3.39	243	1
44	5.79	235	1
45	6.28	226	1
46	12.08	219	0
47	926.60	250	0
48	3.60	220	1
49	7.41	228	1
50	150.98	250	0

Lampiran 2. Hasil Pengujian *Target Policy* di Lingkungan Nyata

Lampiran 2.1 Hasil Pengujian pada Objek Kubus

Episode	*Success*	Keterangan
1	0	*grasped*
2	0	*grasped*
3	0	*grasped*
4	0	*fail grasped*
5	0	*grasped*
6	0	*fail grasped*
7	0	*grasped*
8	0	*grasped*
9	0	*grasped*
10	0	*fail grasped*

Lampiran 2.2 Hasil Pengujian pada Objek Silinder

Episode	***Success***	**Keterangan**
1	0	*fail*
2	0	*grasped*
3	0	*grasped*
4	0	*fail*
5	0	*grasped*
6	0	*fail*
7	0	*fail*
8	1	*grasped*
9	0	*fail*
10	0	*grasped*

Lampiran 2.3 Hasil Pengujian pada Objek Prisma Segitiga

Episode	***Success***	**Keterangan**
1	0	*fail*
2	0	*fail*
3	0	*fail*
4	0	*grasped*
5	0	*fail*
6	0	*grasped*
7	0	*fail*
8	0	*fail*
9	0	*fail*
10	0	*grasped*